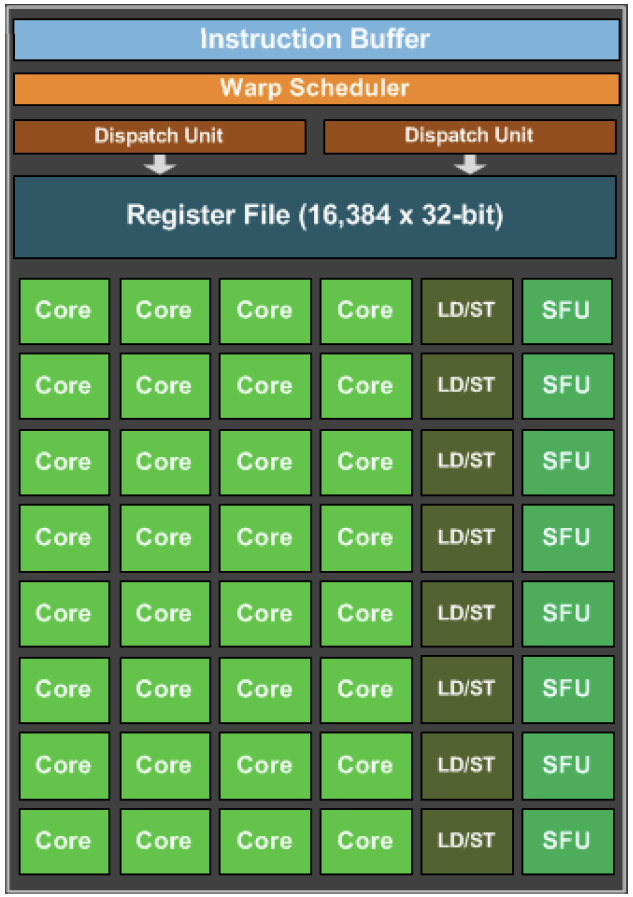
CSE113: Parallel Programming

- **Topics**:
  - Finish up GPUs
  - Homework 4
  - Start on memory models



Announcements

Grading

- Grades for HW 3 should be out by the end of the week

- HW 4 is released.

Kernel constraints

- What can and can’t you do in a GPU kernel?
  - Print?
  - File I/O?
  - C++ standard library? E.g. vectors?
  - Memory allocation?
  - Atomics?

Kernel constraints

- What can and can’t you do in a GPU kernel?
  - Print?
    CUDA now supports printfs directly in the kernel.
  - File I/O?
    You won't be able to implement file I/O directly in CUDA C++, as the GPU does not connect directly to the file system.
  - C++ standard library? E.g. vectors?
    You can't use the STL in CUDA, but you may be able to use the Thrust library to do what you want.
  - Memory allocation?
    In general allocating memory dynamically from the gpu is not possible in CUDA or other GPU APIs.
  - Atomics?
    Every Nvidia GPU that is a core 84 or higher supports CUDA 1.1 or higher and thus supports atomic operations.

The GPU Program

- If the same GPU also drives the display:
  The operating system expects the GPU kernel to be fast to render graphics.
  It kills the kernel if it takes more than ~5 seconds.

- A headless or second GPU has no limit.

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program



1000x slower on GPU?
It didn’t do so well...

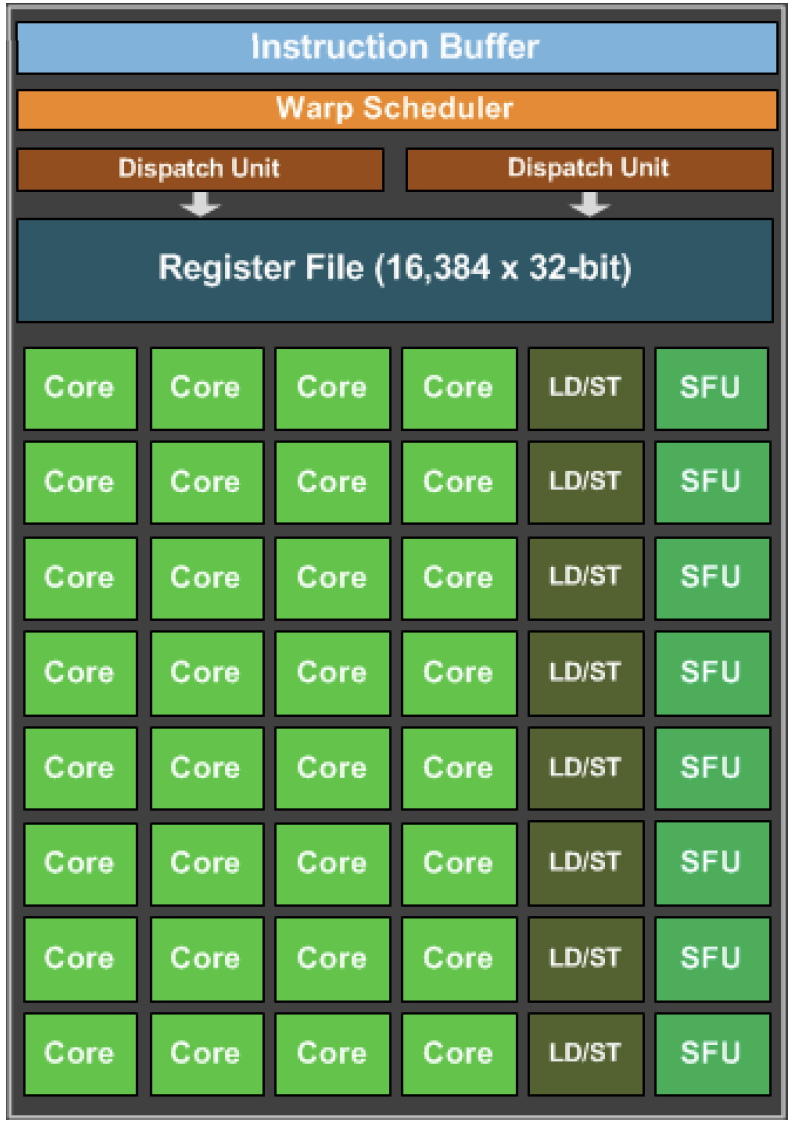
One GPU core is much slower than a single CPU core.

CPU spending silicon on caches, branch prediction and out-of-order logic,
GPU spending it on ALUs and a huge register file.

This saves energy and helps manage many cores.

First parallelization attempt

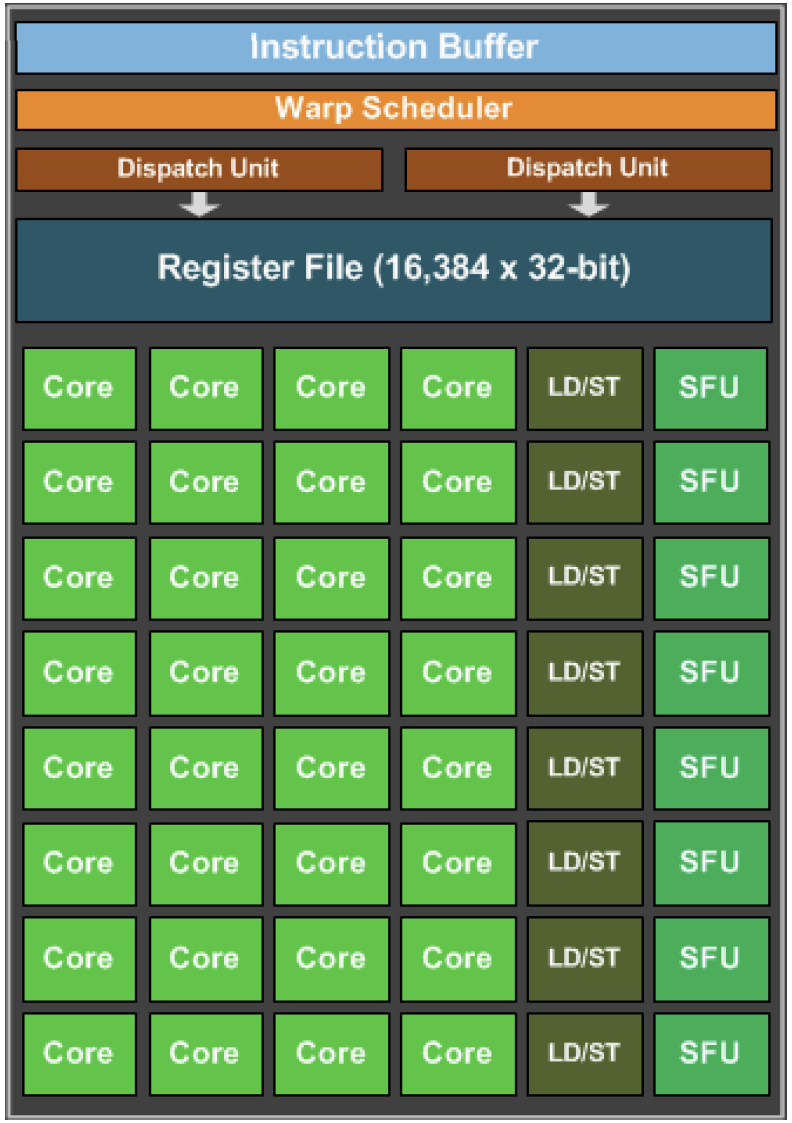
- Let’s look at some GPU documentation.
- The Maxwell whitepaper (Nvidia GPU microarchitecture, the 2014 generation) shows a diagram of one of the GPU cores



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

woah, 32 cores!

We should parallelize our application!



https://www.techpowerup.com/gpu-specs/docs/nvidia-gtx-980.pdf

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

number of threads to launch the program with

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

number of threads

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

number of threads
thread id

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

First parallelization attempt

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

When you write <<<1, 32>>> the integers are converted to dim3(1,1,1) and dim3(32,1,1), so only the x field is non-trivial and threadIdx.x is the only one you read.

First parallelization attempt

That gives us about 14x speedup!

First parallelization attempt



Getting better but we have a long ways to go!

Quiz 11.

Using a few sentences, give examples of workloads that benefit from GPU parallelism.

Free response: a few sentences on the form

Scan to answer

forms.gle/xcAa74cFCvfMcVhT7

chart: docs.google.com/forms/d/e/1FAIpQLSfafsP0Ovn_31uhRJgjO4RqzZL0ir9-cI74cTtjh88RP87VAQ/viewanalytics

Quiz 11.

Using a few sentences, give examples of workloads that benefit from GPU parallelism.

Free response: a few sentences on the form

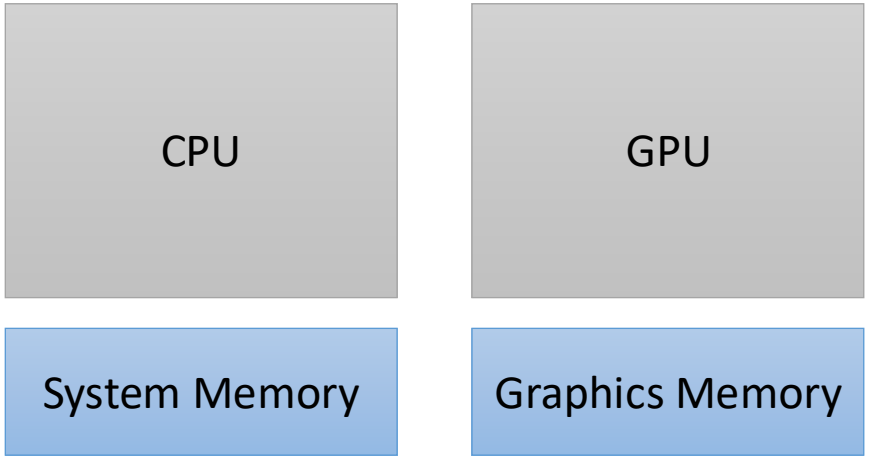
Data parallel.

Scan to answer

forms.gle/xcAa74cFCvfMcVhT7

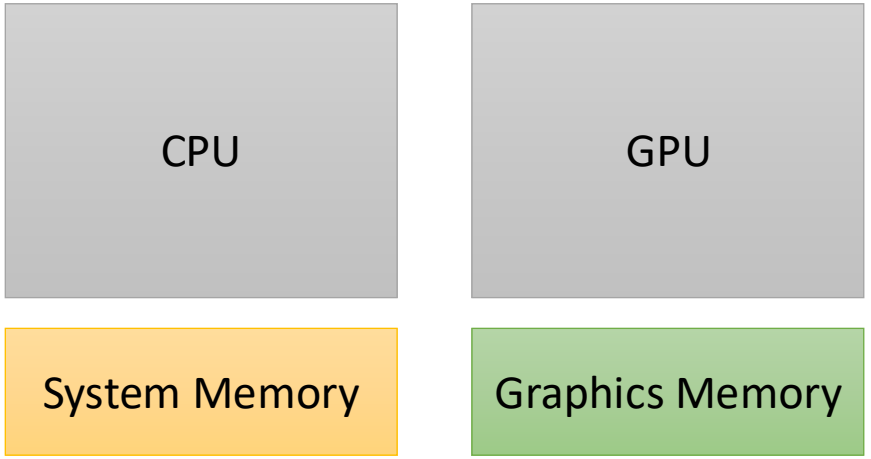
chart: docs.google.com/forms/d/e/1FAIpQLSfafsP0Ovn_31uhRJgjO4RqzZL0ir9-cI74cTtjh88RP87VAQ/viewanalytics

GPU Memory



GPU Memory

CPU Memory:
Fast: Low Latency
Easily saturated: Low Bandwidth
Scales well: up to 1 TB
DDR



GPU Memory:
Slow: High Latency
Hard to saturate: High Bandwidth
Doesn't scale: 80-192 GB
GDDR, HBM

2-lane straight highway driven on by sports cars

Different technologies

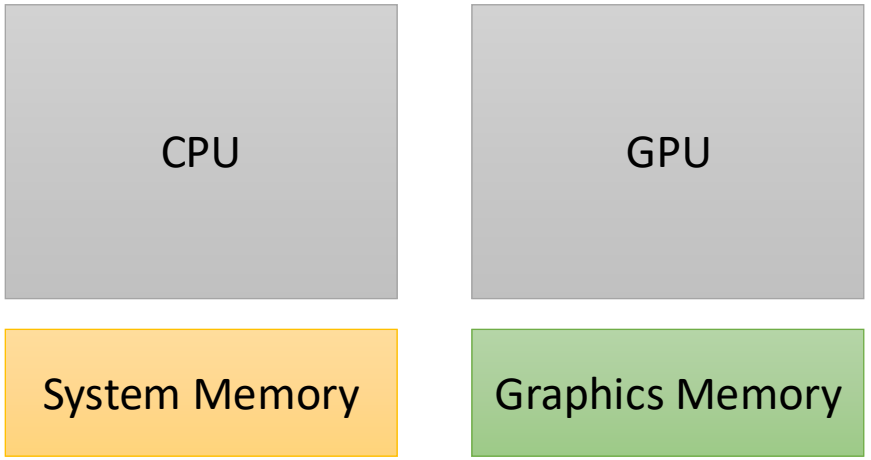
16-lane highway on a windy road driven by semi trucks

GPU Memory

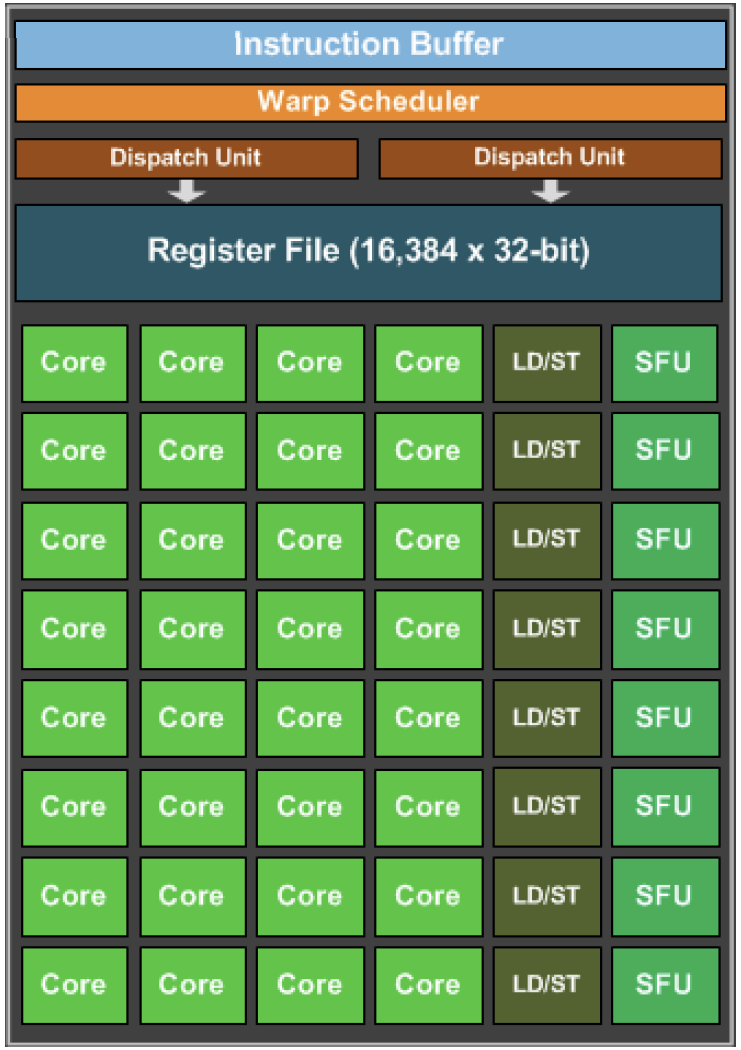
bandwidth:
~**1-3.3 TB/s** for GPU
~**300 GB/s** for CPUs

memory Latency:
~**600** cycles for GPU memory
~**200** cycles for CPU memory

Cache Latency:
~**28** cycles for L1 hit for GPU
~**4** cycles for L1 hit on CPUs



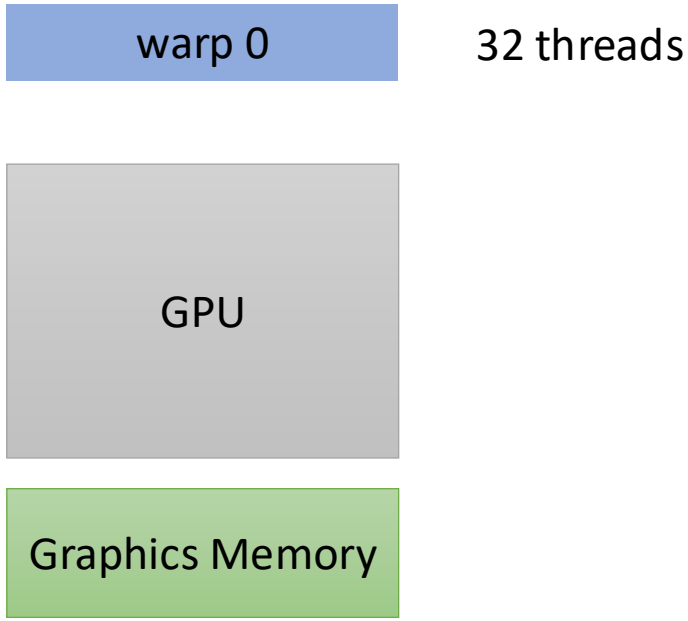
Warp scheduling and concurrency?



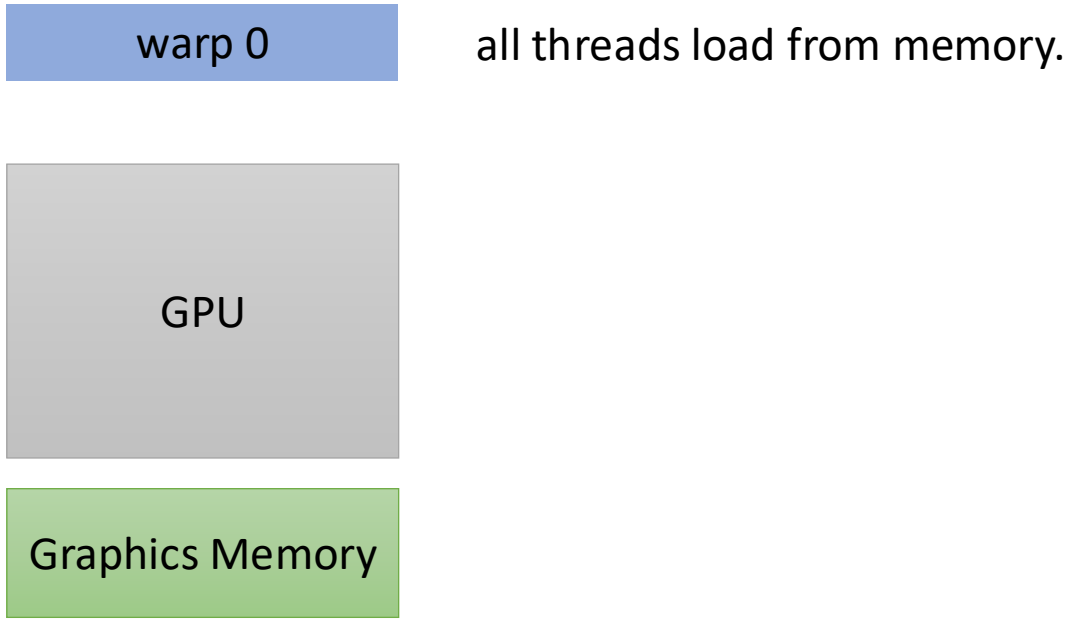
Blocks: multiple warps
Bound on number of warps: 32

Block = 32 warps * 32 blocks = 1024 threads max

Warp scheduling and concurrency?

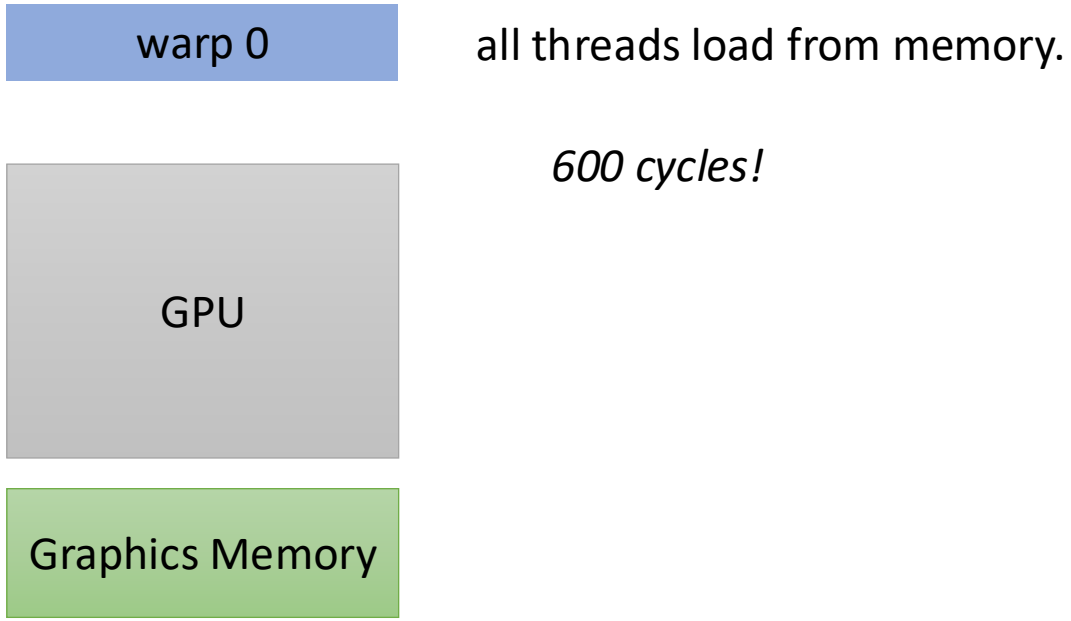


Warp scheduling and concurrency?



all threads load from memory.

Warp scheduling and concurrency?



all threads load from memory.

600 cycles!

Warp scheduling and concurrency?

warp 0

warp 1

warp 2

We can hide latency through scheduling and concurrency!

GPU

Graphics Memory



Warp scheduling and concurrency?

warp 1

warp 2

warp 0

GPU

Graphics Memory



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

Graphics Memory



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

warp 2

warp 0

GPU

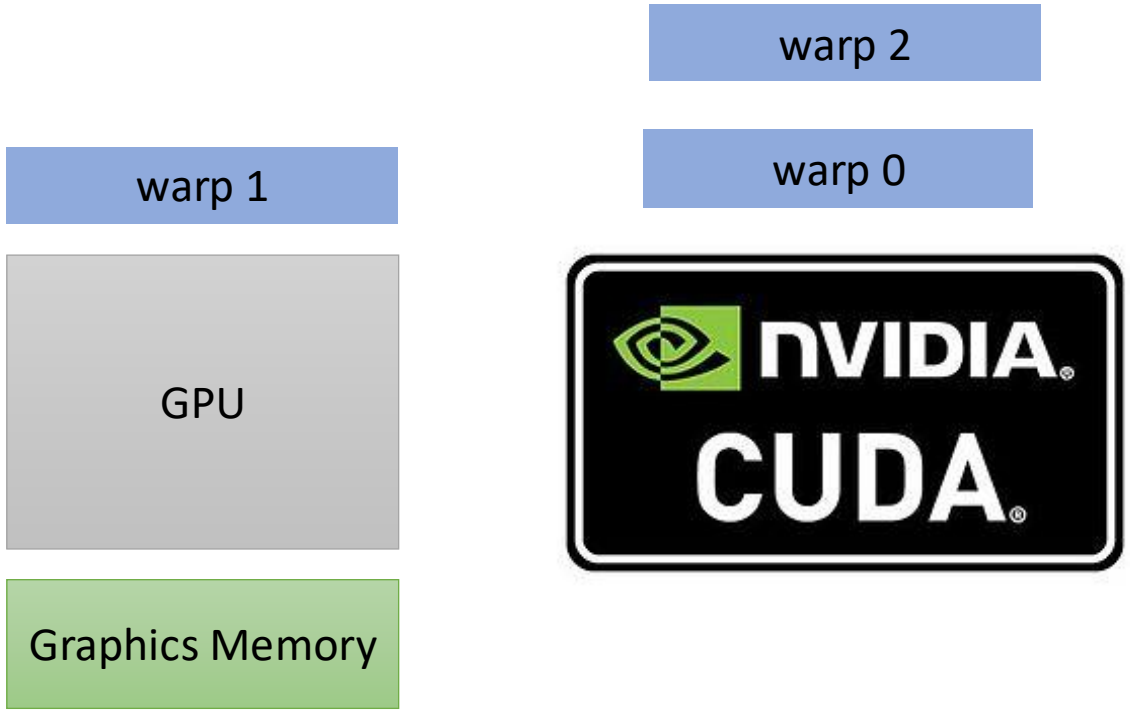
Graphics Memory



preempt warp 0
and put warp 1 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 1

GPU

Graphics Memory

warp 2

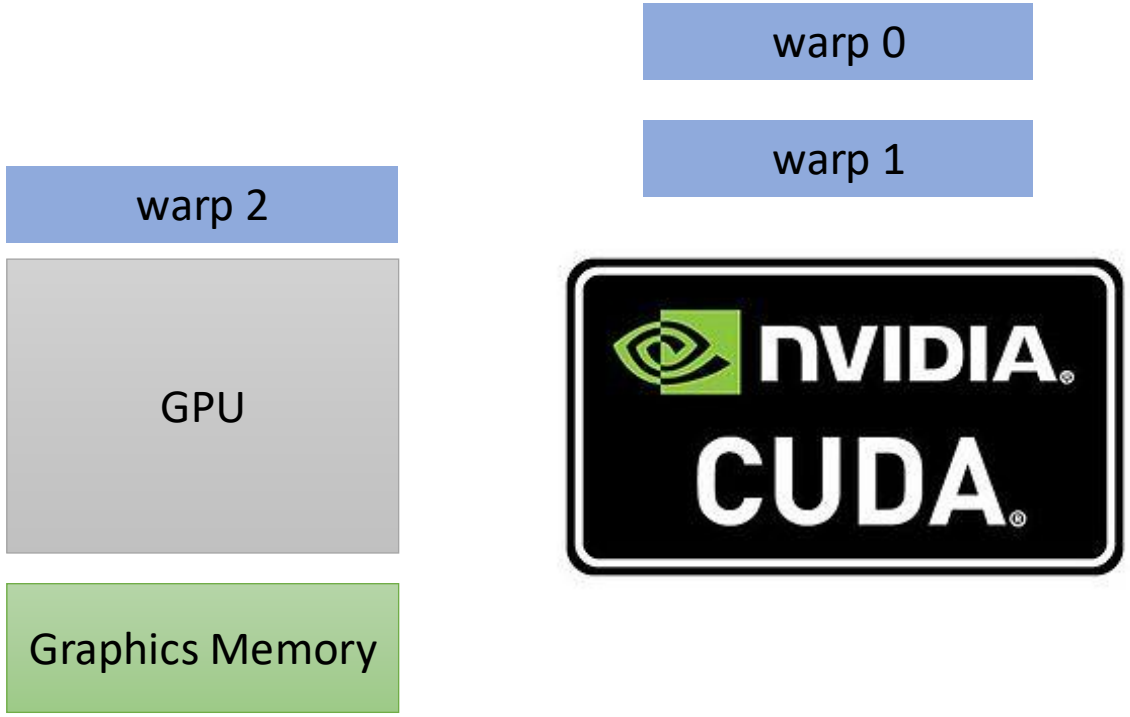
warp 0



preempt warp 1
and put warp 2 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?



We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

memory access
600 cycles

warp 2

GPU

Graphics Memory

warp 0

warp 1



preempt warp 2
and put warp 0 on

We can hide latency through scheduling and concurrency!

Warp scheduling and concurrency?

Hey, my memory has arrived!

warp 0

GPU

Graphics Memory

warp 1

warp 2



preempt warp 2
and put warp 0 on

We can hide latency through scheduling and concurrency!

Quiz 12.

What is a warp?

Select one

A. Group of 32 threads in a GPU

B. Group of 16 threads in a GPU

C. Group of 2 threads in a GPU

D. Group of some threads in a GPU



Scan to answer

forms.gle/vwqLu4z8L7qVbBdd9

chart: docs.google.com/forms/d/e/1FAIpQLSdtiYdVsGaa_pIzyv0wv39CZD-oXhojmrd3oNDMY8qagatO0g/viewanalytics

Quiz 12.

What is a warp?

Select one

A. Group of 32 threads in a GPU

B. Group of 16 threads in a GPU

C. Group of 2 threads in a GPU

D. Group of some threads in a GPU

Nvidia uses 32, AMD uses 64 or 32, Intel varies, and WebGPU calls the same thing a subgroup whose size you query at runtime.

Scan to answer

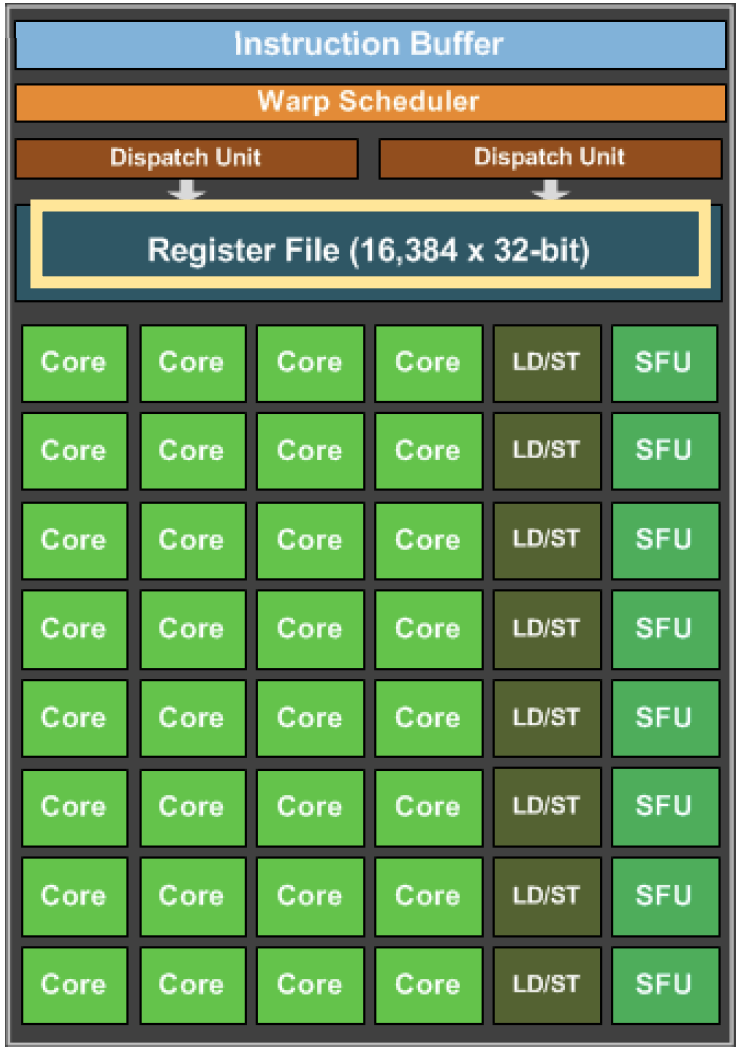
forms.gle/vwqLu4z8L7qVbBdd9

chart: docs.google.com/forms/d/e/1FAIpQLSdtiYdVsGaa_pIzyv0wv39CZD-oXhojmrd3oNDMY8qagatO0g/viewanalytics

Warp scheduling and concurrency?

But wait, I thought rescheduling was expensive?

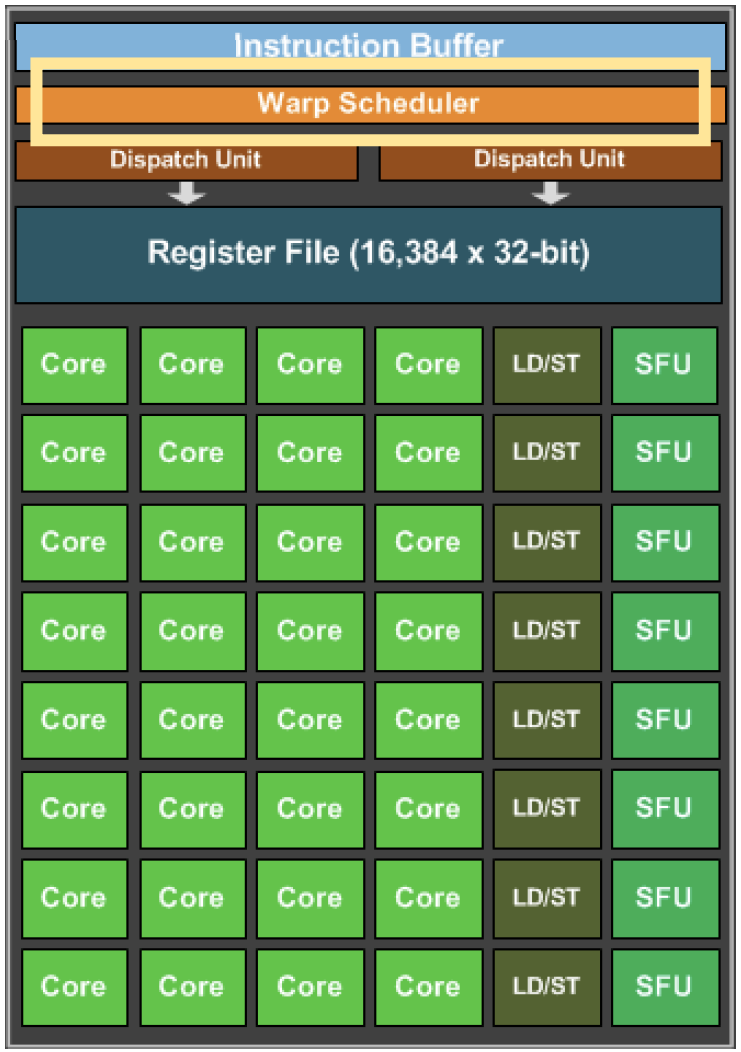
Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?

Registers all stay on chip. Large register file.

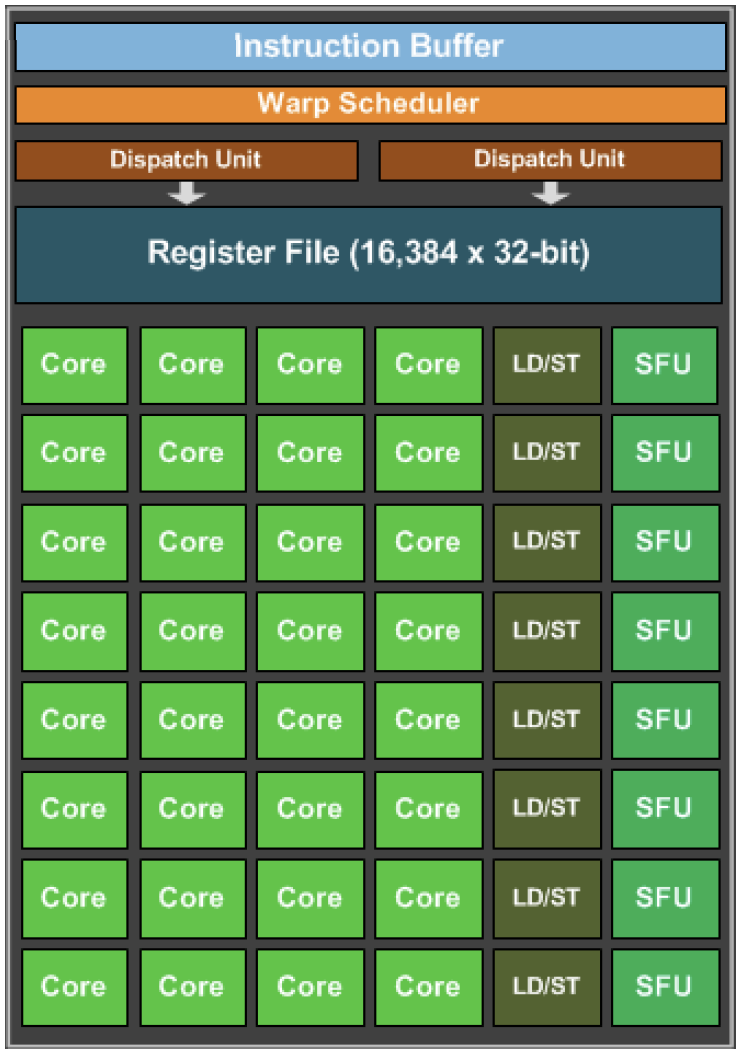
Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?

dedicated scheduler logic

Warp scheduling and concurrency?



But wait, I thought rescheduling was expensive?
Hardware multithreading, not OS-style preemption.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,32>>>(d_a, d_b, d_c, size);
```

Let's launch with 32 warps
32*32=1024

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Let's launch with 32 warps
32*32=1024

Concurrent warps

This is about 2x faster.

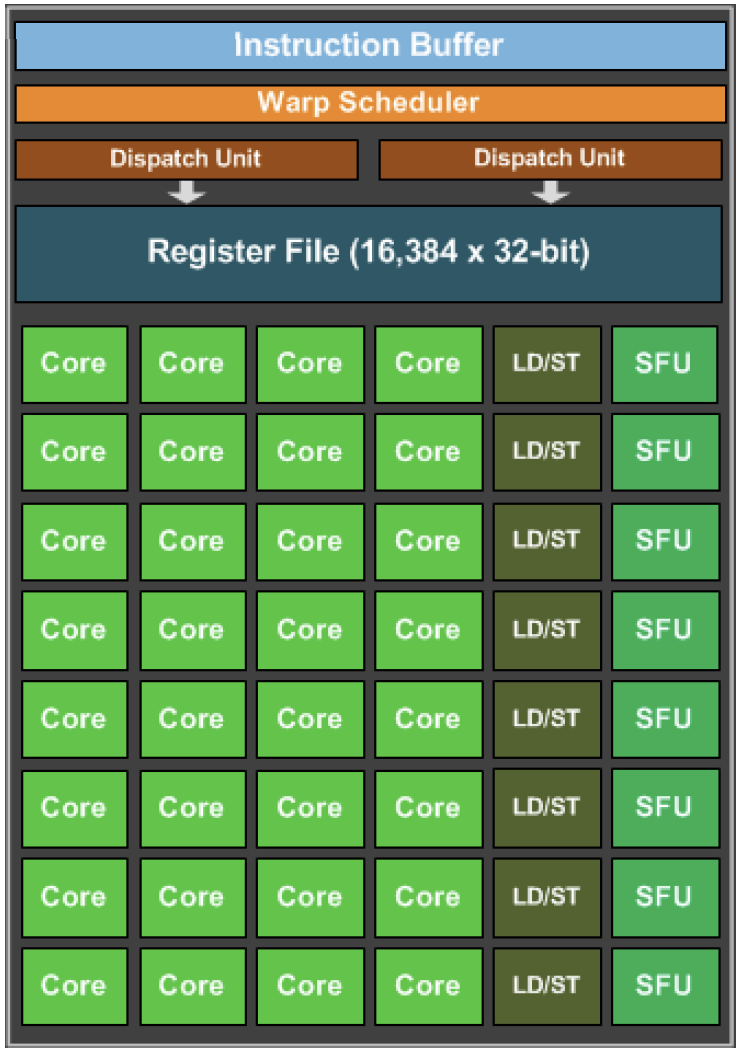
Concurrent warps

This is about 2x faster.

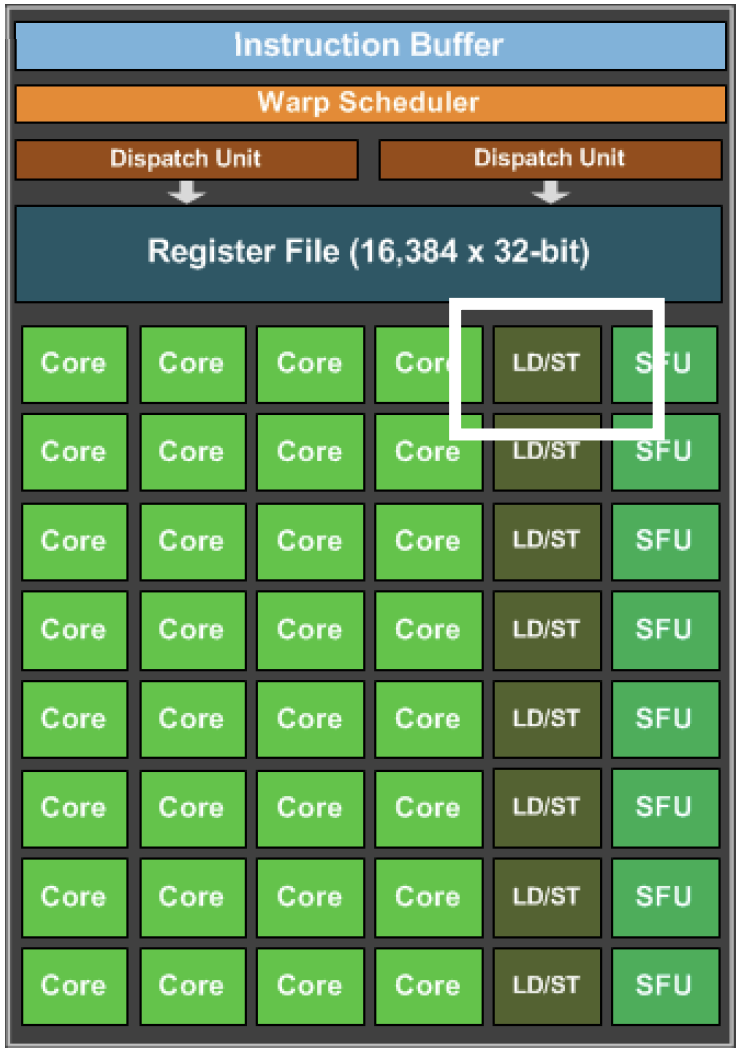


Getting better!

Optimizing memory accesses

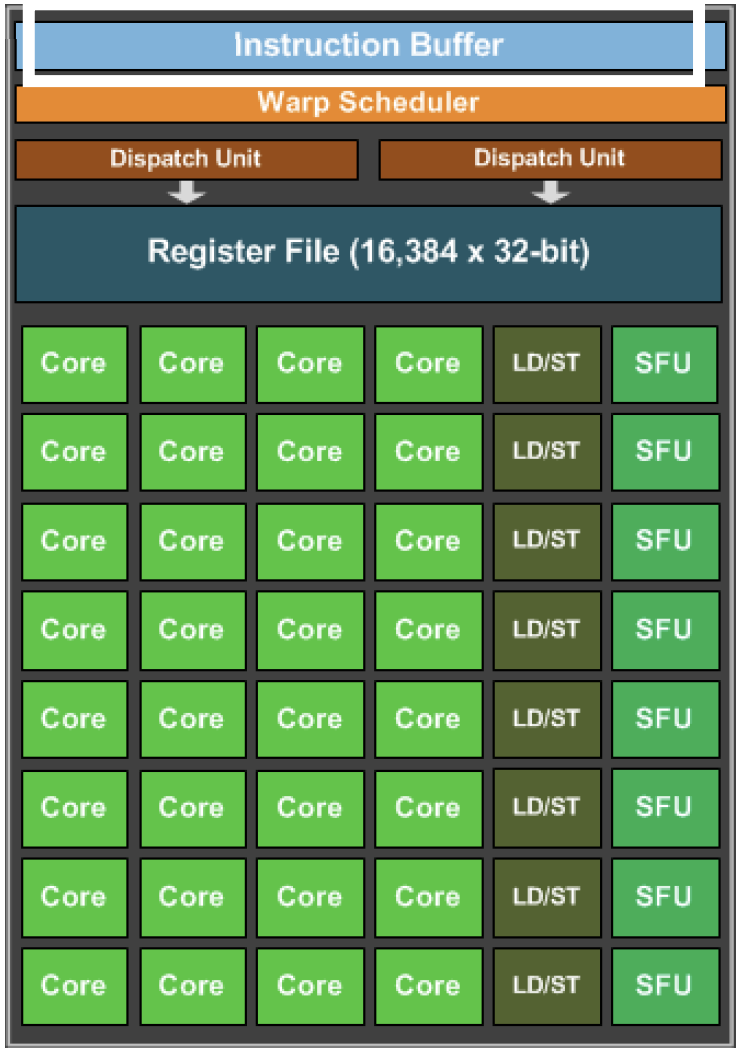


Optimizing memory accesses



This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

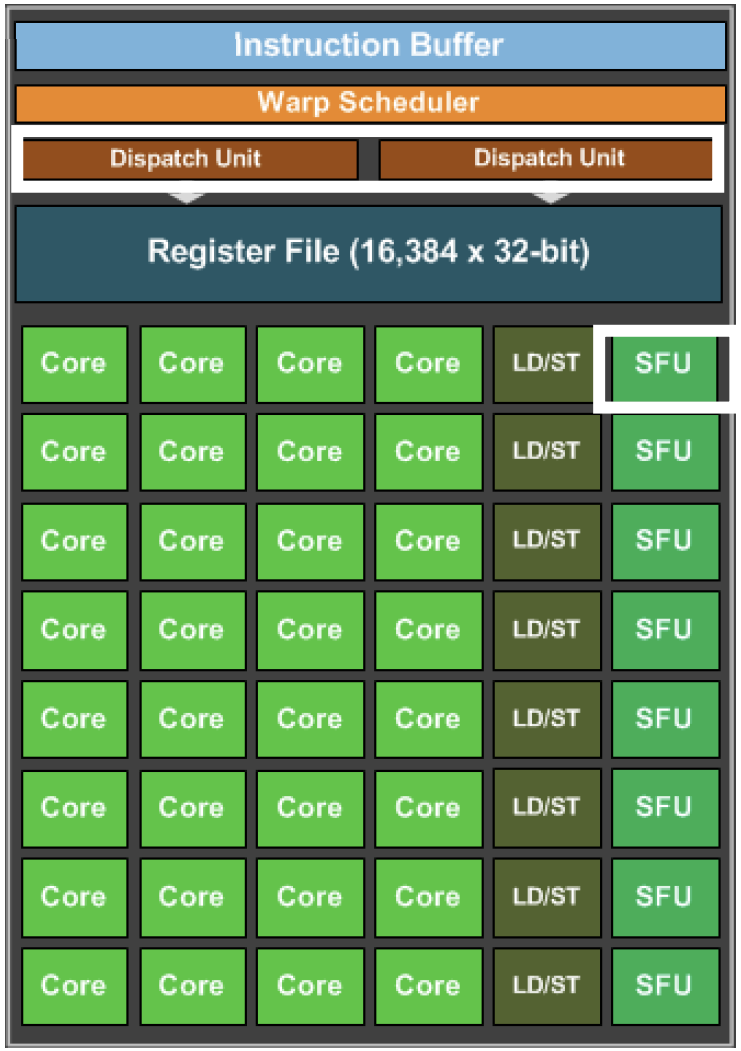
Optimizing memory accesses



This is the instruction cache
Shared between all cores of the warp.

This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

Optimizing memory accesses

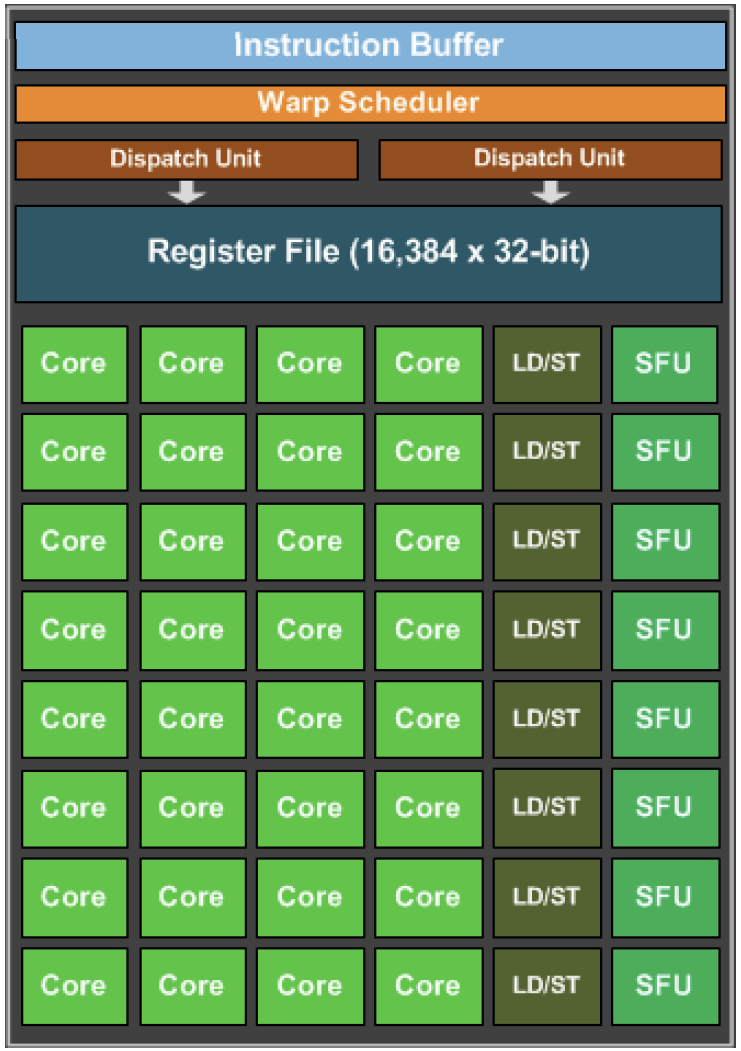


This is the instruction cache
Shared between all cores of the warp.
A dispatch unit issues instructions to a group of cores,
load/store units, or special function units (SFUs).

This is the load/store unit. The hardware component responsible for issuing loads and stores.
Shared between 4 cores.

SFU (The Special Function Unit) is an accelerator to perform tracendental and trigonometric operations such as sine and cosine.
These operations are not executed in parallel; an SFU is shared for those.

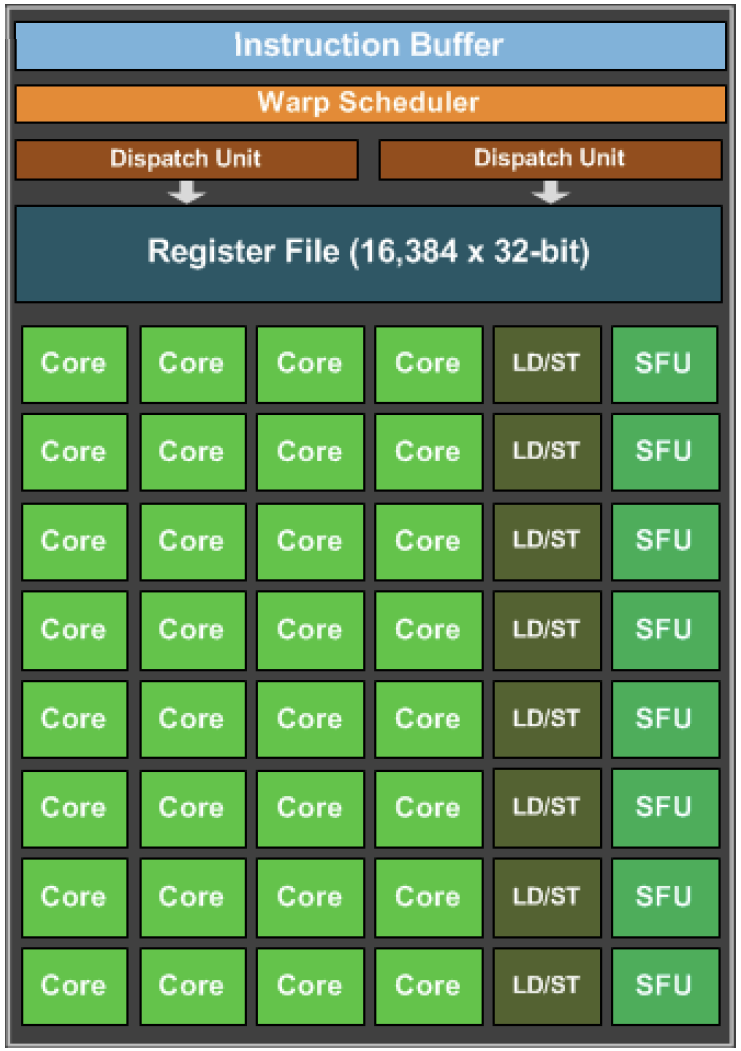
Warp execution



Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

Warp execution



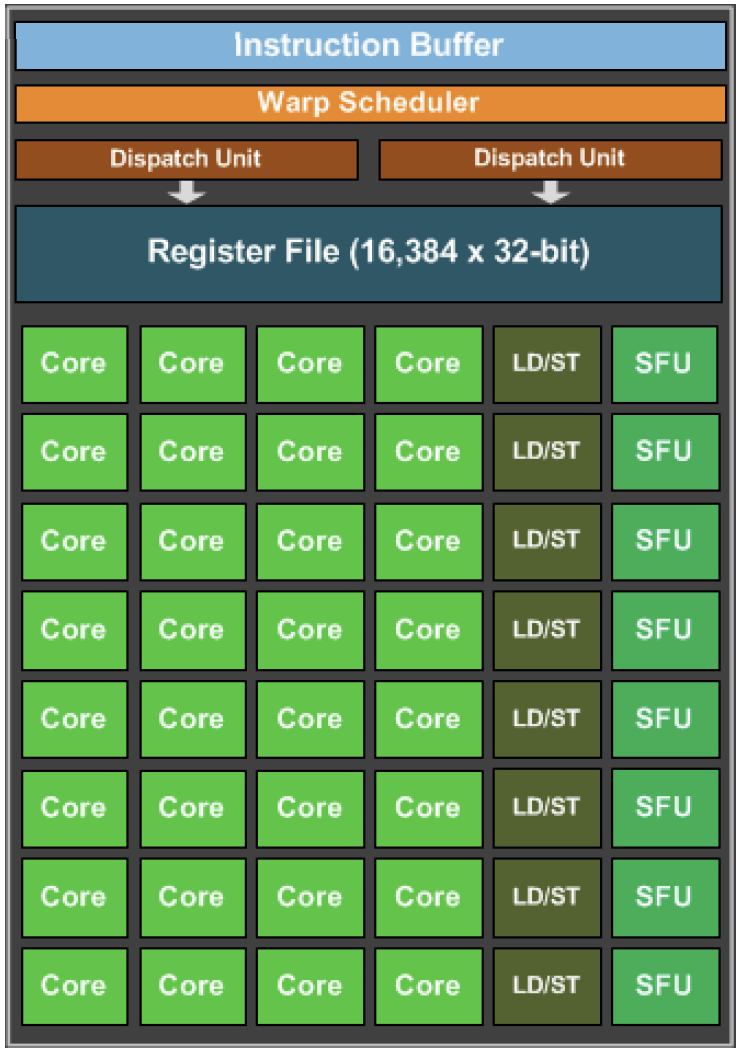
Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution



Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

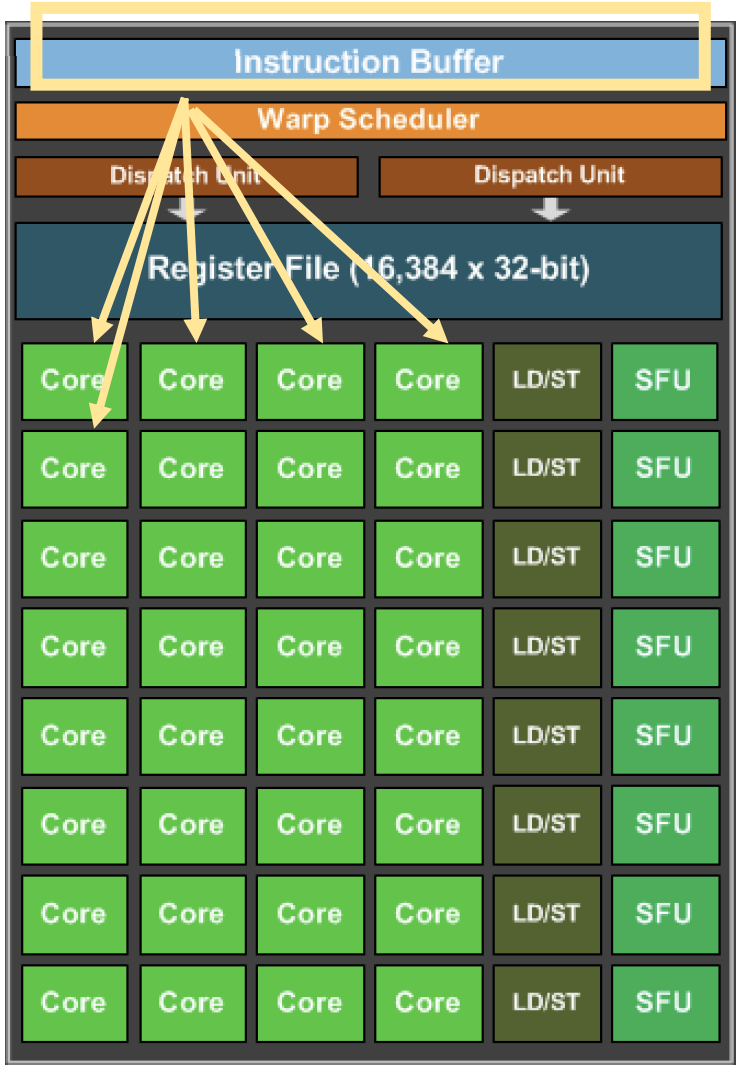
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



instruction is fetched from the buffer and distributed to all the cores.

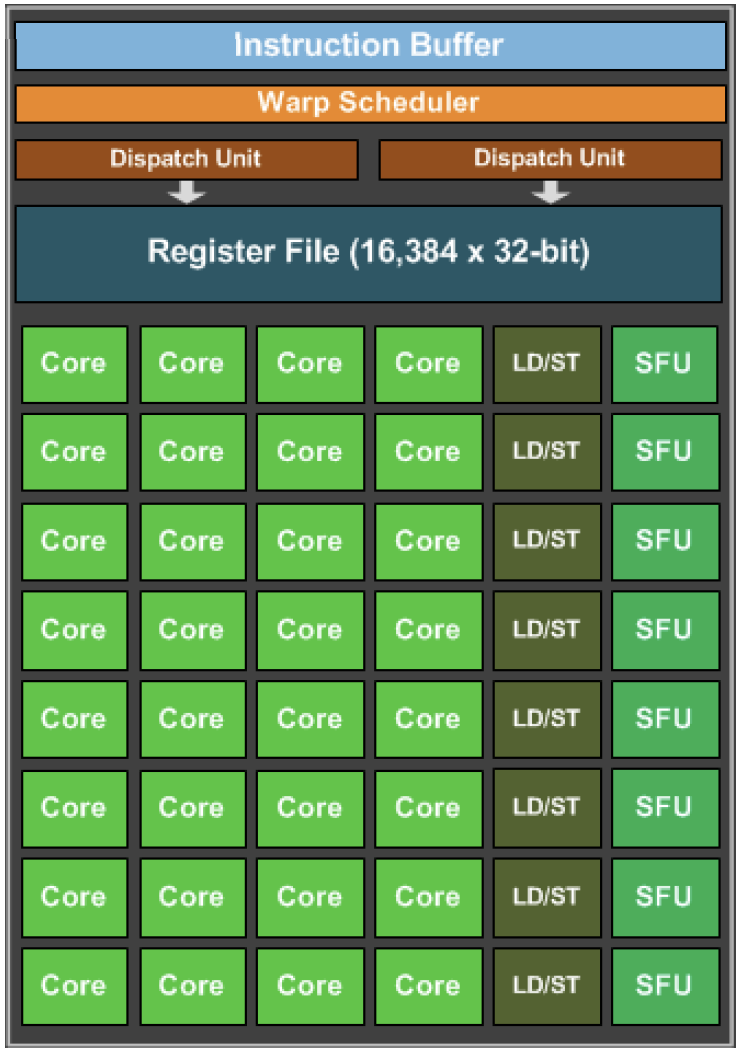
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



All cores need to wait until all cores finish the first instruction

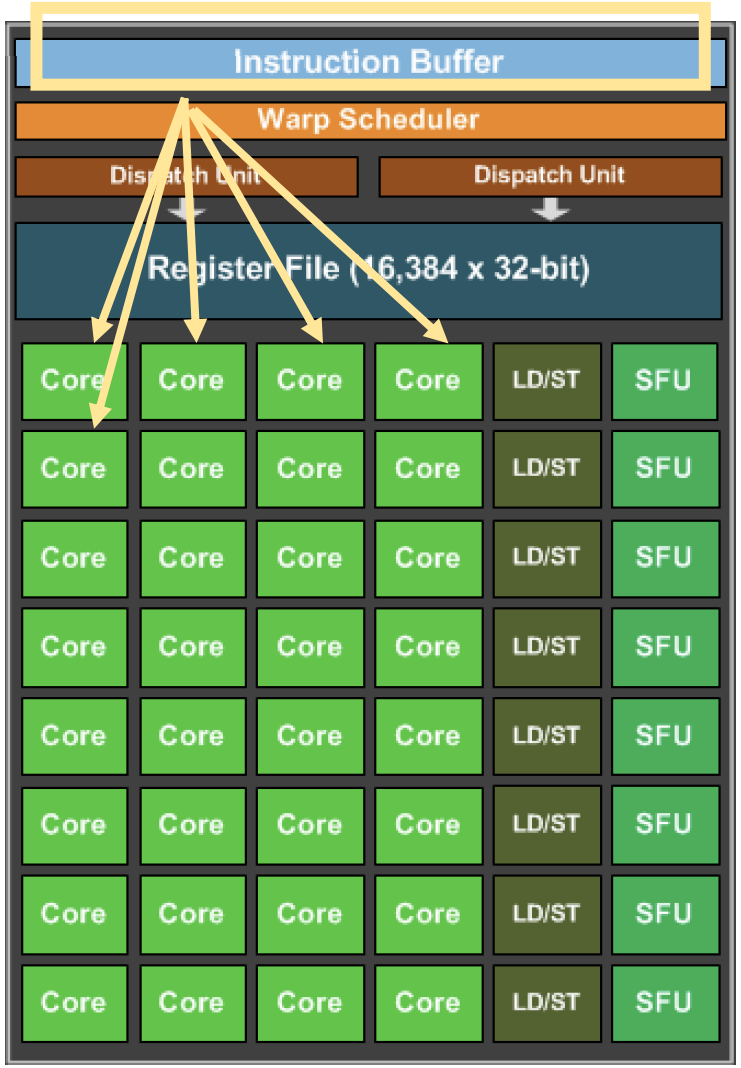
Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



Start the next instruction.

Program:

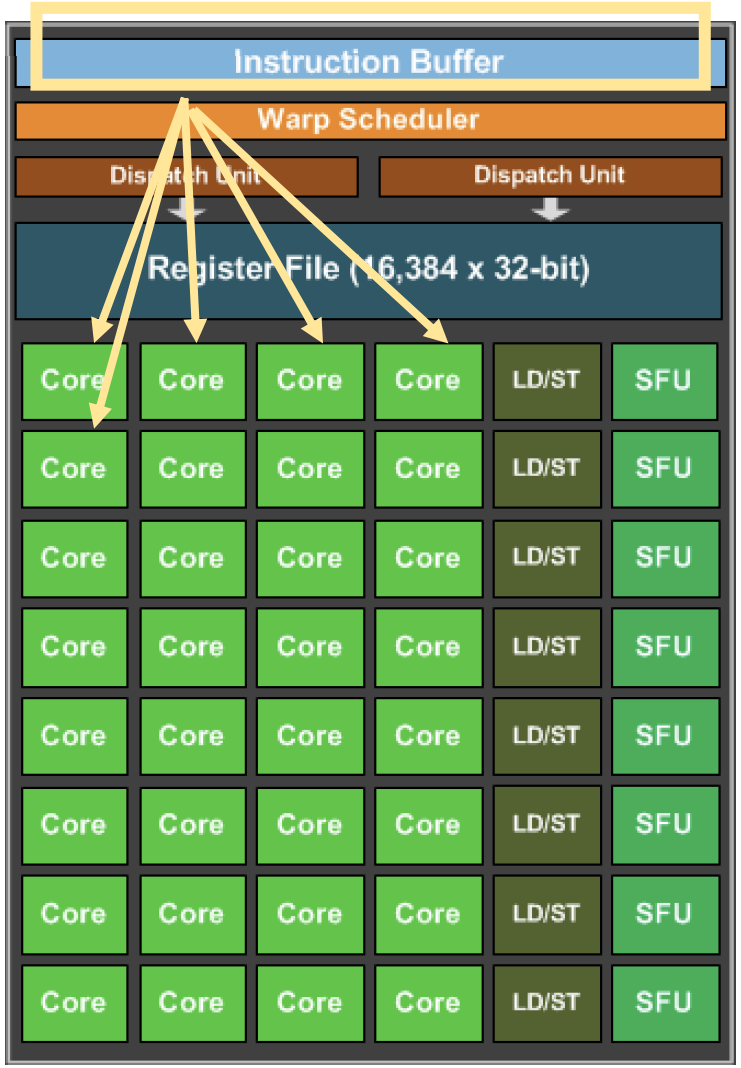
```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Why would we have a programming model like this?

Warp execution

Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time



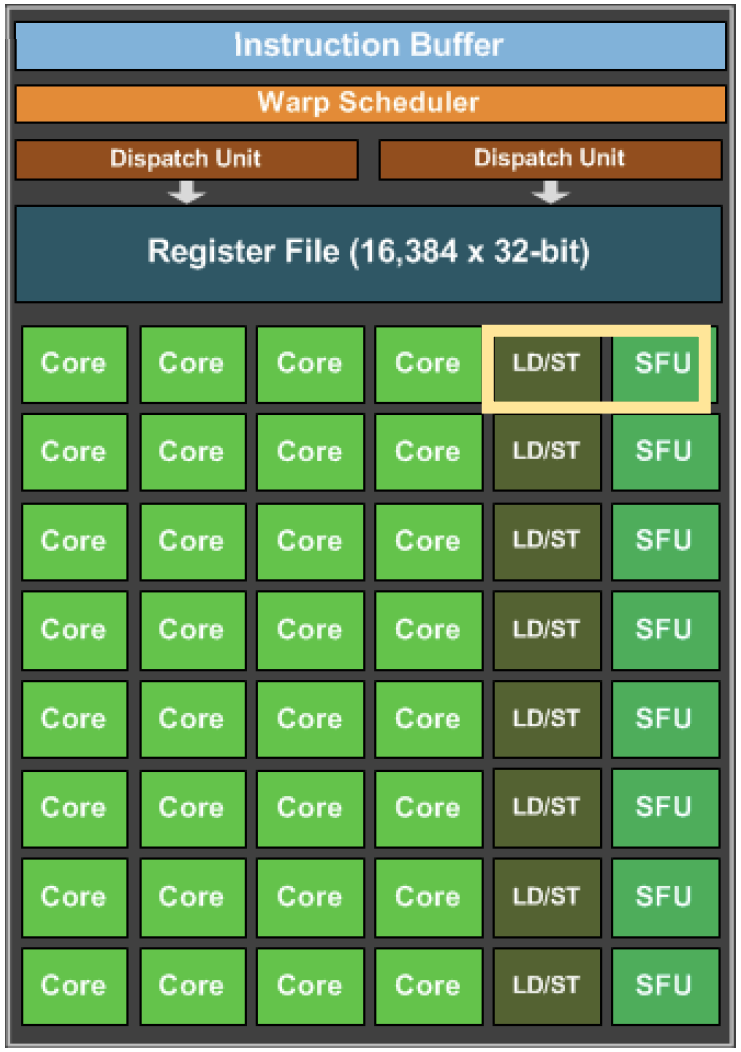
Start the next instruction.

Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Why would we have a programming model like this?
More cores (share program counters)
Can be efficient to share hardware resources

Warp execution



Groups of 32 threads are called a “warp”

They are executed in lock-step, i.e. they all execute the same instruction at the same time

Program:

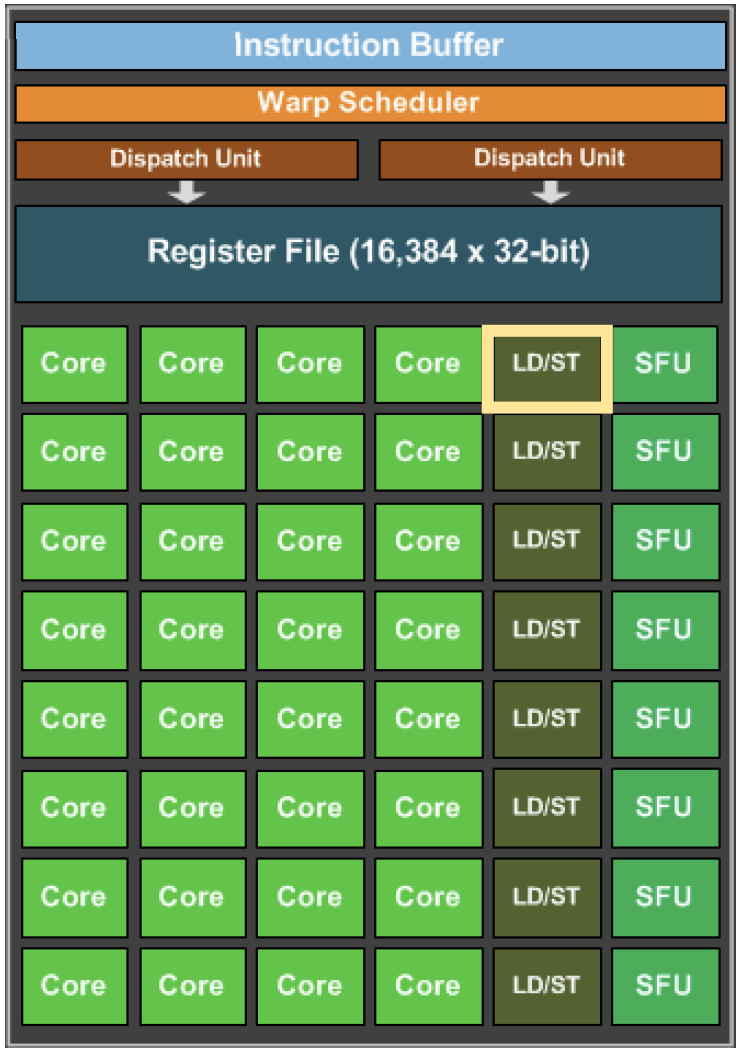
```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

Why would we have a programming model like this?
More cores (share program counters)
Can be efficient to share hardware resources

Cores share expensive HW units (load/store and special functions)

Warp execution

Lets look closer at memory

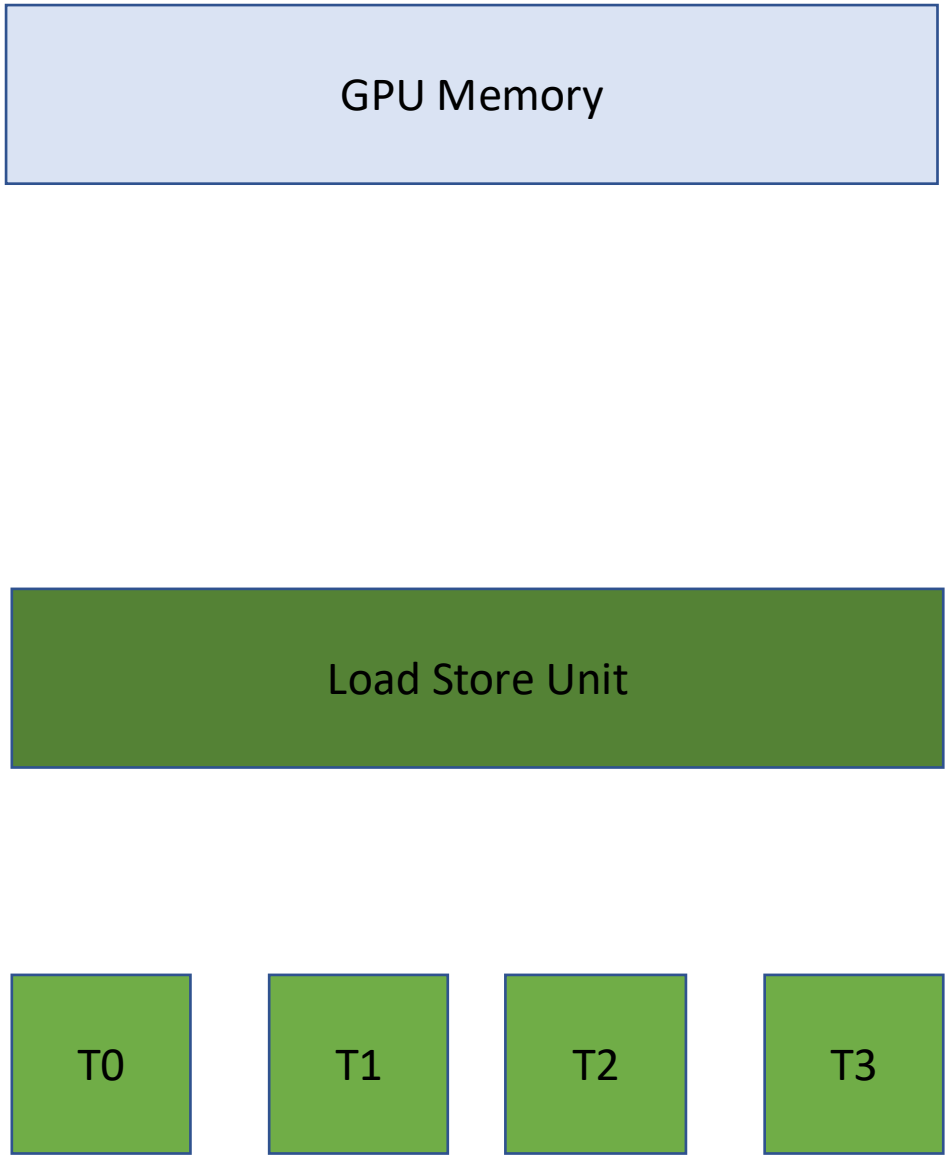


Program:

```
int variable1 = b[0];
int variable2 = c[0];
int variable3 = variable1 + variable2;
a[0] = variable3;
```

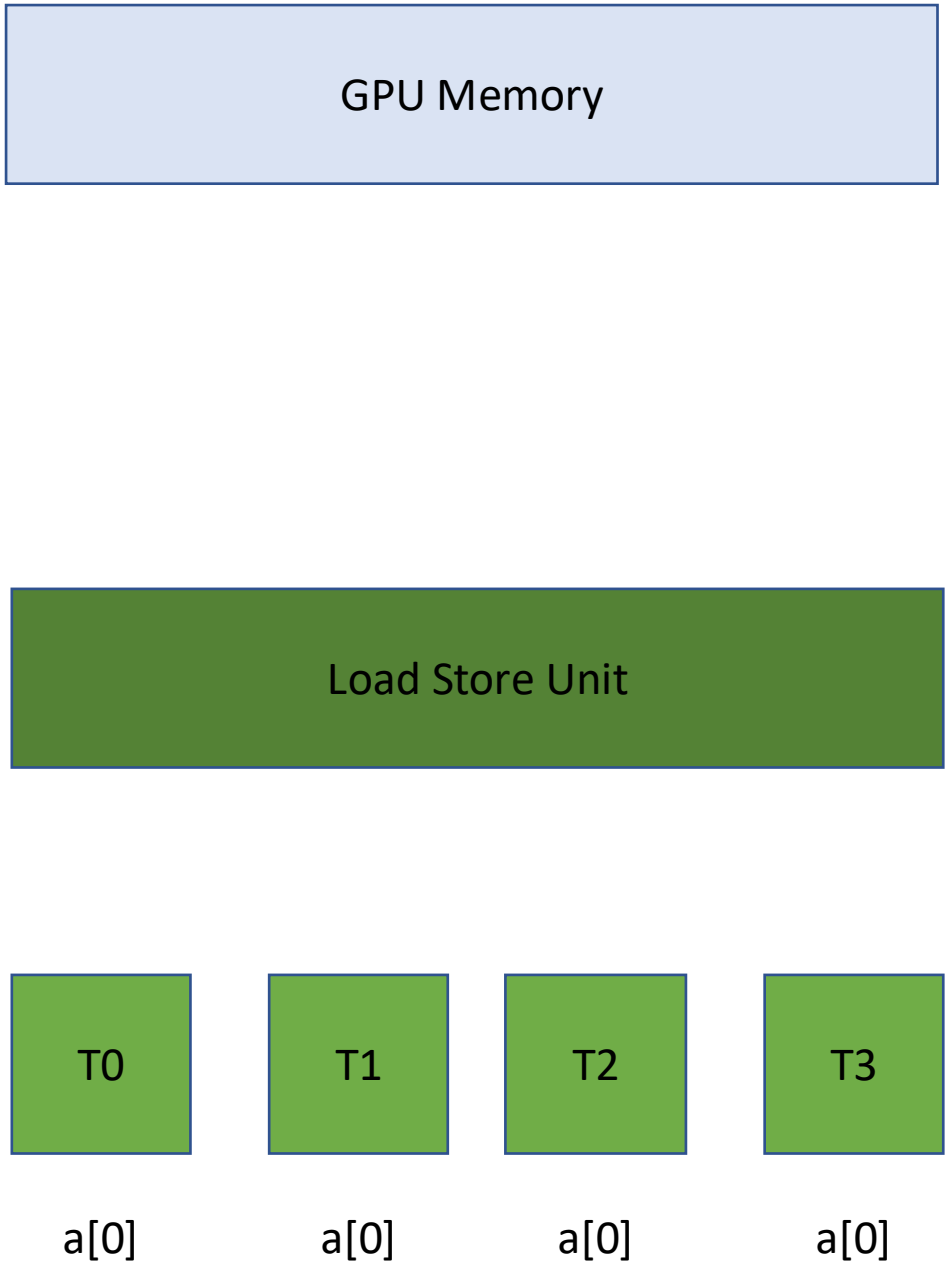
4 cores are accessing memory. what happens if they access the same value?

4 cores are accessing memory. What can happen



4 cores are accessing memory. What can happen

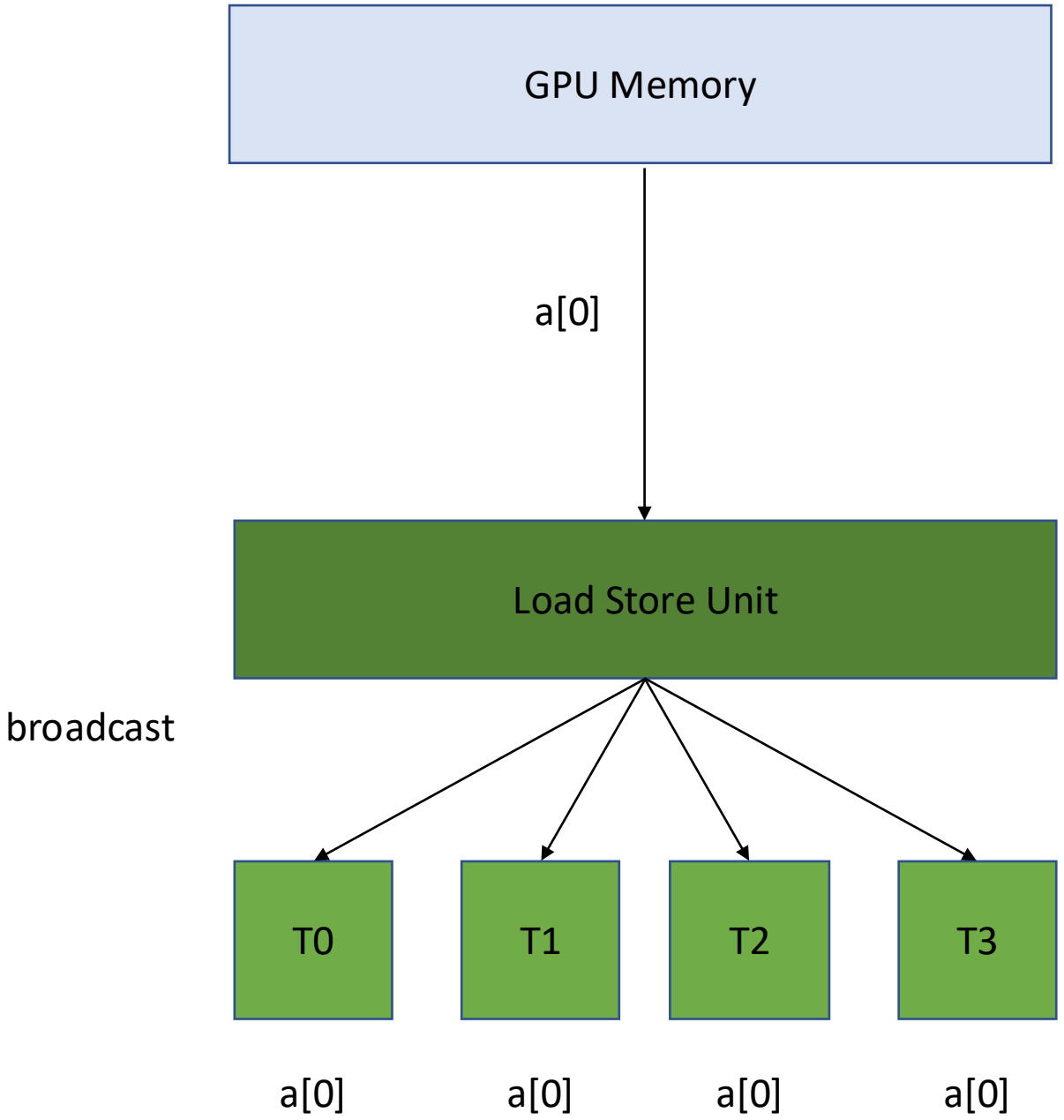
All read the same value



4 cores are accessing memory. What can happen

All read the same value

This is efficient: the load store unit can ask for the value and then broadcast it to all cores.

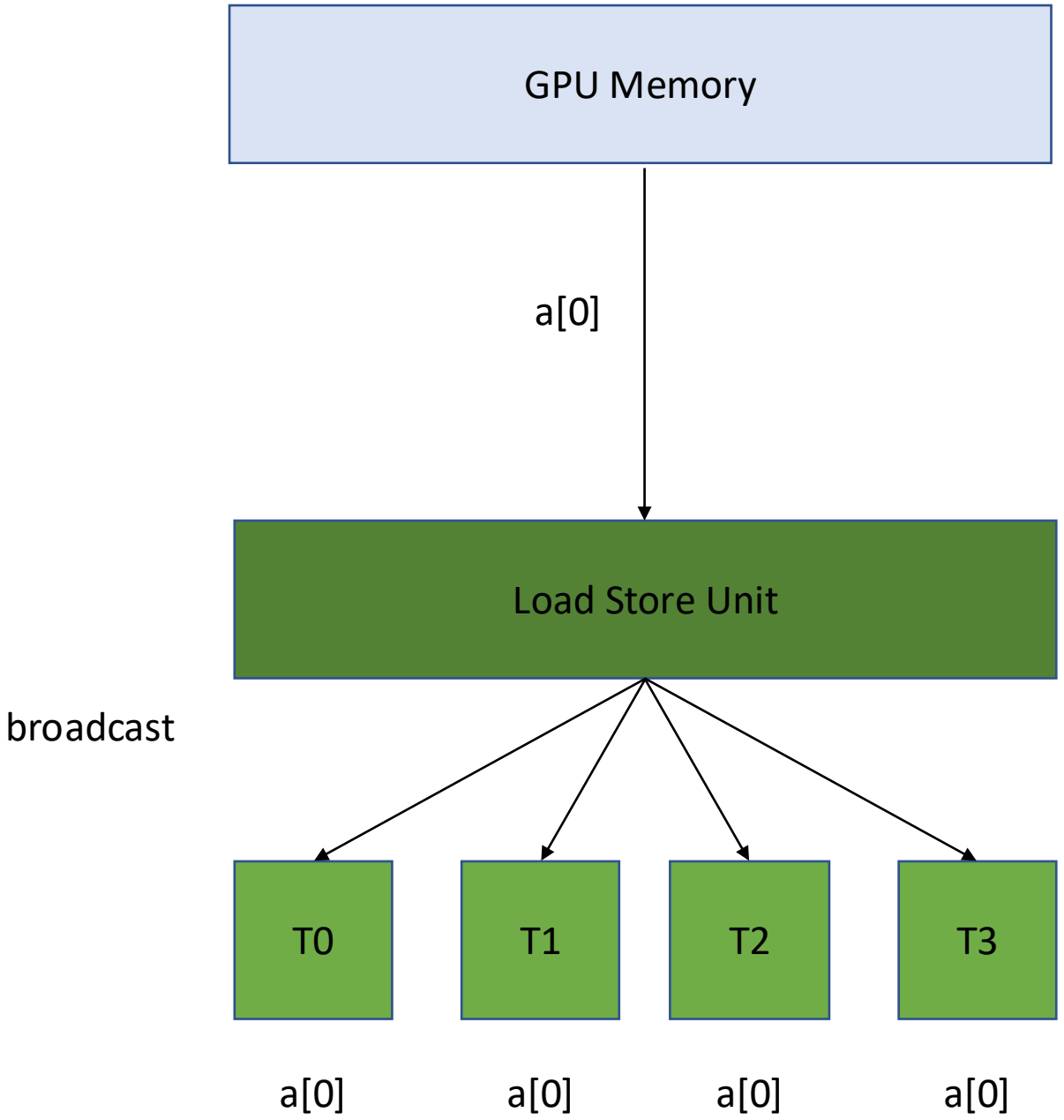


4 cores are accessing memory. What can happen

All read the same value
This is efficient: the load store unit can ask for the value and then broadcast it to all cores.

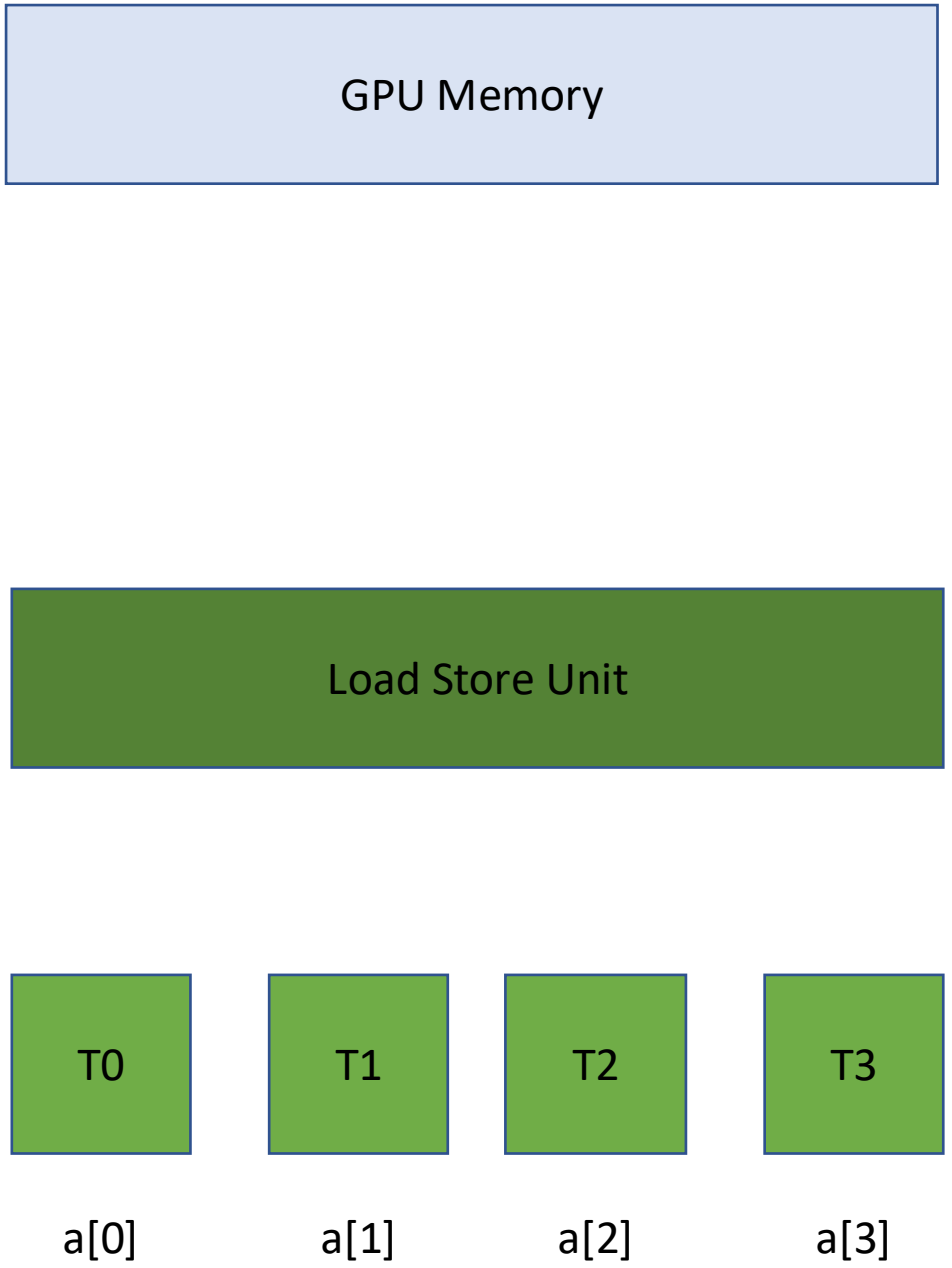
1 request to GPU memory

Efficient, but probably not too common.



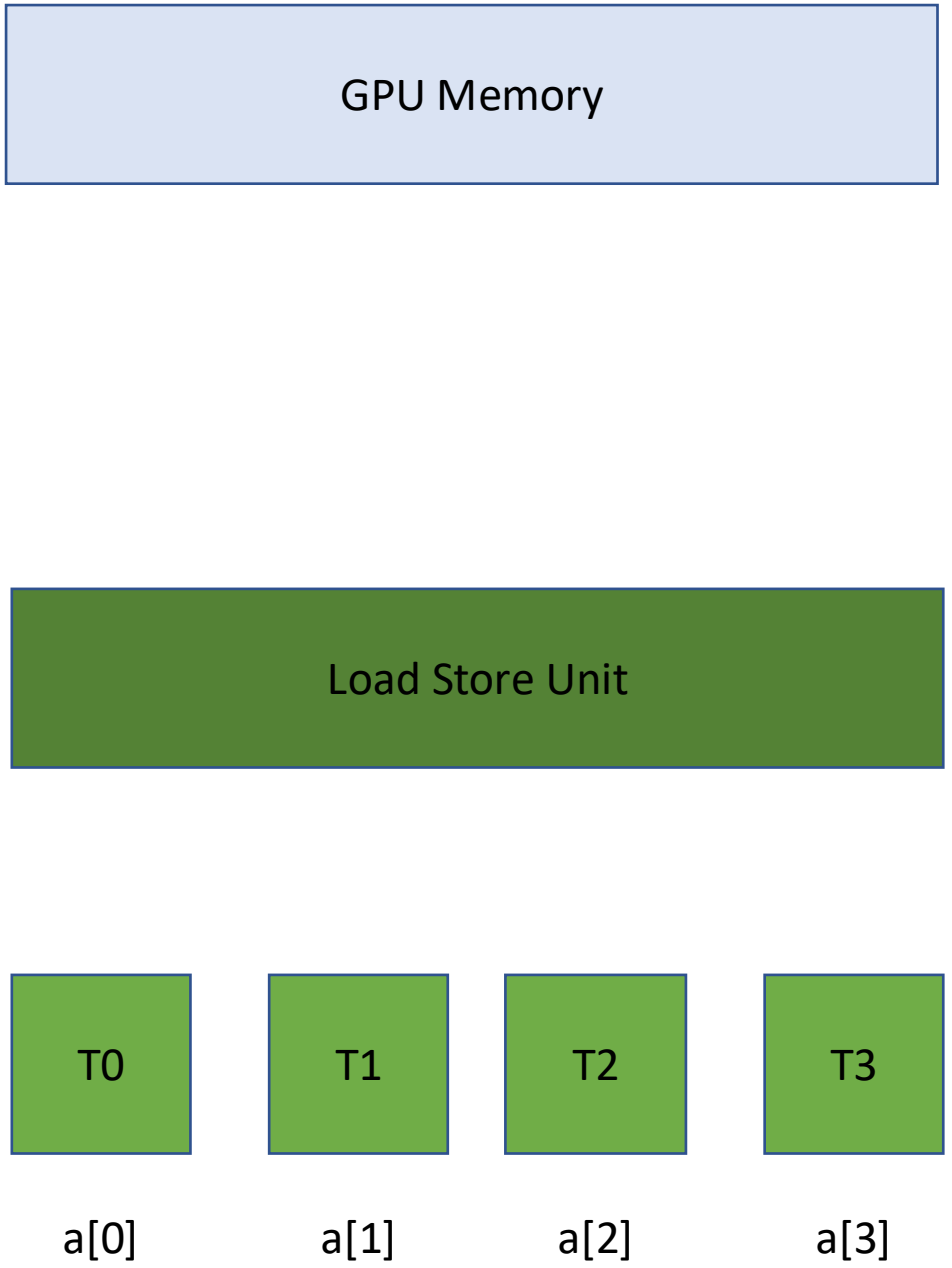
4 cores are accessing memory. What can happen

Read contiguous values



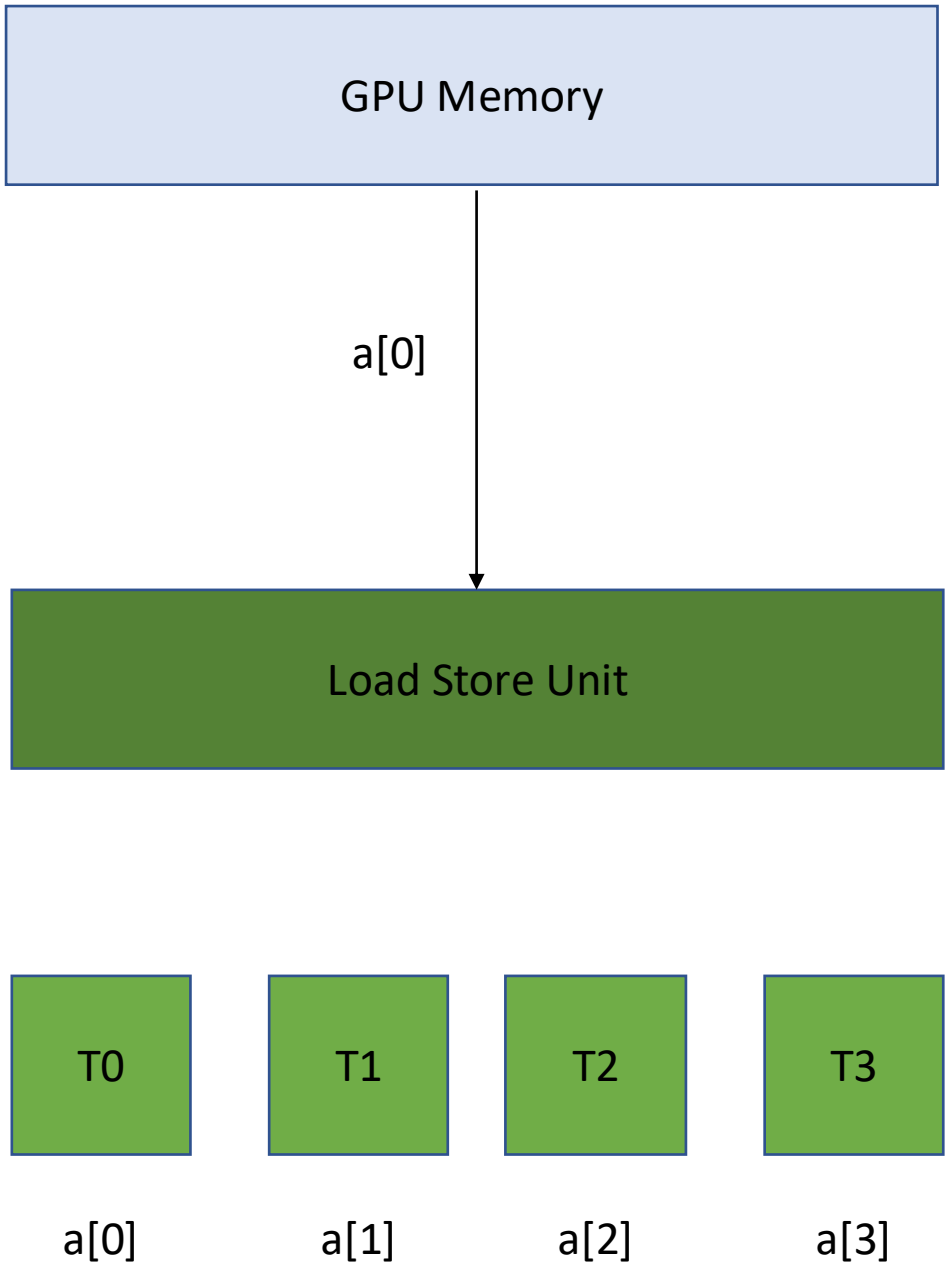
4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes



4 cores are accessing memory. What can happen

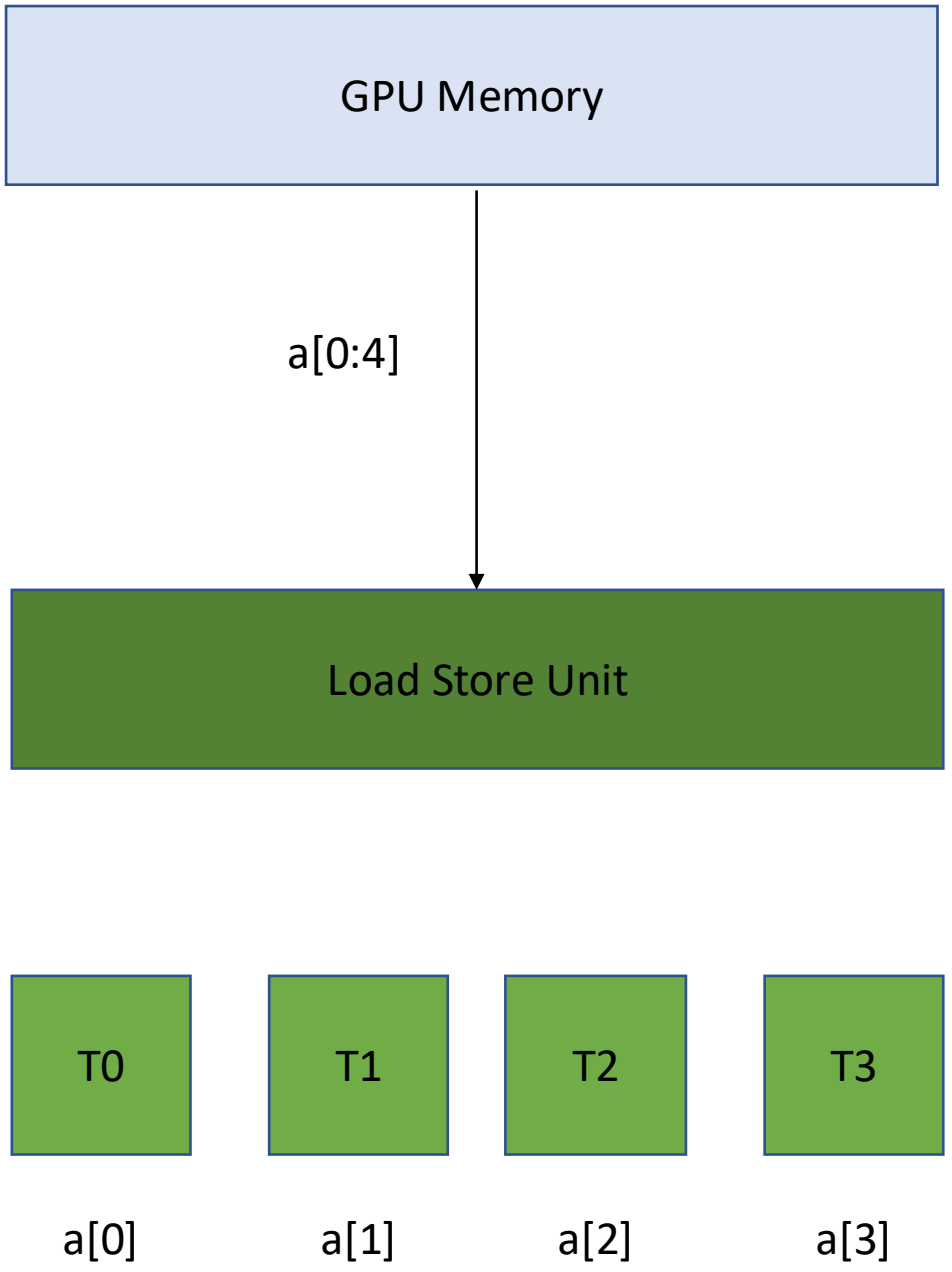
Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes



4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes

Can easily distribute the values to the threads

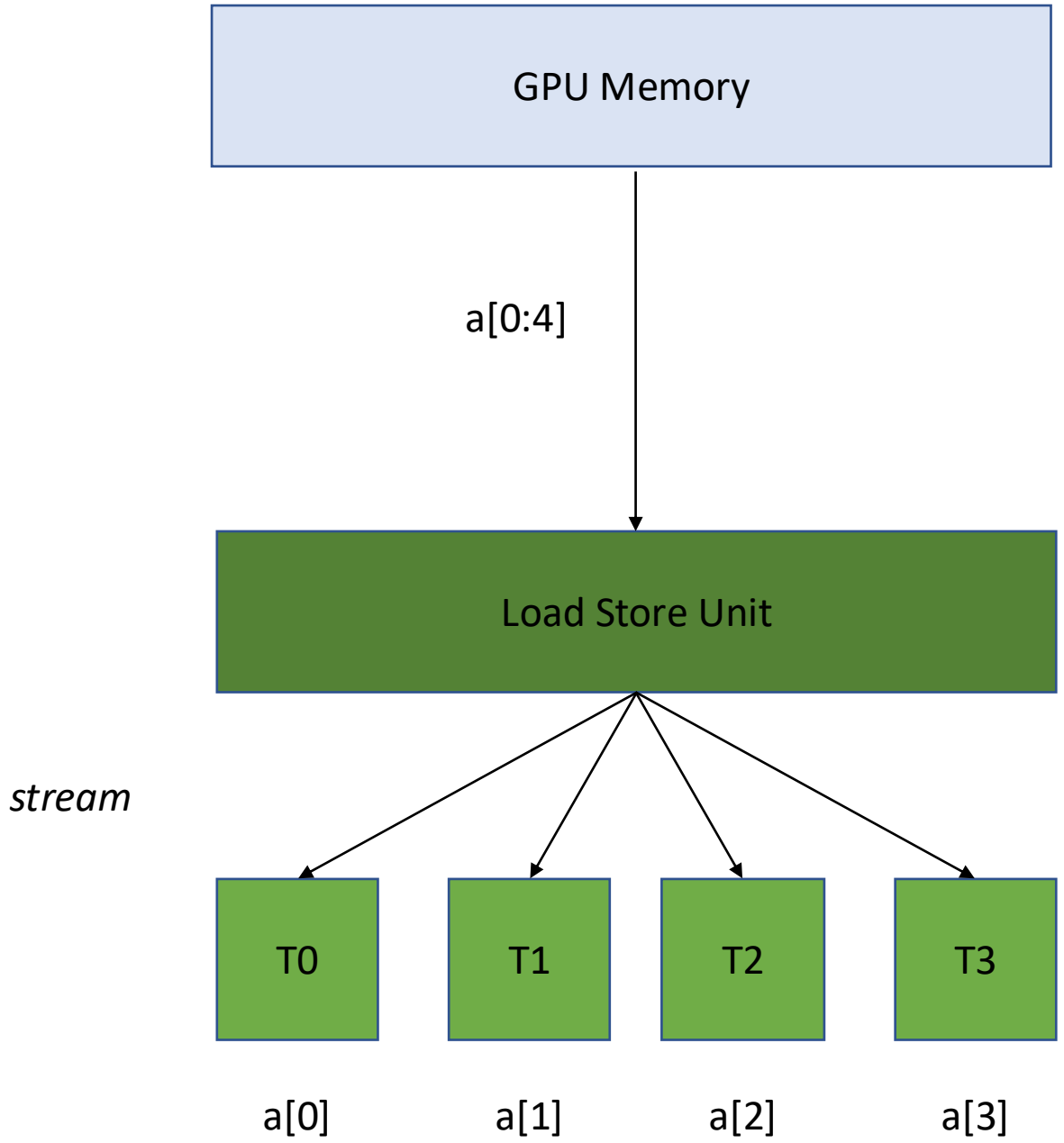


4 cores are accessing memory. What can happen

Read contiguous values
Like the CPU cache, the Load/Store Unit reads in memory in chunks. 16 bytes

Can easily distribute the values to the threads

1 request to GPU memory

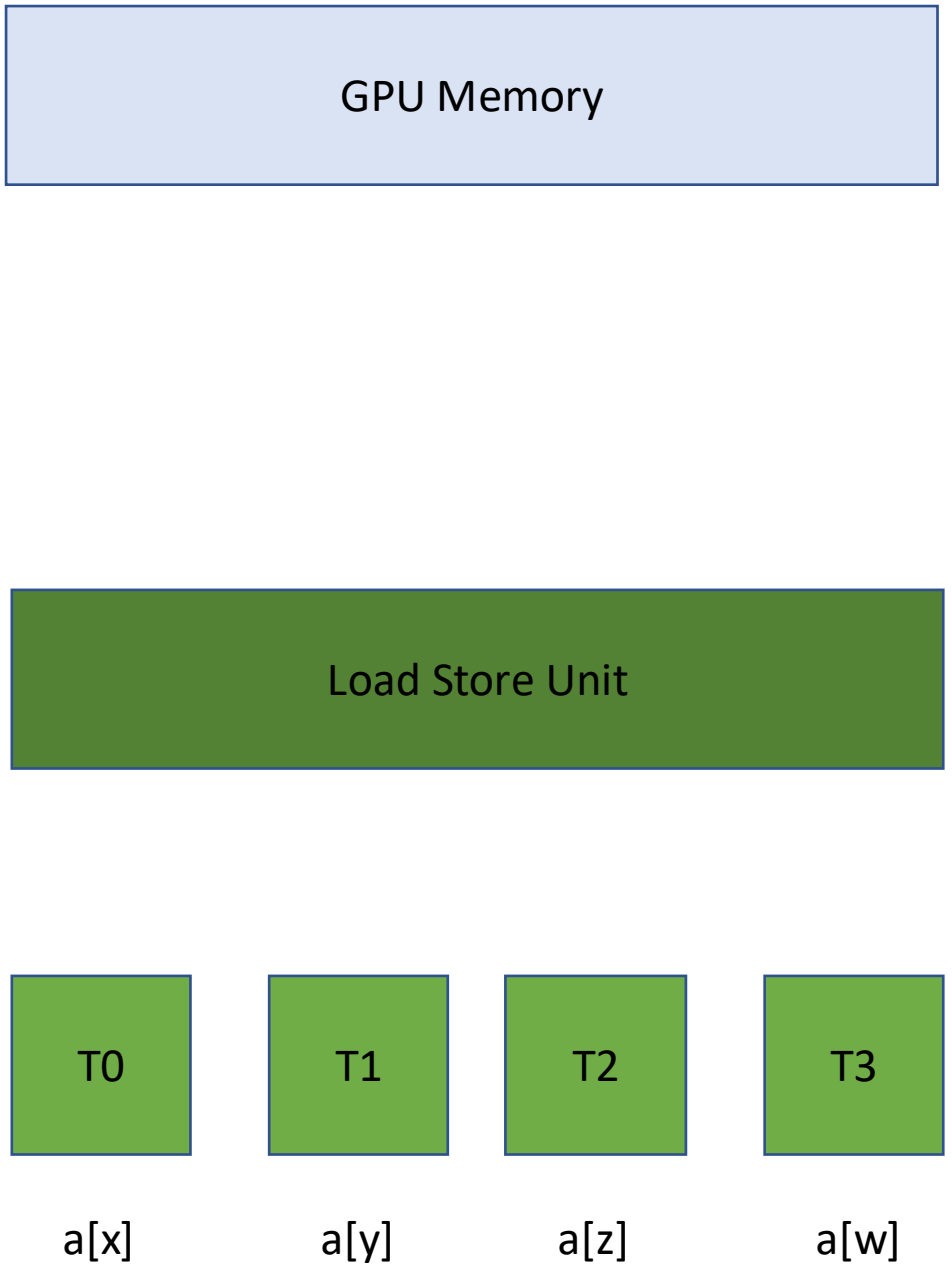


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

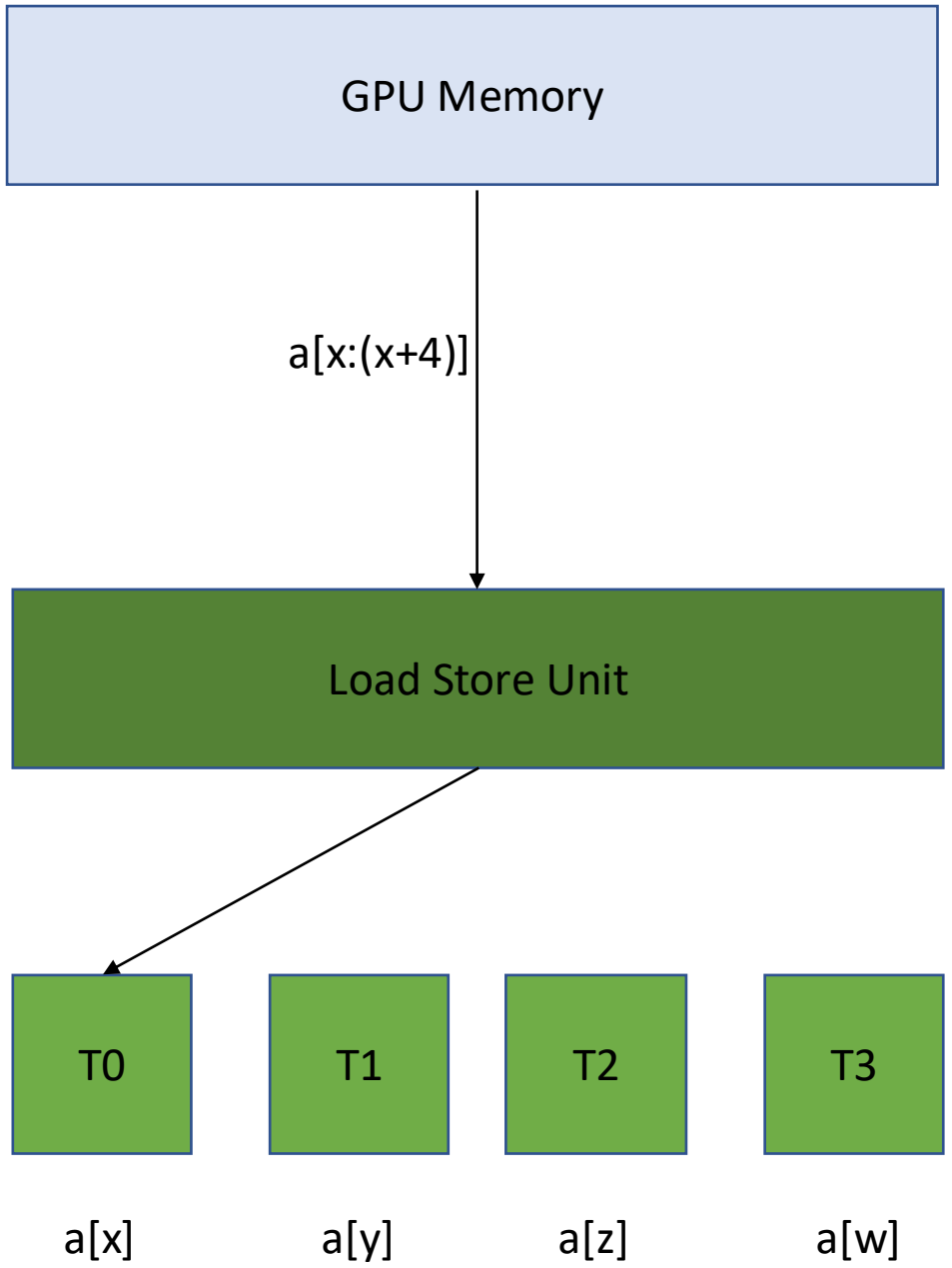


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

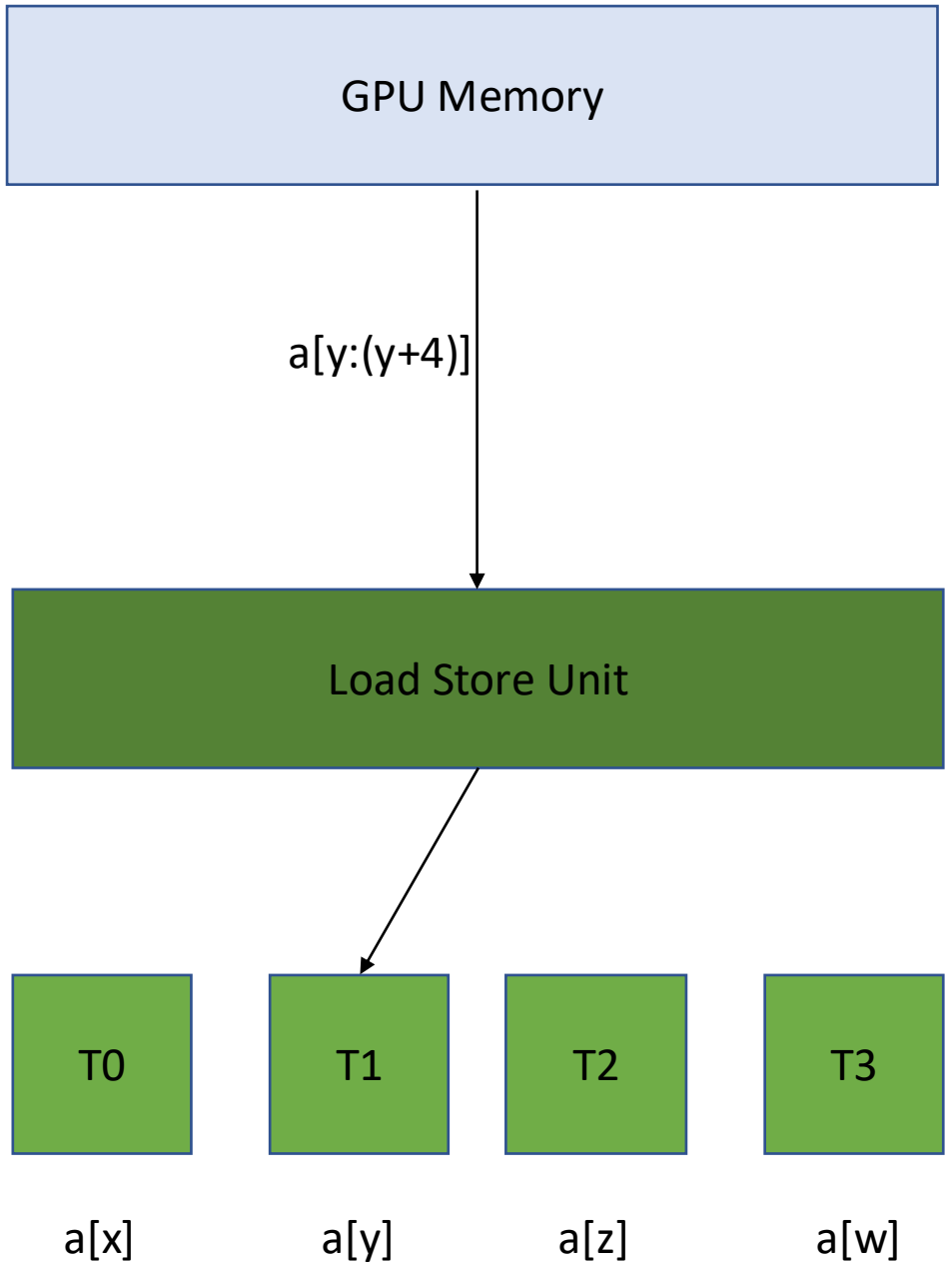


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

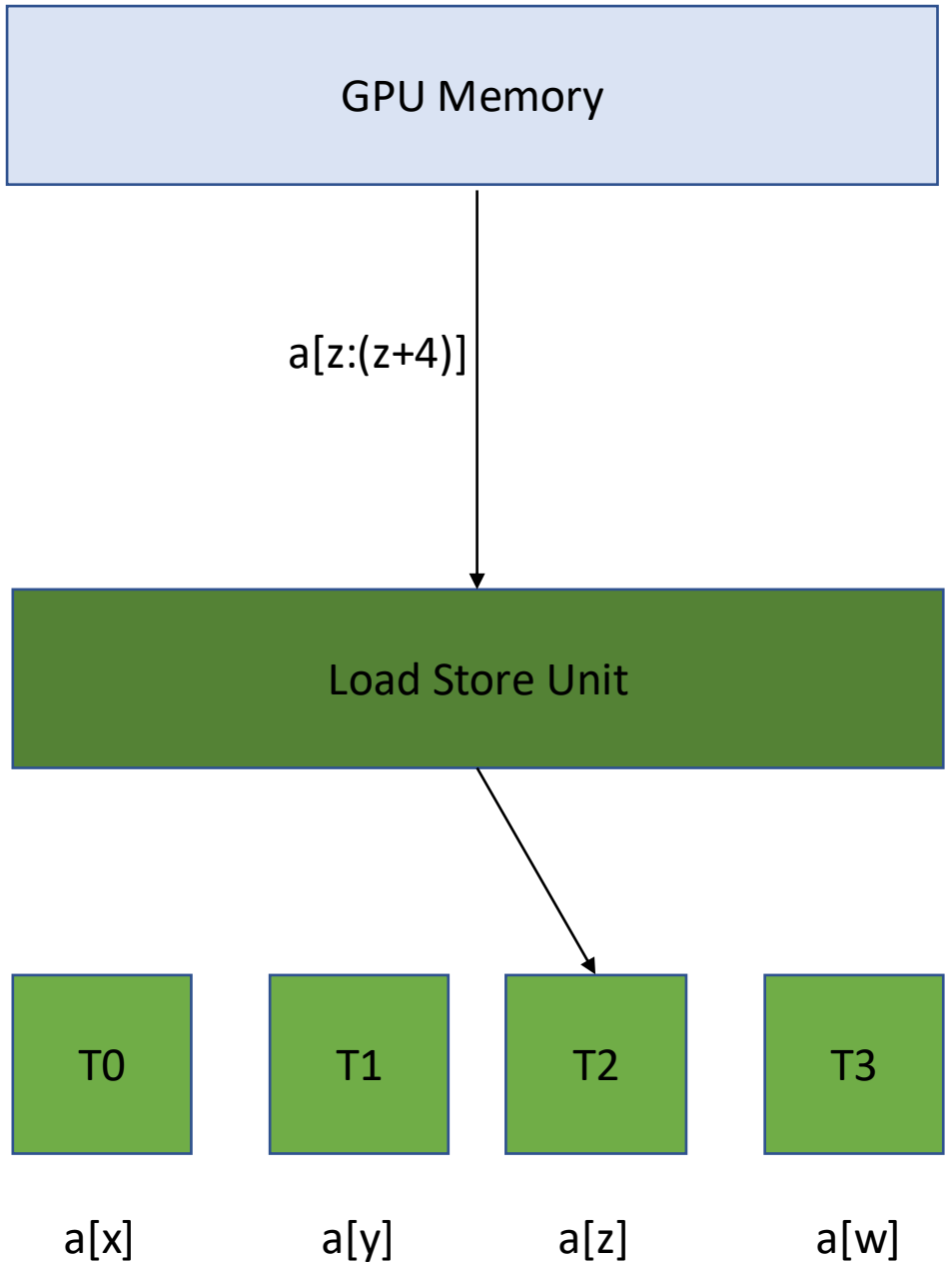


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory

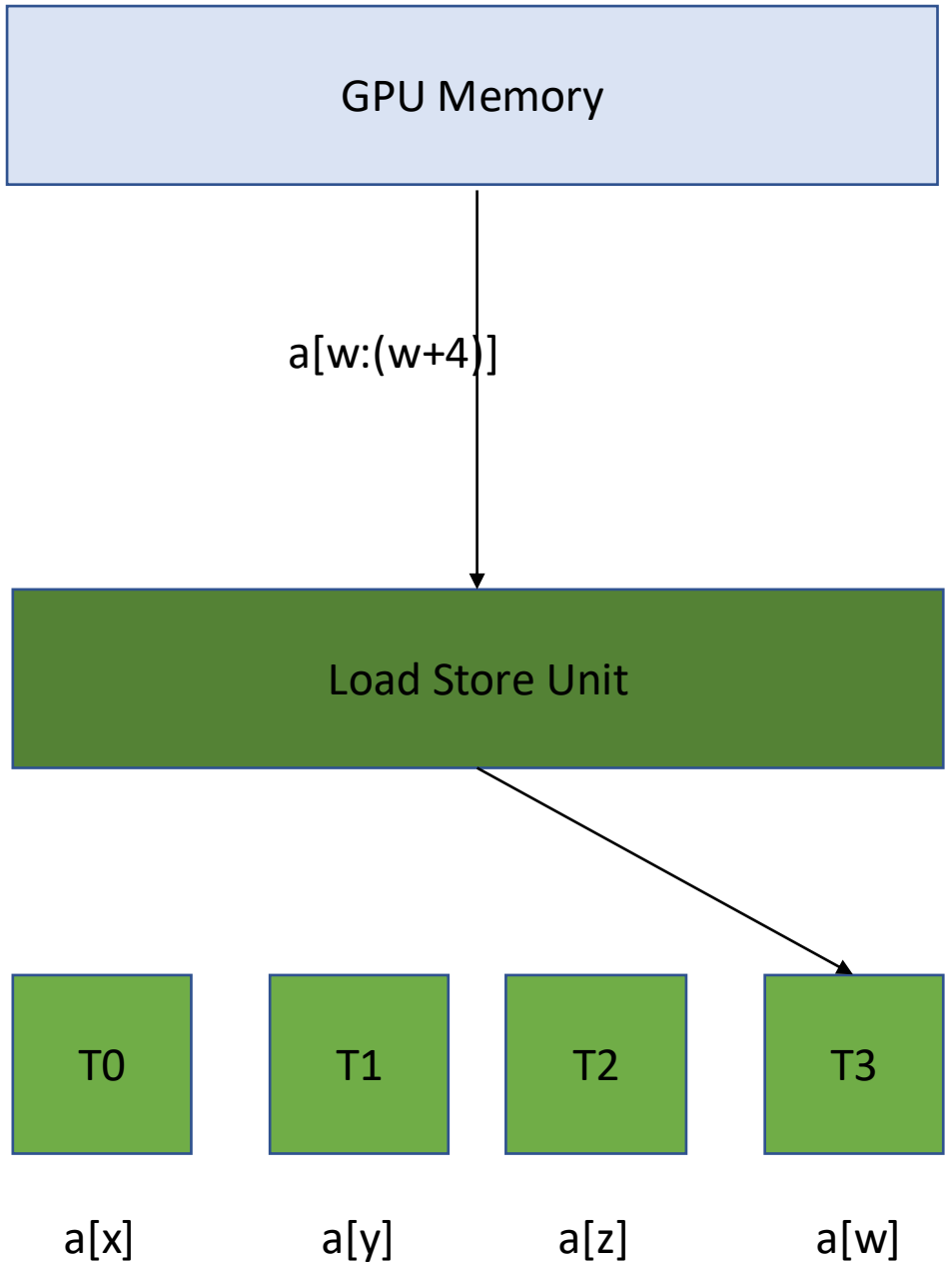


4 cores are accessing memory. What can happen

Read non-contiguous values

Not good!

Accesses are Serialized.
You need 4 requests to GPU memory



Quiz 12.

Like the CPU cache, the Load/Store Unit reads in memory in chunks. Is this correct?

Select one

A. True

B. False

Scan to answer

forms.gle/c8hpnGUtrui63zBcA

chart: docs.google.com/forms/d/e/1FAIpQLSc53ivr_o0JWDVCwQwmlR4_7xFgn_3t4vDOA2QgQjEa9rBsbQ/viewanalytics

Access pattern

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

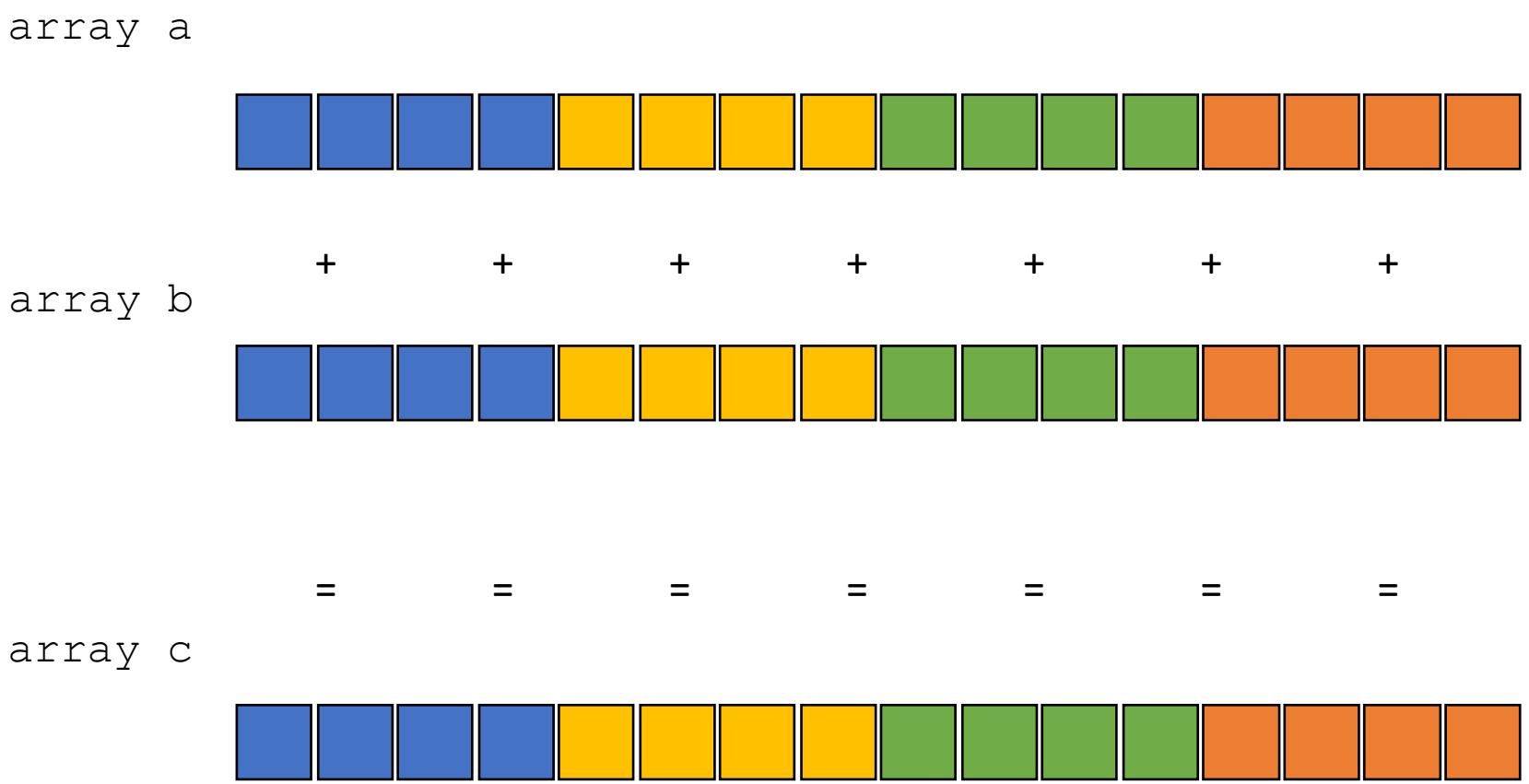
calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Chunked Pattern

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

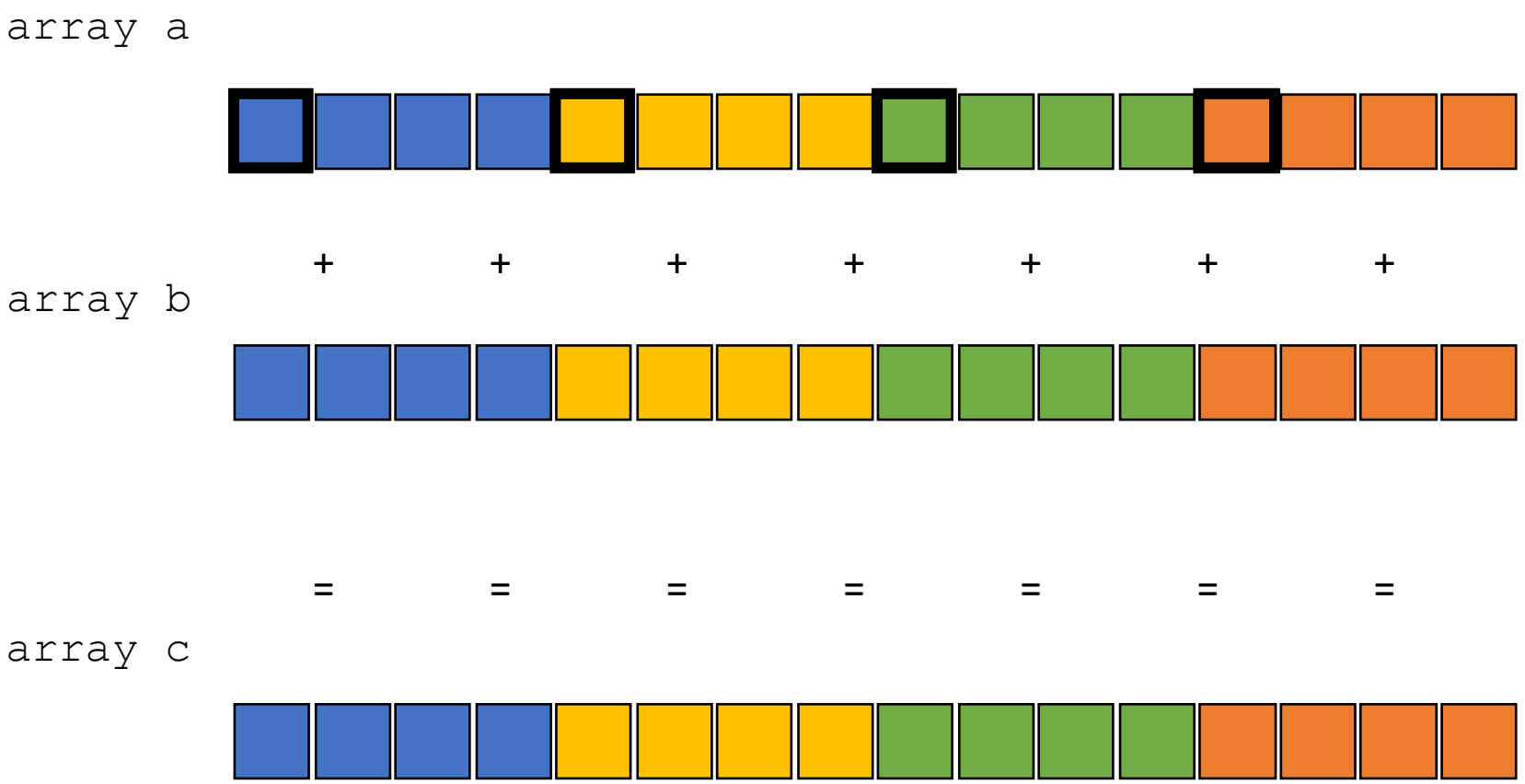


Chunked Pattern

the first element accessed by the 4 threads sharing a load store unit. What sort of access is this?

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



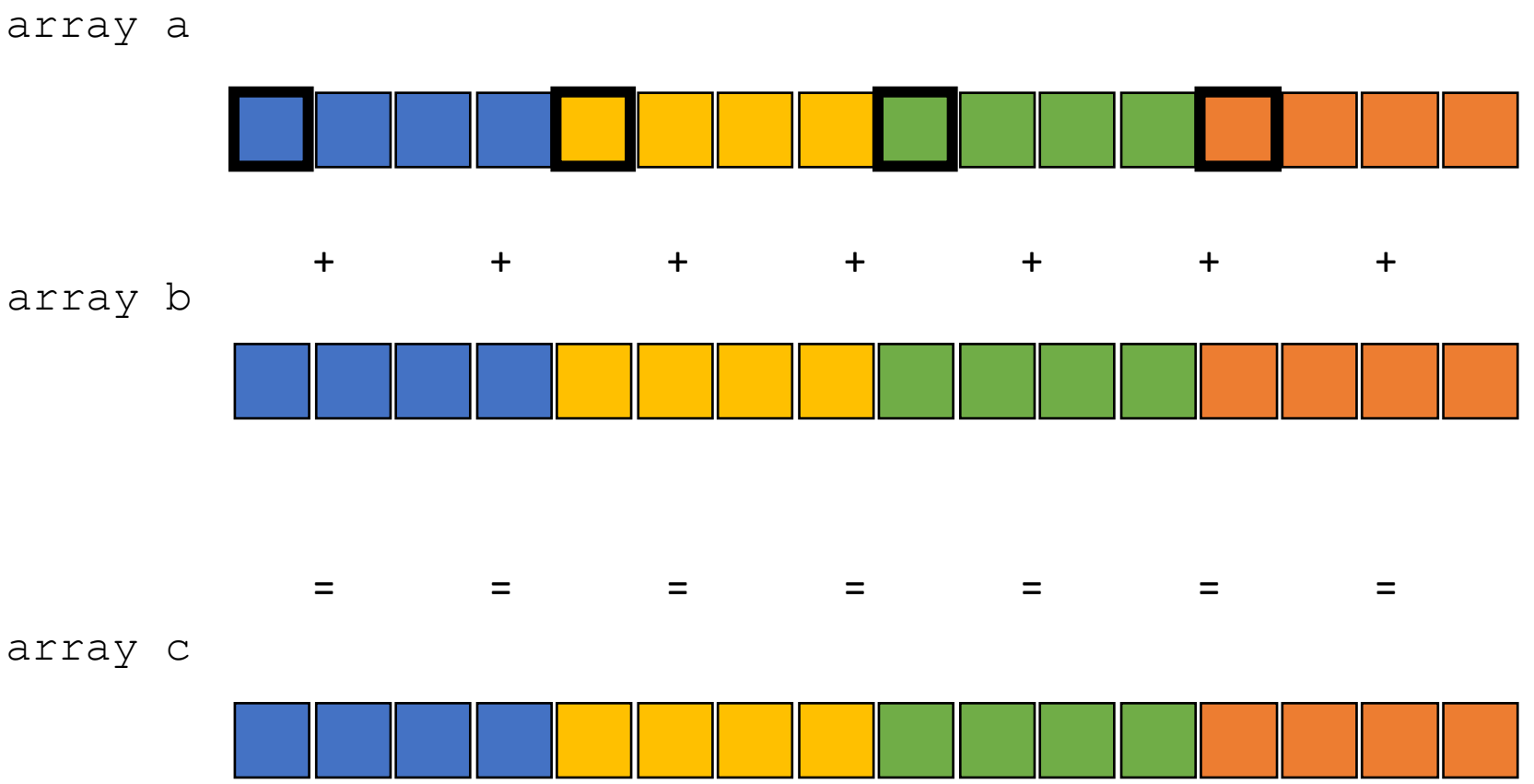
Chunked Pattern

the first element accessed by the 4 threads sharing a load store unit. What sort of access is this?

How can we fix this?

Computation can easily be divided into threads

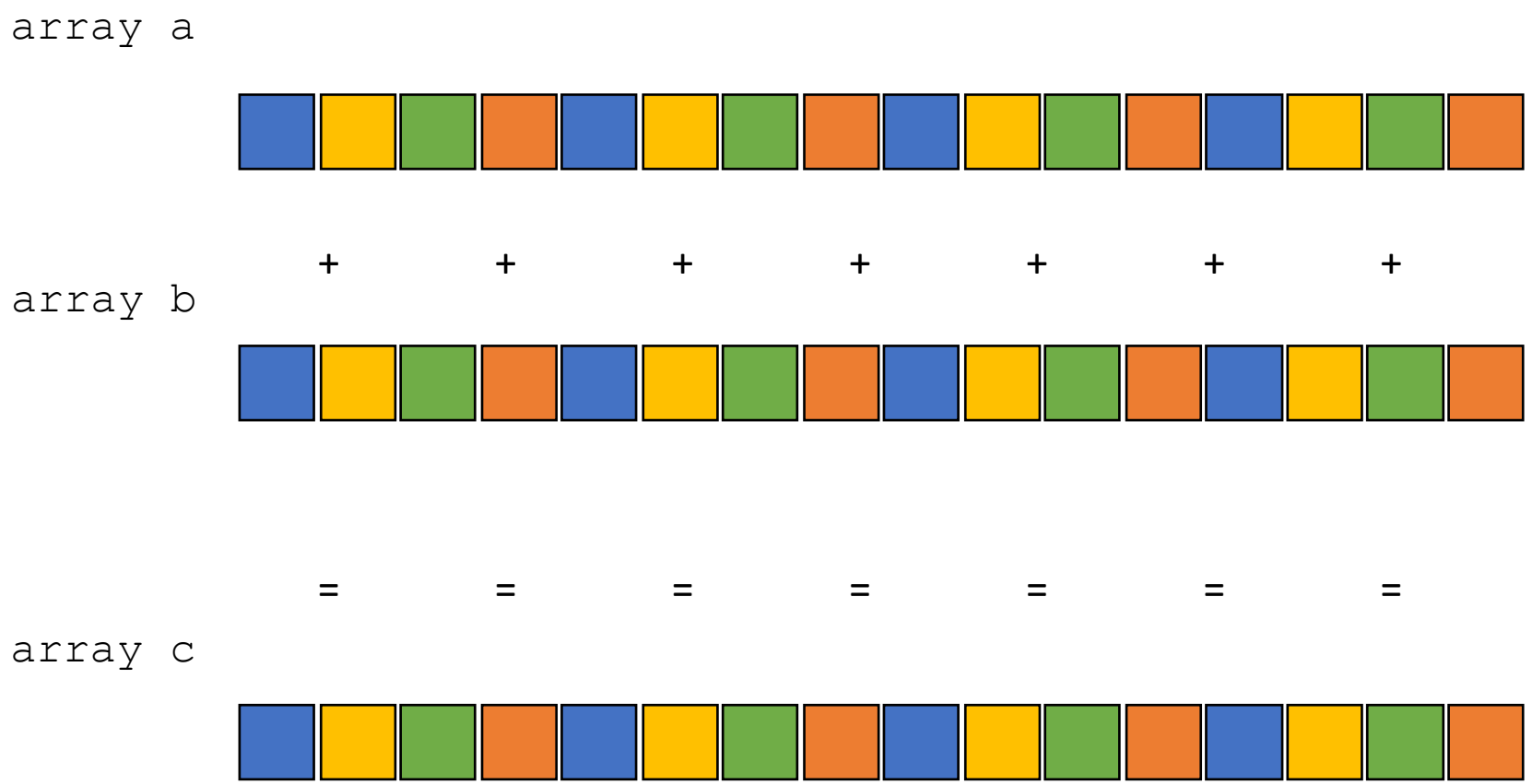
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Stride Pattern

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange

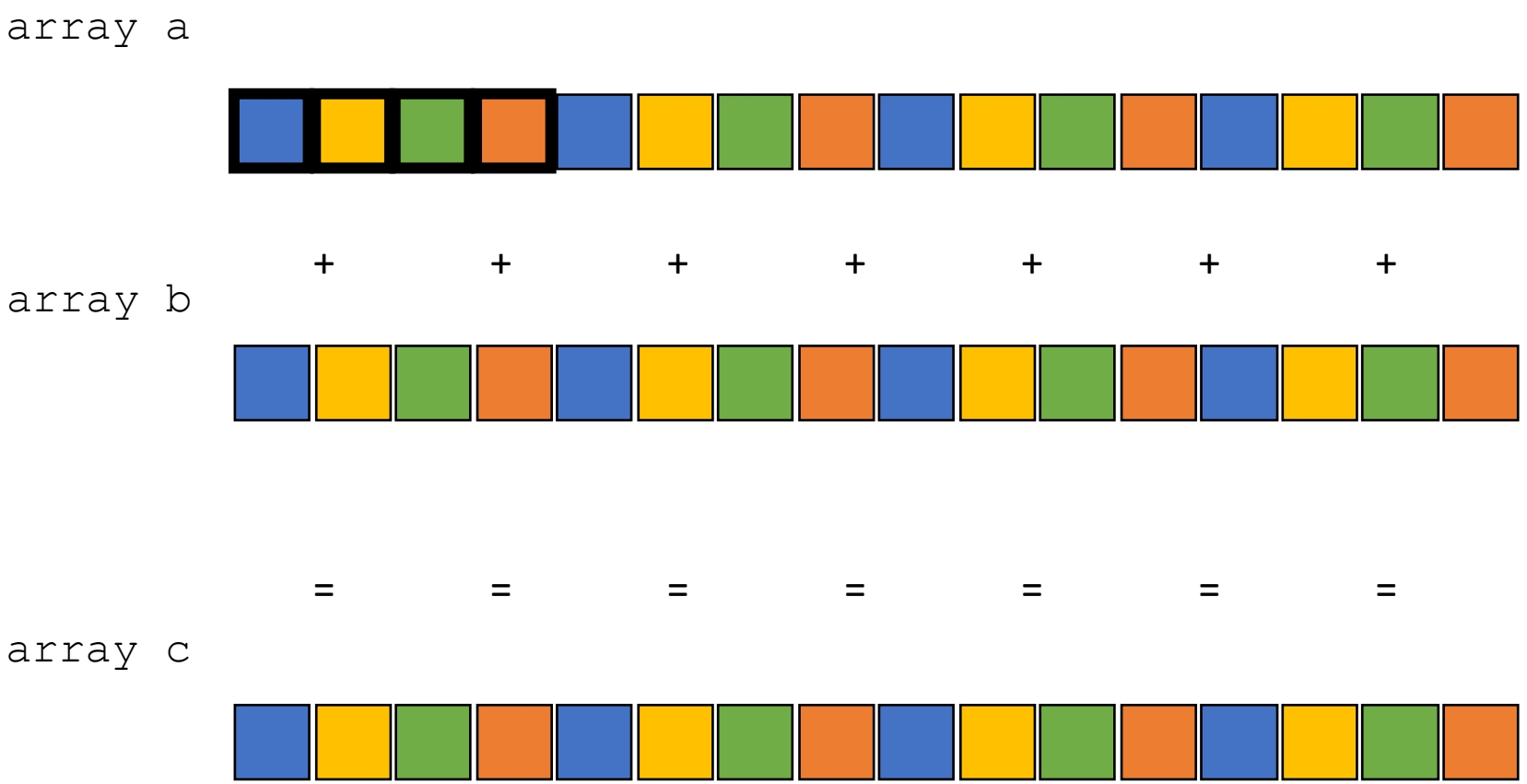


Stride Pattern

What sort of pattern is this?

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    int chunk_size = size/blockDim.x;
    int start = chunk_size * threadIdx.x;
    int end = start + chunk_size;
    for (int i = start; i < end; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Chunked pattern
Let's change this to a stride pattern

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  for (int i = threadIdx.x; i < size; i+=blockDim.x) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Coalesced memory accesses

This is about 4x faster.

Quiz 12.

What can accelerate memory accesses on GPUs?

Free response: a few sentences on the form



Scan to answer

forms.gle/u2GrZU9t4BPo1iES9

chart: docs.google.com/forms/d/e/1FAIpQLSfqLApVqPTdL8tkgO51FhYpmg-18blEHVSUIyKfQ_NwODqnow/viewanalytics

Quiz 12.

What can accelerate memory accesses on GPUs?

Free response: a few sentences on the form

Memory coalescing accelerates execution.

Scan to answer

forms.gle/u2GrZU9t4BPo1iES9

chart: docs.google.com/forms/d/e/1FAIpQLSfqLApVqPTdL8tkgO51FhYpmg-18blEHVSUIyKfQ_NwODqnow/viewanalytics

Coalesced memory accesses

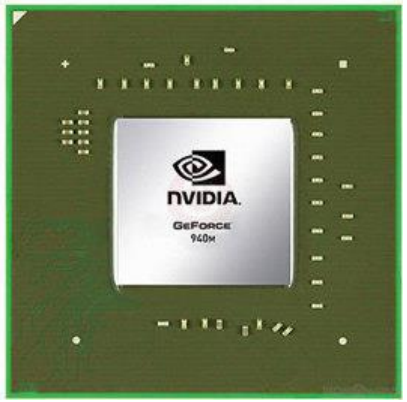
This is about 4x faster.



What else can we do?

Final Round

Tiny GPU in an embedded system



Fight!



The CPU in my professor workstation



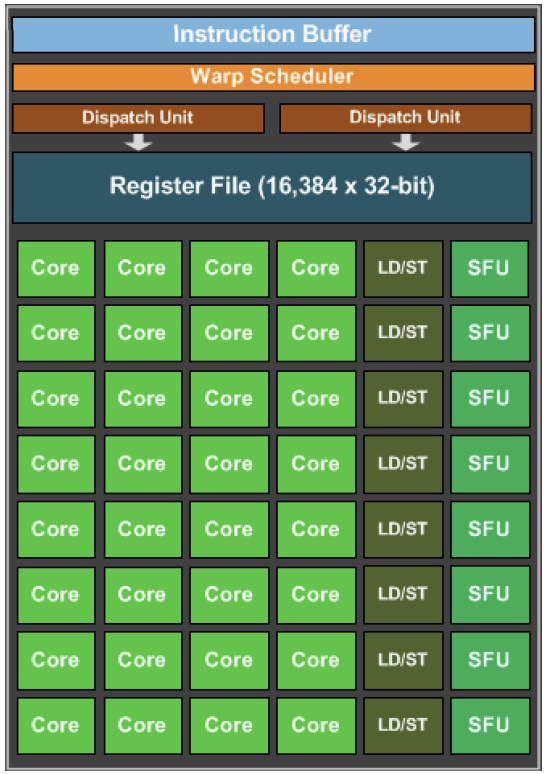
Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP
Est. $99

Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

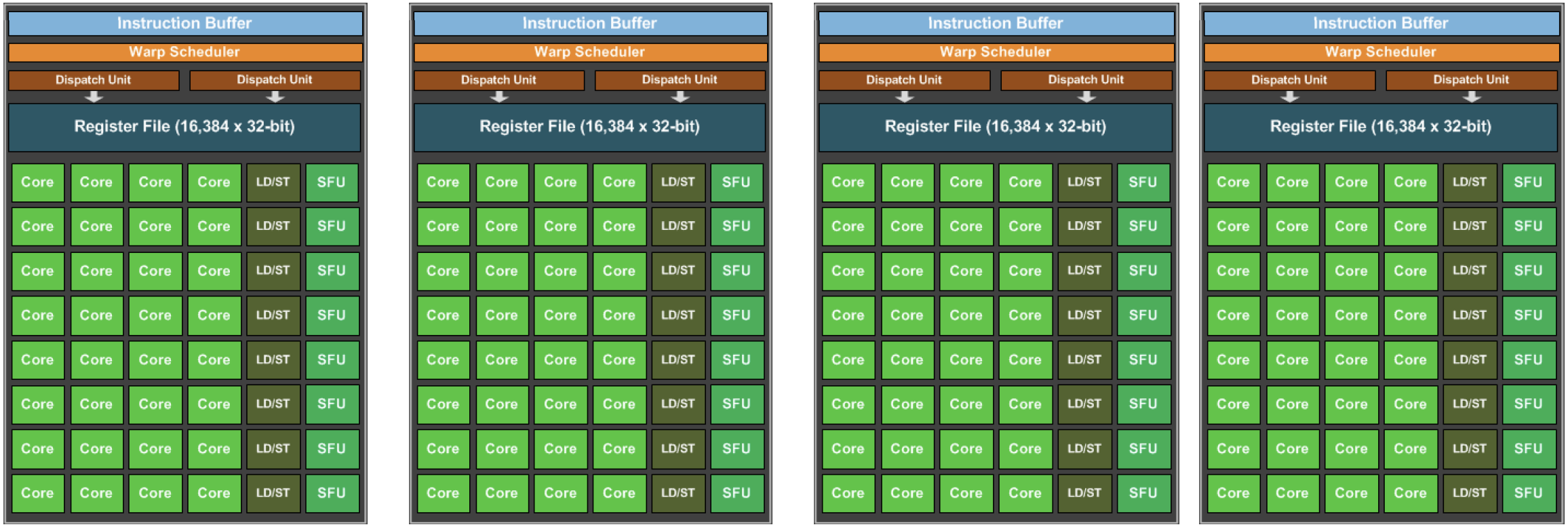
Multiple streaming multiprocessors

We've been talking only about 1 streaming multiprocessor, most GPUs have multiple SMs big ML GPUs have 32.



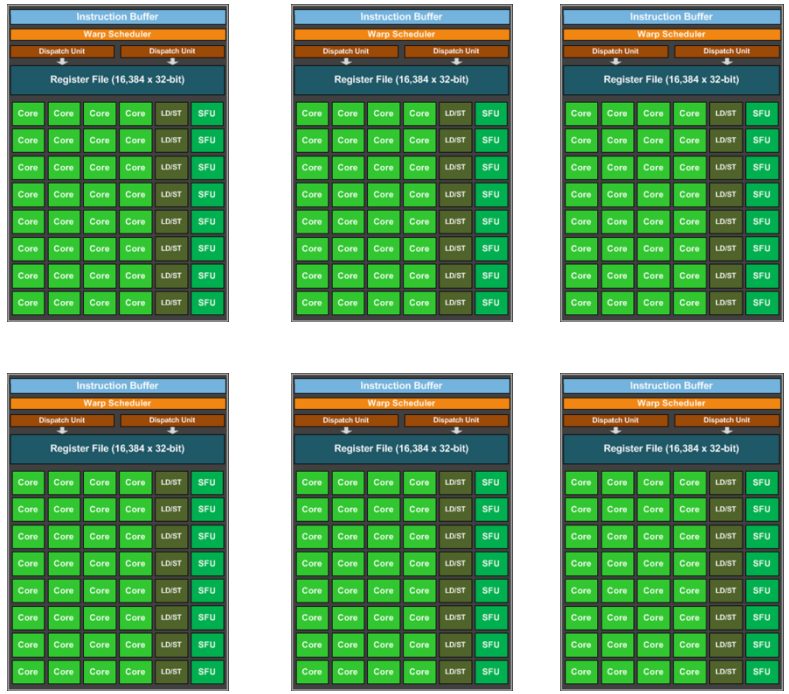
Multiple streaming multiprocessors

We've been talking only about 1 streaming multiprocessor, most GPUs have multiple SMs big ML GPUs have 32. This little GPU has 4



Multiple streaming multiprocessors

CUDA provides virtual streaming multiprocessors called **blocks**
They hide memory latency
Very efficient at launching and joining **blocks.**
No limit on blocks: launch as many as you need to map 1 thread to 1 data element



Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  for (int i = threadIdx.x; i < size; i+=blockDim.x) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```

calling the function

```
vector_add<<<1,1024>>>(d_a, d_b, d_c, size);
```

Launch with many thread blocks

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Go back to our program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  d_a[i] = d_b[i] + d_c[i];
}
```

calling the function

```
#define SIZE (1024*1024)
vector_add<<<1024,1024>>>(d_a, d_b, d_c, SIZE);
```

Launch with many thread blocks

Need to recalculate some thread ids.

Now we have 1 thread for each element

blockIdx.x is the block id.
blockDim.x is the number of threads per block.
threadIdx.x is the thread id within block.

Quiz 12. Q4

Why do we need to calculate int i = blockIdx.x * blockDim.x + threadIdx.x?

Select one

A. To get the index in the matrix we want to compute in that thread

B. To get the whole matrix we want to compute in that thread

C. To get the index in the matrix we want to compute in that warp



Scan to answer

forms.gle/nCPhnjB89Kb1iF8t6

chart: docs.google.com/forms/d/e/1FAIpQLSdPYUn4_ybwmH3S_r8JaguP6oDhihRmp0GREkT-t_5T_nlGKA/viewanalytics

Final round

With 10x energy efficiency, GPU is as fast as CPU.

A datacenter GPU: an H100 at 700 W.
With the same energy efficiency, GPU is about 33x as CPU.

2D or 3D

```
dim3 block(32, 32);                              // 1024 threads per block
                                                 // defaulting to 1
dim3 grid((W + 31) / 32, (H + 31) / 32);         // enough blocks to cover W x H
                                                 // A grid is all the blocks created
                                                 //   by one kernel launch.
blur<<<grid, block>>>(in, out, W, H);

__global__ void blur(float *in, float *out, int W, int H) {
  int x = blockIdx.x * blockDim.x + threadIdx.x;
  int y = blockIdx.y * blockDim.y + threadIdx.y;
  if (x < W && y < H)
    out[y * W + x] = ...;
}
```

thread, warp (32 threads, scheduled as a unit), block (up to 1024 threads, 32 warps on one SM), grid.

Consequences of Warps, Stride Instructions

```
__global__ void vector_add(float * d_a, float * d_b, float * d_c, int size) {
  int global_id = (blockIdx.x * blockDim.x) + threadIdx.x;
  if (global_id % 4 == 0)
      d_a[global_id] = d_b[global_id] + d_c[global_id];
  else if (global_id % 4 == 1)
      d_a[global_id] = d_b[global_id] * d_c[global_id];
  else if (global_id % 4 == 2)
      d_a[global_id] = d_b[global_id] / d_c[global_id];
  else if (global_id % 4 == 3)
      d_a[global_id] = d_b[global_id] - d_c[global_id];
}
```

Stride Array:
First quarter: +
Second quarter: *
Third quarter: /
Fourth quarter: -

Consequences of Warps, Chunked Instructions

```
__global__ void vector_add_chunked(float * d_a, float * d_b, float * d_c, int size) {
  int global_id = (blockIdx.x * blockDim.x) + threadIdx.x;
  if (global_id < (size/4)*1)
      d_a[global_id] = d_b[global_id] + d_c[global_id];
  else if (global_id < (size/4)*2)
      d_a[global_id] = d_b[global_id] * d_c[global_id];
  else if (global_id < (size/4)*3)
      d_a[global_id] = d_b[global_id] / d_c[global_id];
  else if (global_id < (size/4)*4)
      d_a[global_id] = d_b[global_id] - d_c[global_id];
}
```

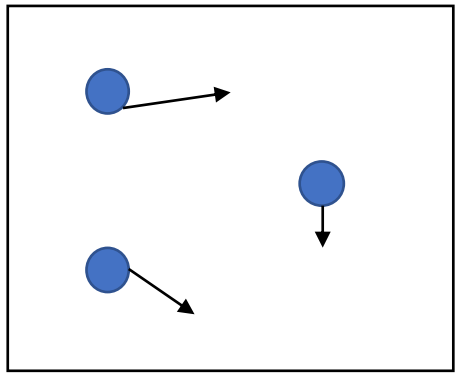
What is the right way to program GPUs?

- Still an open question!

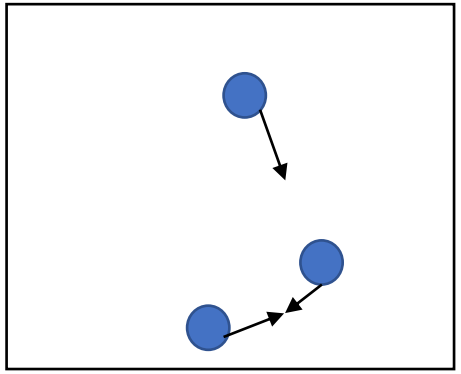
Homework 4 - first look

- Your assignment:
  - N-body simulation

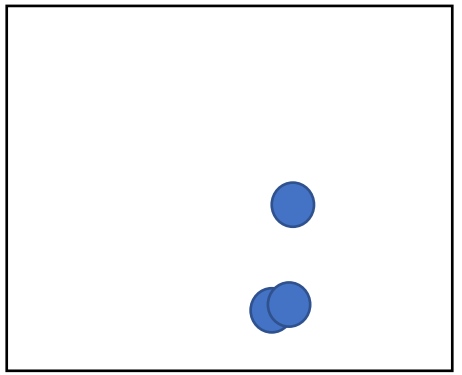
- Each particle interacts with every other particle



time = 0



time = 1

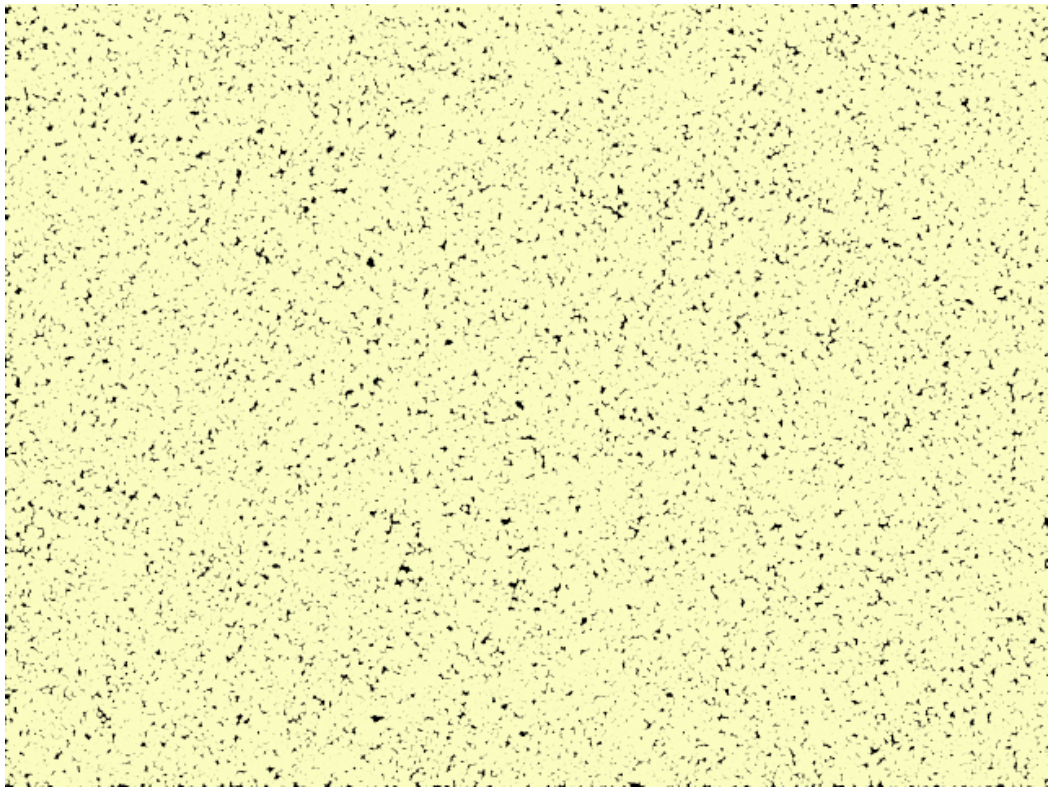


time = 2

Examples

- Gravity

- Boids (bird-oid object)
  An artificial life algorithm which simulates the flocking behavior of birds, and related group motion.
  https://en.wikipedia.org/wiki/Boids



Your homework

- N-body require a little bit of physics background so we will do something simpler.
  - If you want to explore with physics, please feel free

- **Local attraction clustering**:
  - For each particle: find your closest neighbor
  - You can take one step in the x direction and one step in the y direction towards your closest neighbor.

Your homework

- Part 1 of your homework will do this on a single javascript thread

Your homework

- Looks good, but with more particles, things start to go slower...

Your homework

- Looks good, but with more particles, things start to go slower...
- Part 2 of the homework is to implement with multiple CPU threads using javascript webworkers
  - Should get a linear speedup
- Part 3 is to implement with webGPU
  - Should get a BIG speedup!
- You need to explore how many particles you can simulate while keeping a 60 FPS (Frame Per Second) rate.

Homework 4 - first look

- Prerequisits
  - Google Chrome, Now should work on Firefox too.
  - Should be stable on Linux, Windows and Mac.

- No docker container for this assignment.

Homework 4 - first look

- Javascript shared array buffer:
  - How javascript threads can actually share memory
  - Similar to memory in C++

```
var buffer = new SharedArrayBuffer(Float32Array.BYTES_PER_ELEMENT * NUM);
var array = new Float32Array(buffer);
```

Shared Array Buffer

- Like Malloc, allocates a ”pointer” to a contagious array of bytes
- Can pass the “pointer” to different threads
- Need to instantiate a typed array to access the values

Web Workers

- How to do multi-threading in javascript

- Async
  - Concurrent (executes on the same thread)
  - Good for I/O and user interactions

- Web Workers will execute on multiple cores
  - Better for compute intensive applications
  - Better performance

How to use?

- Create a new worker with a file worker.js
  - Not a thread. It is a separate JavaScript environment with its own global scope, no access to your page's variables or the DOM, and (apart from SharedArrayBuffer) no shared memory.
  - File contains a “onmessage” function:
    “self.onmessage = function (event) { … }”

- Main file main.js
  - Create a new worker
    var worker = new Worker('./webWorker.js’);
  - Doesn’t do anything yet.
  - calls “postMessage” along with arguments to start the thread
    worker.postMessage(transferData);

- Worker sends a message back to the main file, it can catch the data
  worker.onmessage = function() { /* draw the frame*/ }

- In html file

```
<script src="main.js"></script>
<script src=”worker.js"></script>
```

WebGPU

- The language is wgsl (WebGPU Shading Language)
  - It is relatively new (and the specification changes!)
  - Official specification is here: https://www.w3.org/TR/WGSL/

WebGPU

- wgsl is NOT javascript

- Javascript is interpreted, has dynamic typing and garbage collection: not possible on GPUs.
  WGSL has static types and fixed layouts.

- The browser's compiler (Tint in Chrome, Naga in Firefox) translates
  - into SPIR-V for Vulkan on Linux and Android,
  - into MSL for Metal on Apple, and
  - into HLSL for Direct3D 12 on Windows.
- No printing (so GPU code can be difficult to debug)

WebGPU

- variables (optional types):

var <name> = <value>;

```
var cluster_dist = 3.0;
```

var <name> : <type> = <value>;

```
var cluster_dist : f32 = 3.0;
```

WebGPU

- types:
  - i32
  - u32
  - f32
  - vec2<f32>
  - array<*type*>
- structures

```
struct Particle {
    pos : vec2<f32>,
}
```

```
struct Particles {
    particles : array<Particle>,
}
```

```
var index_pos : vec2<f32> = particlesA.particles[index].pos;
```

WebGPU

- Built in functions:
  - arrayLength
  - sqrt
  - pow
  - distance

WebGPU

For loops:

```
for (var i : u32 = 0u; i < arrayLength(&particlesA.particles); i++) {
  ...
}
```

The kernel, signature

CUDA

```
__global__ void vector_add(int *d_a, int *d_b, int *d_c, int size) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
```

WGSL (updateSprite.js)

```
@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id)
      GlobalInvocationID : vec3<u32>) {
   var index : u32 = GlobalInvocationID.x;
```

What each piece means

- **@compute** marks this fn as a compute kernel, the __global__.
- **@workgroup_size(64)** threads per block, the second number in <<<_, n>>>. In WGSL it is fixed in the shader, not chosen at launch.
- **fn** declares a function. Types come after the name: name : type.
- **@builtin(global_invocation_id)** blockIdx.x * blockDim.x + threadIdx.x, computed for you. The parameter name is yours to choose.
- **vec3<u32>** blocks and grids can be 1D, 2D or 3D, so the id is a 3-vector; .x is the 1D index. Unsigned 32-bit.

Every built-in is a parameter of the entry point, declared with @builtin(...) as on the previous slide.

The kernel, thread vocabulary

CUDA	WGSL	Meaning
warp	subgroup	size depends on the GPU: 32 on Nvidia and Apple, 32 or 64 on AMD
block	workgroup	same thing, new name
grid	dispatch	one dispatchWorkgroups(…) call
threadIdx	@builtin(local_invocation_id)	thread id inside the block
blockIdx	@builtin(workgroup_id)	block id
blockDim	@workgroup_size(…) attribute	threads per block, a constant in the shader
gridDim	@builtin(num_workgroups)	number of blocks
blockIdx*blockDim+threadIdx	@builtin(global_invocation_id)	global thread id

Compile the kernel

CUDA: ahead of time, on your machine

```
$ nvcc main.cu -o main
```

WebGPU: at run time, in the browser (main.js)

```
const computePipeline = device.createComputePipeline({
  layout: 'auto',
  compute: {
    module: device.createShaderModule({ code: updateSpriteWGSL }),
    entryPoint: 'main',
  },
});
```

What each piece means

- **createShaderModule** compiles the WGSL string for whatever GPU is present, every time the page loads. Errors go to the browser console; module.getCompilationInfo() returns them to the program.
- **entryPoint** which fn to run. One module may contain several entry points.
- **layout: 'auto'** derive the argument slots from the @group/@binding lines in the WGSL.
- **createComputePipeline** a pipeline is the compiled kernel plus its argument layout. Create it once, use it every frame.

Launch

CUDA

```
vector_add<<<numBlocks, 256>>>(d_a, d_b, d_c, size);
cudaDeviceSynchronize();
```

WebGPU, every frame (main.js)

```
const commandEncoder = device.createCommandEncoder();
const passEncoder = commandEncoder.beginComputePass();
passEncoder.setPipeline(computePipeline);
passEncoder.setBindGroup(0, bindGroup);
passEncoder.dispatchWorkgroups(numBlocks);
passEncoder.end();
device.queue.submit([commandEncoder.finish()]);
```

What each piece means

- **createCommandEncoder** records commands into a list. Nothing runs while you record.
- **beginComputePass** a group of kernel launches. HW4 has one compute pass and one render pass per frame.
- **setBindGroup(0, ...)** attach the arguments: group 0 gets this bind group.
- **dispatchWorkgroups(n)** the launch. n is the number of blocks, the first number in <<<n, _>>>; threads per block came from @workgroup_size.
- **submit** now it runs, asynchronously, like a CUDA launch. Commands on the queue run in order, so the render pass recorded after this one sees the new positions. To make the CPU wait: await device.queue.onSubmittedWorkDone().

WebGPU

- Types can be complicated

- But compiler errors will help you, and you can do casts.

Moving on to memory consistency!

- One of my favorite topics!

Moving on to memory consistency!

- One of my favorite topics!
- What do other people think?

> *Look, memory ordering pretty much _is_ the rocket science of CS,*
>
> Linus Torvalds

Memory Consistency

- We have been very strict about using atomic types in this class

- why?
  - Architectures do very strange things with memory loads and stores
  - Compilers do too (but we won't talk too much about them today)
- C++
  - C++ gives us **sequential consistency if we use atomic types** and operations.
  - What do we remember sequential consistency from?

SC

- Data Structures
  - Why wasn't SC very good for concurrent data structures?
  - Compossibility: two objects that are SC might not be SC when used together
- Memory
  - Programs contain only 1 shared memory though; no reason to compose different main memories.

Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

Thread 0:

```
x.store(1);
y.store(1);
```

Is it possible for
`t0 == 0 and t1 == 1`



Thread 1:

```
int t0 = y.load();
int t1 = x.load();
```

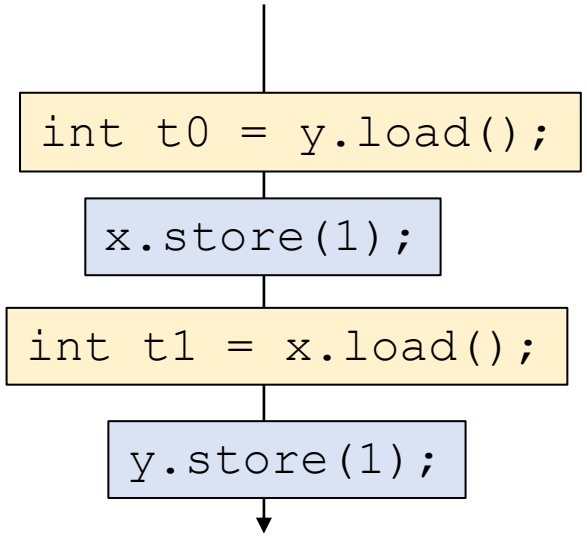
Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

Thread 0:

```
x.store(1);
y.store(1);
```

Is it possible for
`t0 == 0 and t1 == 1`



Thread 1:

```
int t0 = y.load();
int t1 = x.load();
```

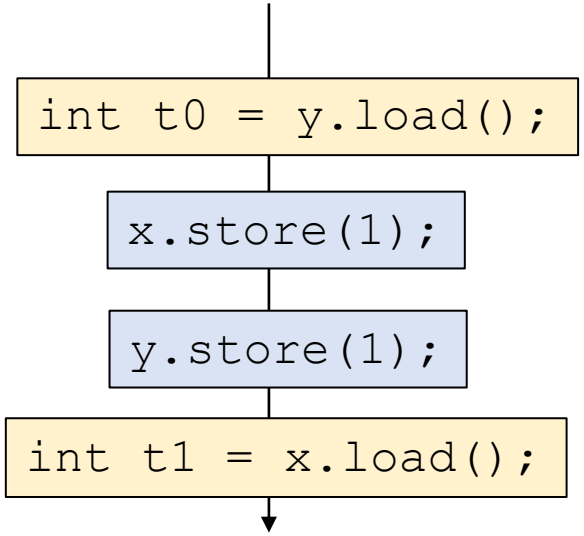
Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

Thread 0:

```
x.store(1);
y.store(1);
```

Is it possible for
`t0 == 0 and t1 == 1`



Thread 1:

```
int t0 = y.load();
int t1 = x.load();
```

Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

How about:

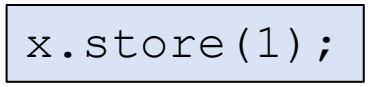
Is it possible for
`t0 == 1 and t1 == 0`

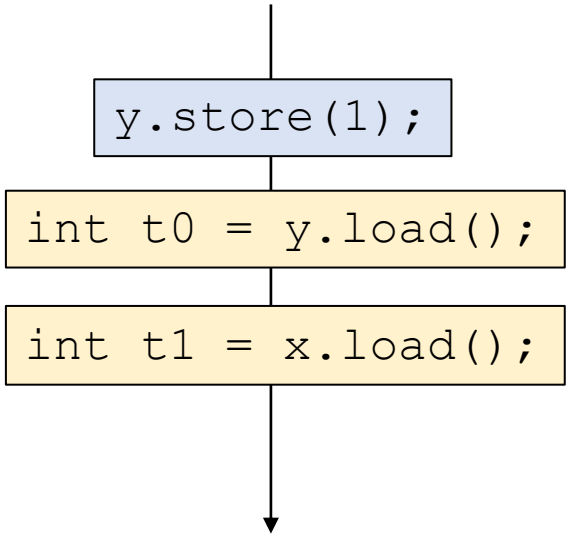
Thread 0:

```
x.store(1);
y.store(1);
```

Thread 1:

```
int t0 = y.load();
int t1 = x.load();
```





Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

How about:

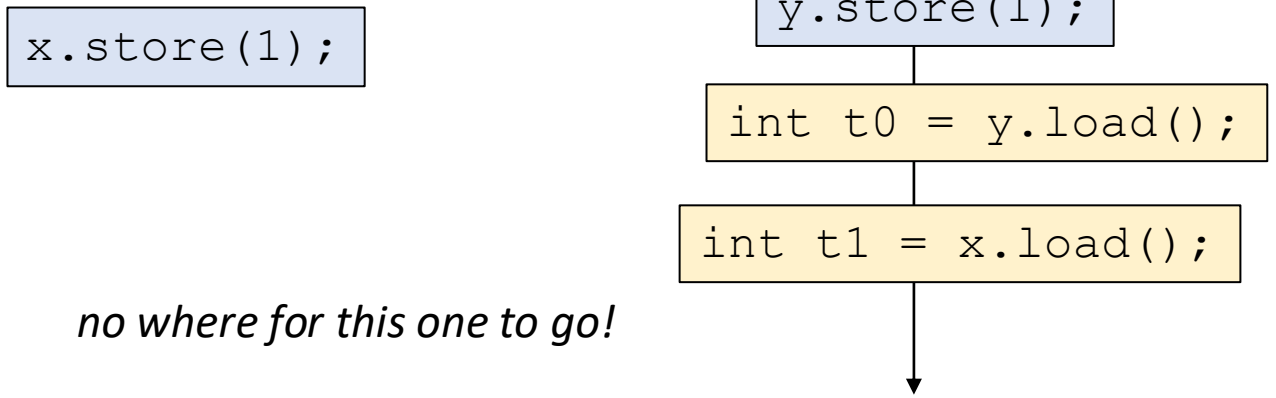
Is it possible for
`t0 == 1 and t1 == 0`

Thread 0:

```
x.store(1);
y.store(1);
```

Thread 1:

```
int t0 = y.load();
int t1 = x.load();
```



no where for this one to go!

Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

Dekker mutual exclusion:
Another test
Can `t0 == t1 == 0`?

Thread 0:

```
x.store(1);
int t0 = y.load();
```


Thread 1:

```
y.store(1);
int t1 = x.load();
```

Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

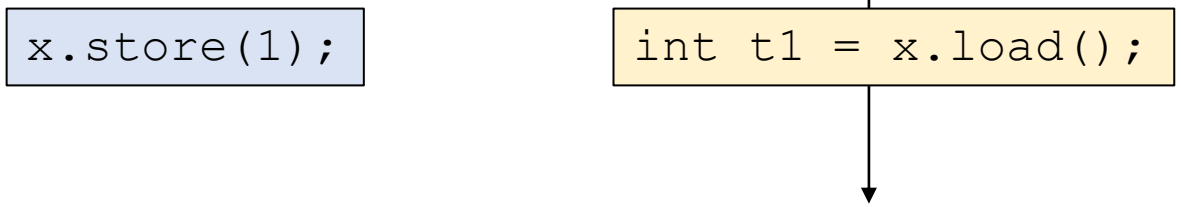
Another test
Can `t0 == t1 == 0`?

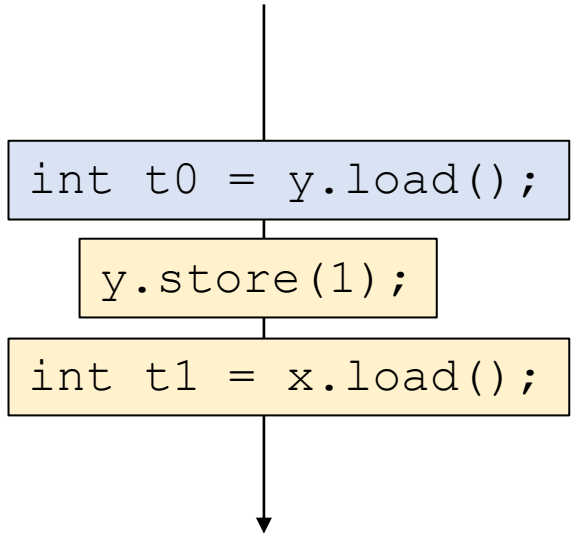
Thread 0:

```
x.store(1);
int t0 = y.load();
```

Thread 1:

```
y.store(1);
int t1 = x.load();
```





Global variable:

```
atomic_int x(0);
atomic_int y(0);
```

Another test

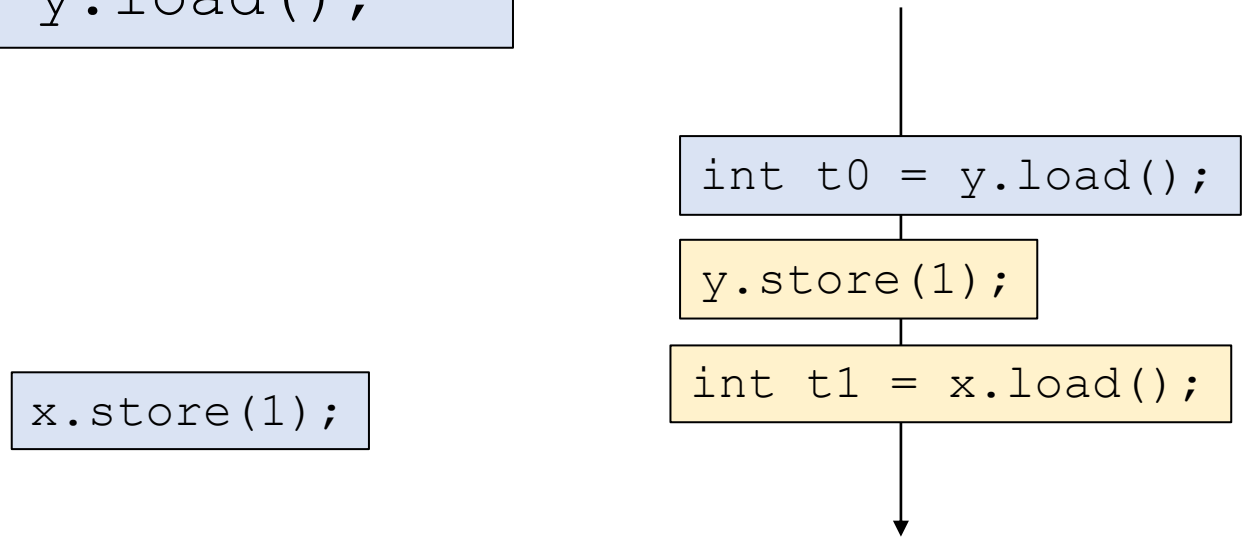
Can `t0 == t1 == 0`?

Thread 0:

```
x.store(1);
int t0 = y.load();
```

Thread 1:

```
y.store(1);
int t1 = x.load();
```



no place for this one!

SC

- Atomic accesses are sequentially consistent (SC)
- Atomics load and store are not plain **ISA instructions**.

What about ISAs?

- Remember, it is important for us to understand how our code executes on the architecture to write high performing programs

- Let's think about x86
  - Instructions:
  - `MOV %t0 [x]` - loads the value at x to register t0
  - `MOV [y] 1` - stores the value 1 to memory location y

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

Can `t0 == t1 == 0`?

Thread 0:

```
mov [x], 1
mov %t0, [y]
```


Thread 1:

```
mov [y], 1
mov %t1, [x]
```

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

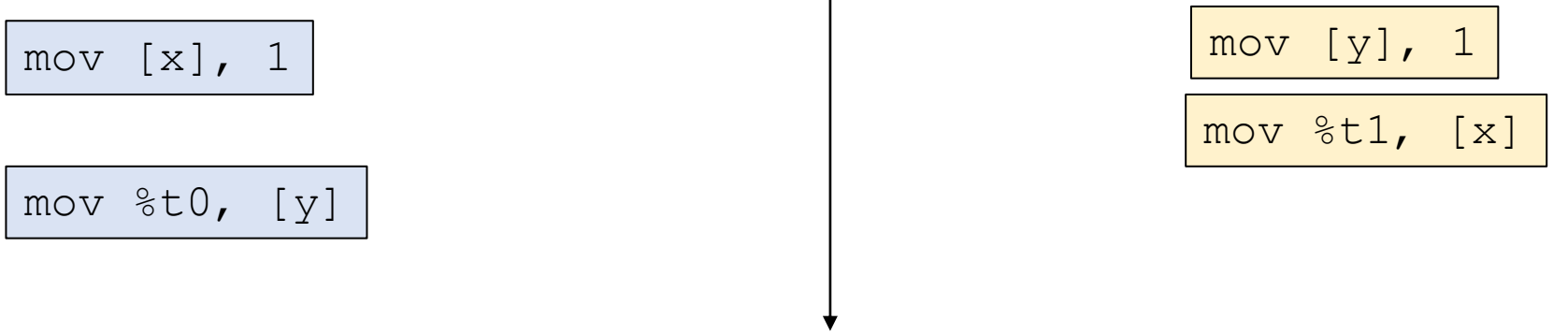
Can `t0 == t1 == 0`?

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mov %t1, [x]
```



Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

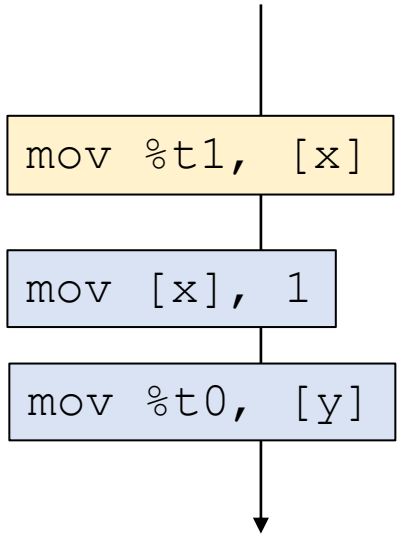
Can `t0 == t1 == 0`?

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mov %t1, [x]
```




```
mov [y], 1
```

no place for this event!

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

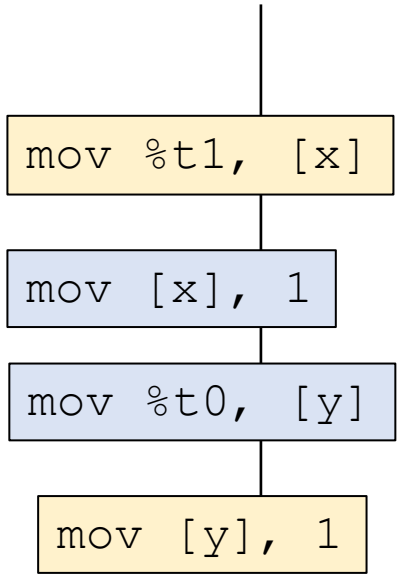
Can `t0 == t1 == 0`?

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mov %t1, [x]
```



ISA is not SC

- We’d like to be able to compile atomic instructions just to regular ISA loads and stores
- What if we actually run this code?
- Demo

Schedule

- Memory consistency models:
  - **Total store order**
  - Relaxed memory consistency

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Core 0

Thread 1:

```
mov [y], 1
mov %t1, [x]
```

Core 1



Thread 0:

```
mov %t0, [y]
```

Core 0

```
mov [x], 1
```

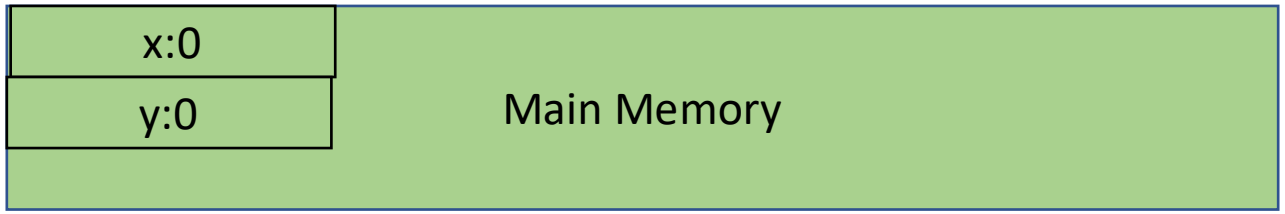
execute first instruction
what happens to the stores?

Thread 1:

```
mov %t1, [x]
```

Core 1

```
mov [y], 1
```



Thread 0:

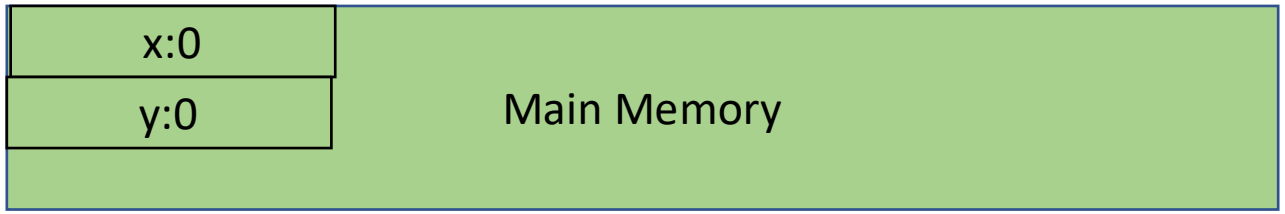
Thread 1:

X86 cores contain a store buffer; holds stores before going to main memory

```
mov %t0, [y]
```

```
mov %t1, [x]
```





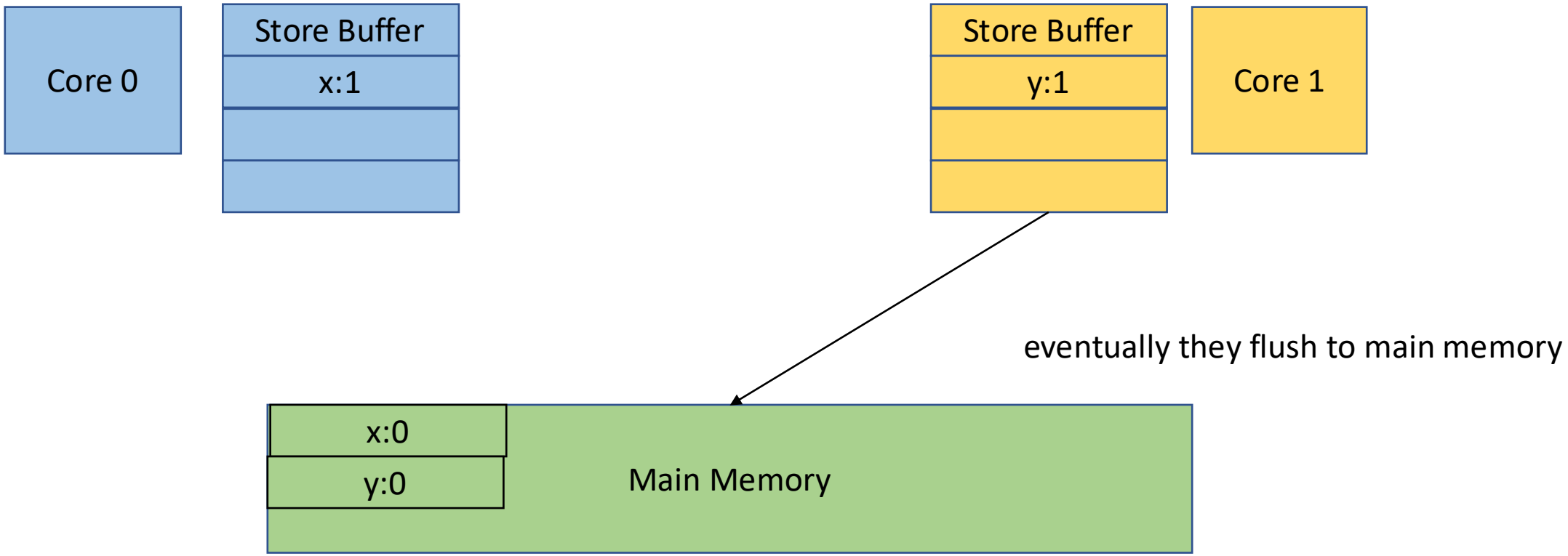
Thread 0:

Thread 1:

X86 cores contain a store buffer; holds stores before going to main memory

```
mov %t0, [y]
```

```
mov %t1, [x]
```



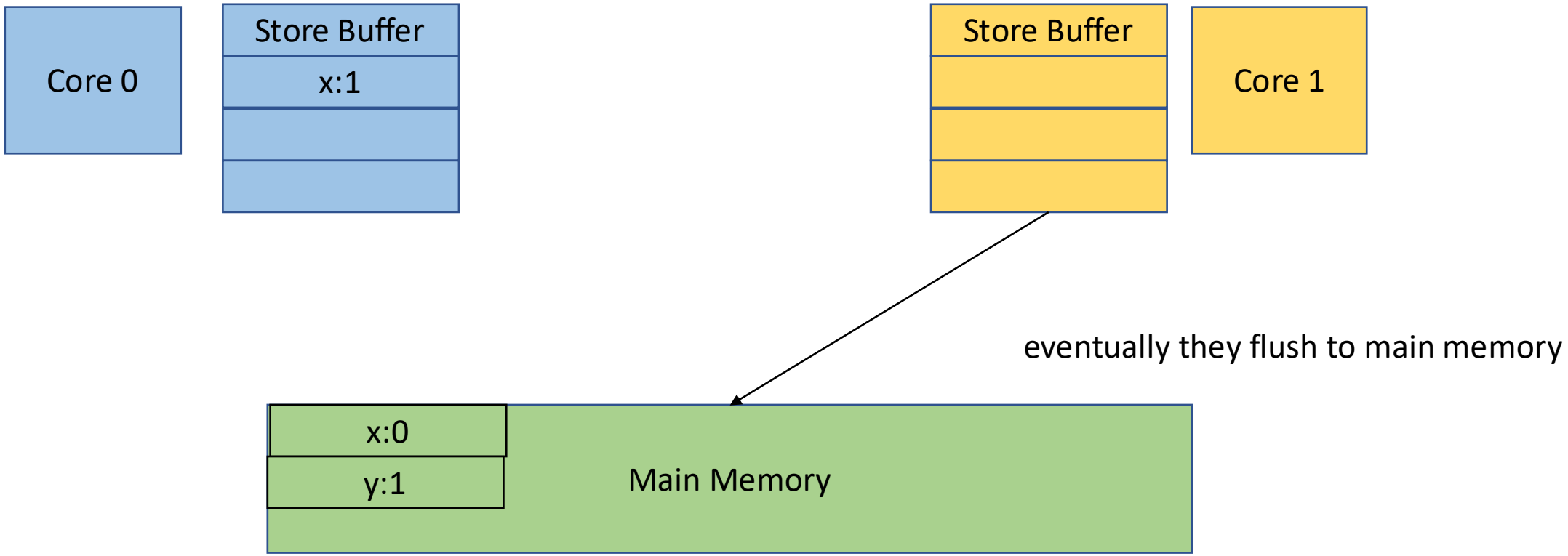
Thread 0:

Thread 1:

X86 cores contain a store buffer; holds stores before going to main memory

```
mov %t0, [y]
```

```
mov %t1, [x]
```



Thread 0:

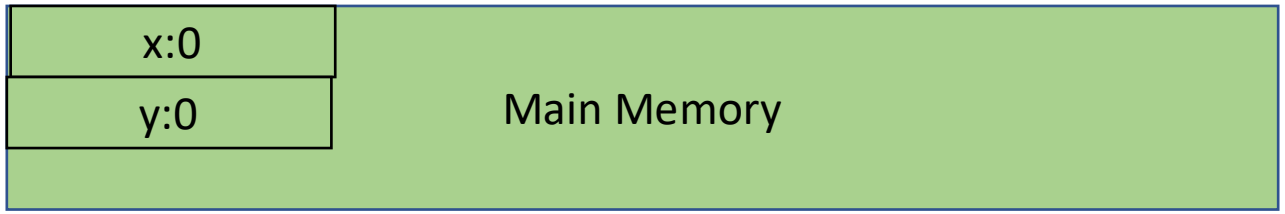
```
mov [x], 1
mov %t0, [y]
```

rewind

Thread 1:

```
mov [y], 1
mov %t1, [x]
```





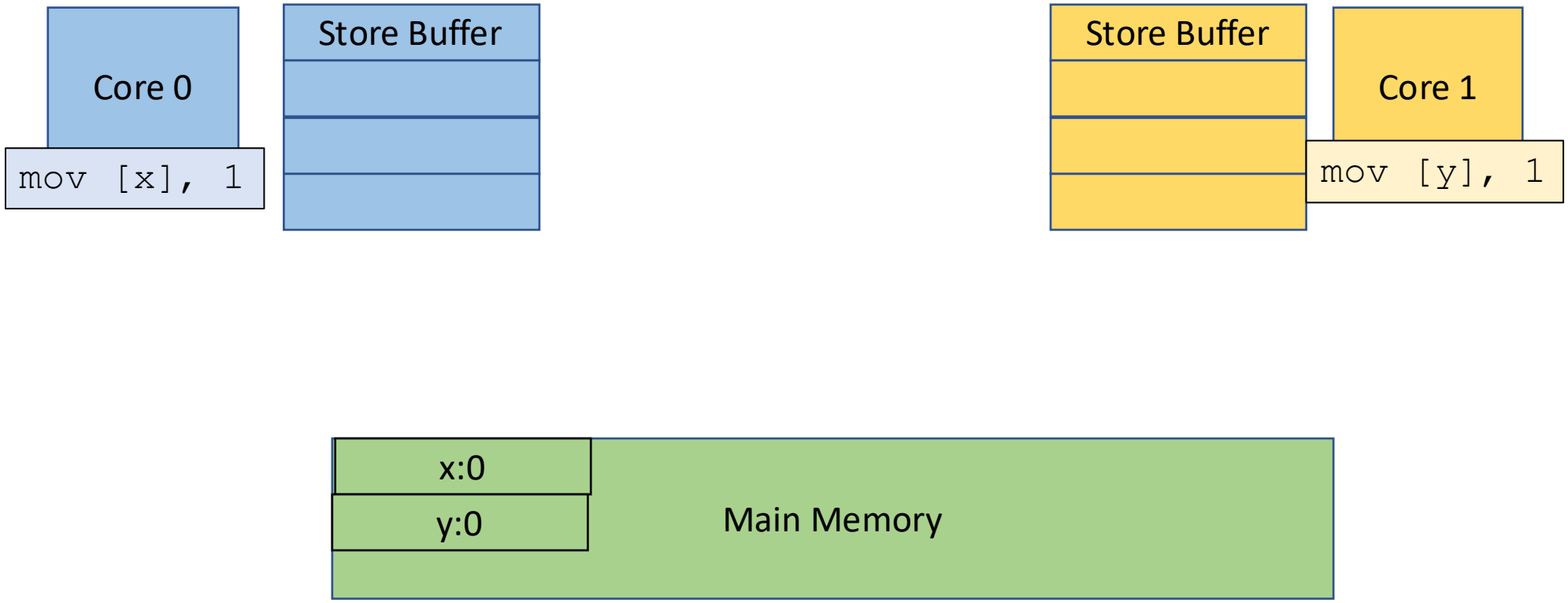
Thread 0:

Thread 1:

```
mov %t0, [y]
```

execute first instruction

```
mov %t1, [x]
```



Thread 0:

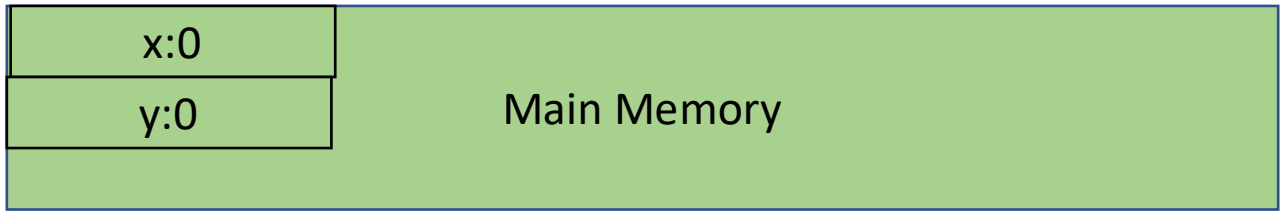
Thread 1:

```
mov %t0, [y]
```

values get stored in SB

```
mov %t1, [x]
```



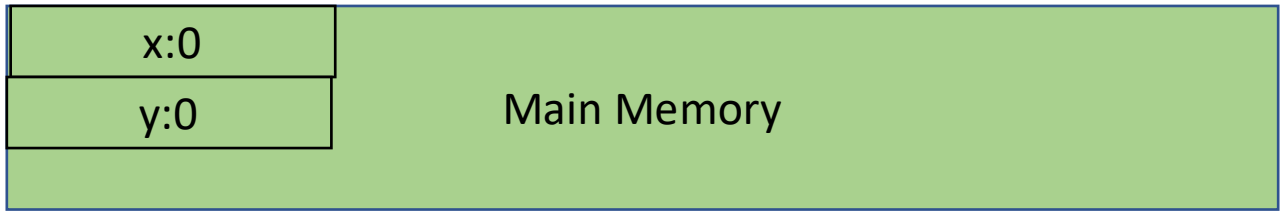


Thread 0:

Thread 1:

Execute next instruction

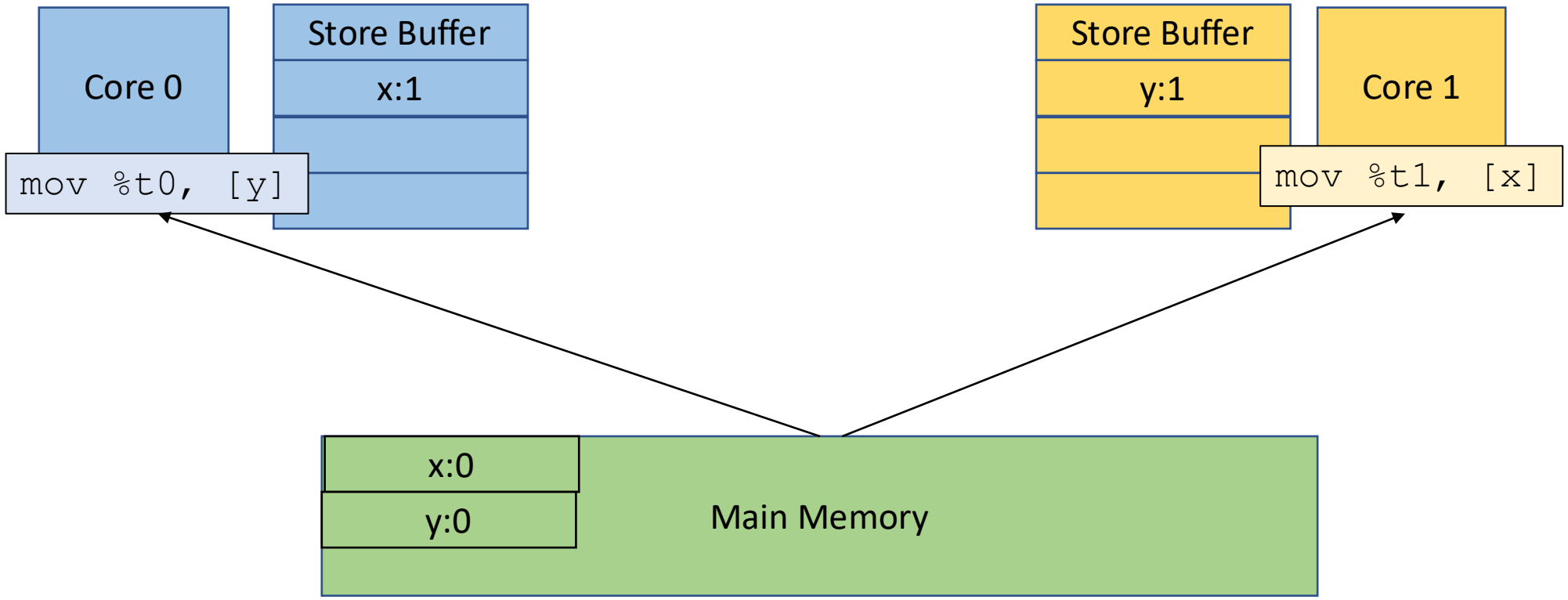




Thread 0:

Thread 1:

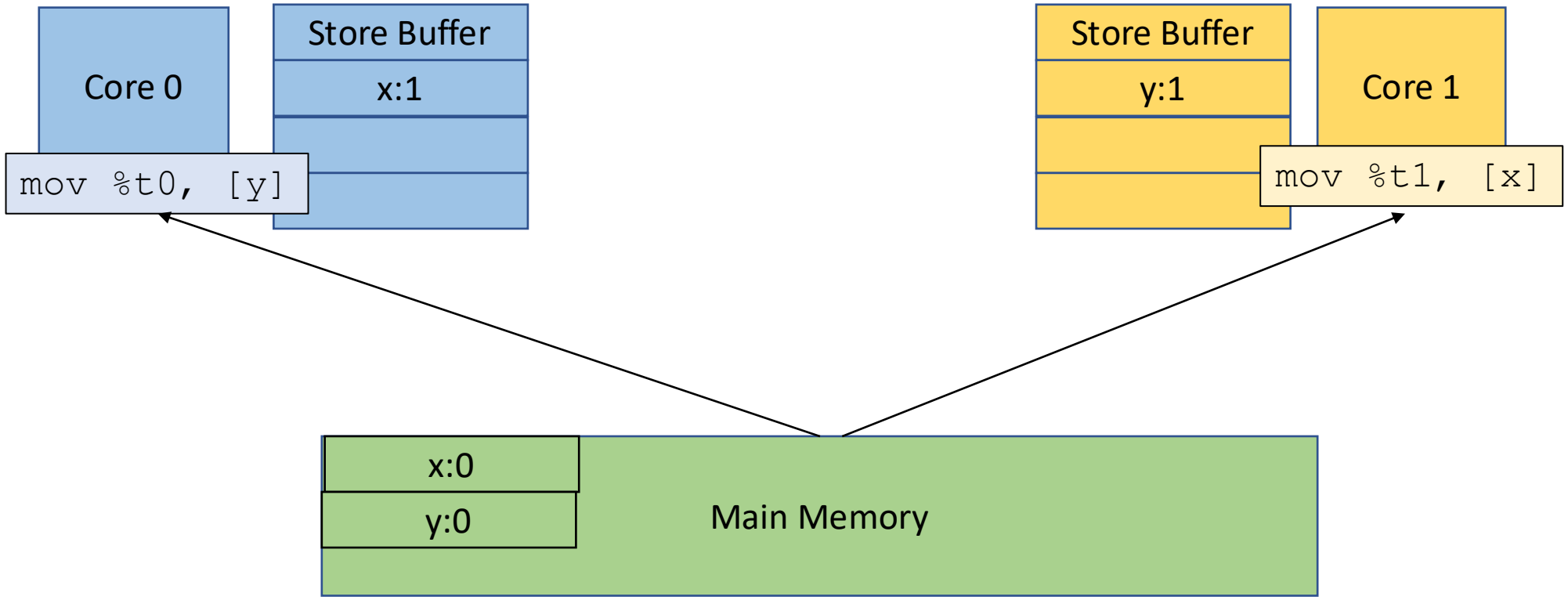
Values get loaded from memory



Thread 0:

Thread 1:

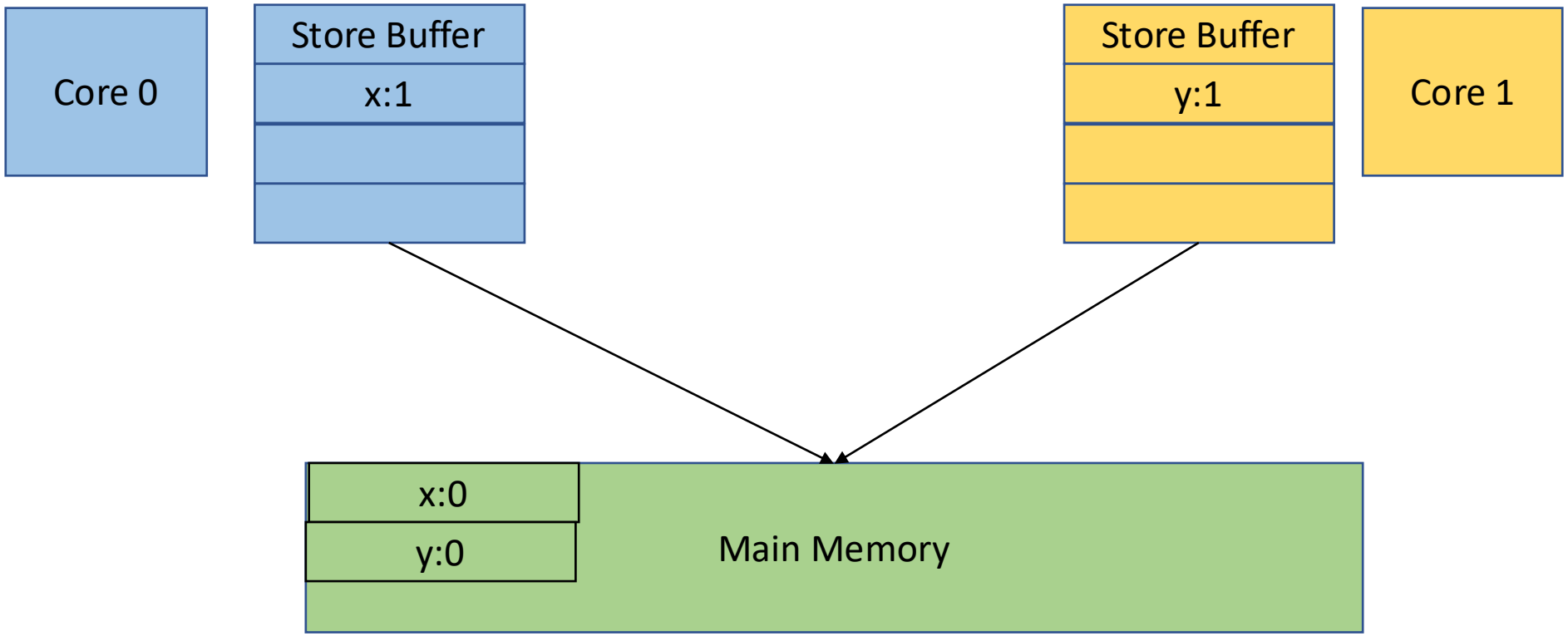
we see `t0 == t1 == 0`!



Thread 0:

Thread 1:

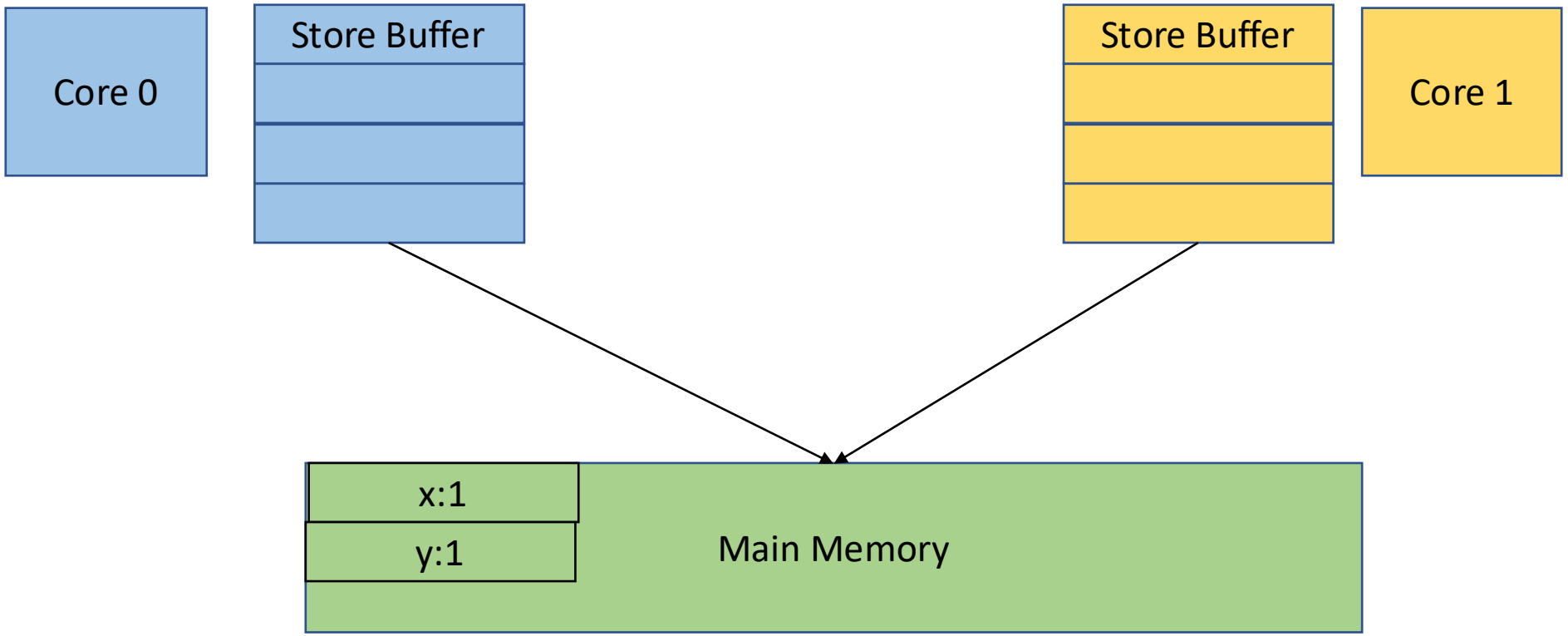
Store buffers are drained eventually



Thread 0:

Thread 1:

Store buffers are drained eventually
but we've already done our loads



Our first relaxed memory execution!

- **Relaxed memory** (also known as **Weak memory)** behaviors
- An execution that is **NOT allowed by sequential consistency**
- **Relaxed memory model**: a memory model that allows relaxed memory executions
  - **X86** has a relaxed memory model due to store buffering
  - If you restrict yourself to use only default atomic operations, **C++** does NOT have a weak memory model

Litmus tests

- Small concurrent programs that check for relaxed memory behaviors
- Vendors have a long history of under documented memory consistency models
- Academics have empirically explored the memory models
  - X86 behaviors were documented by researchers before Intel!
  - Many vendors have unofficially endorsed academic models

Litmus tests

This test is called “store buffering”

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mov %t1, [x]
```

Can `t0 == t1 == 0`?

Fences: restoring sequential consistency

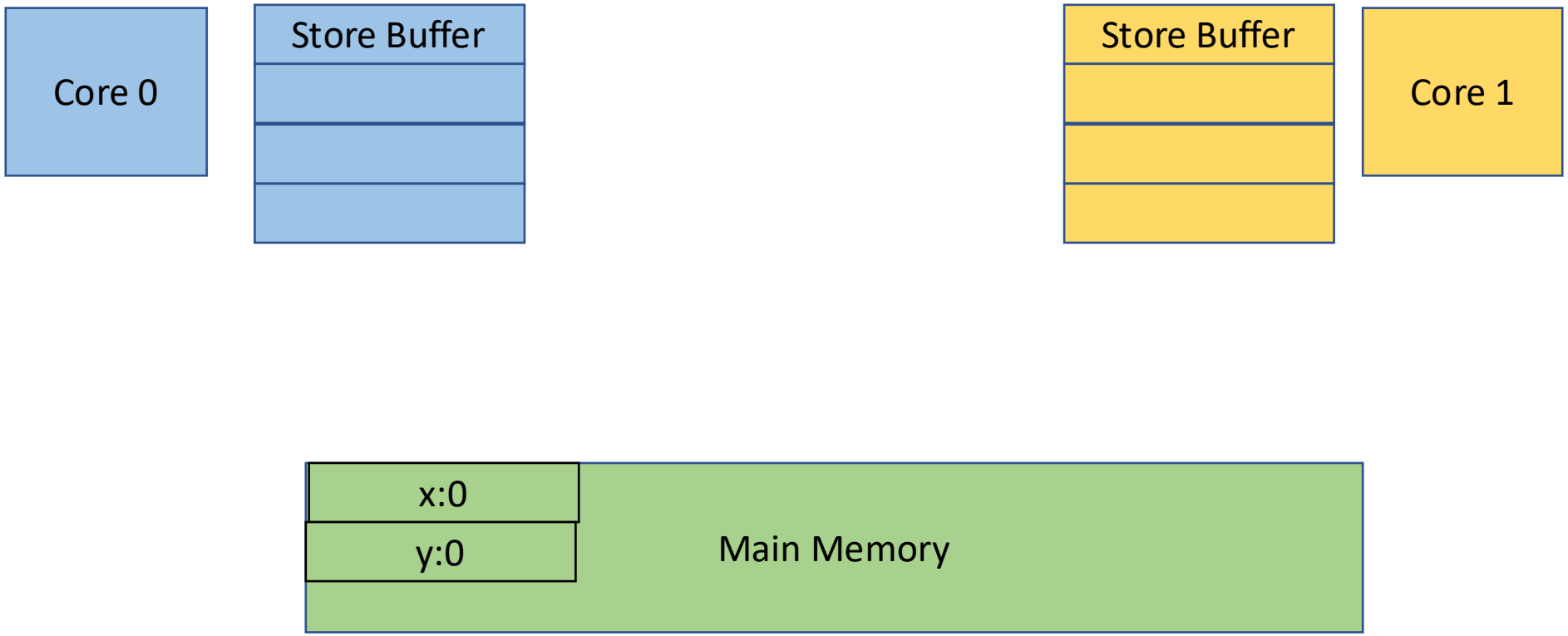
- It is typical that relaxed memory models provide special instructions which can be used to disallow weak behaviors.
- These instructions are called Fences
- The X86 fence that flushes the store buffer is called `mfence`.

Thread 0:

```
mov [x], 1
mfence
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mfence
mov %t1, [x]
```



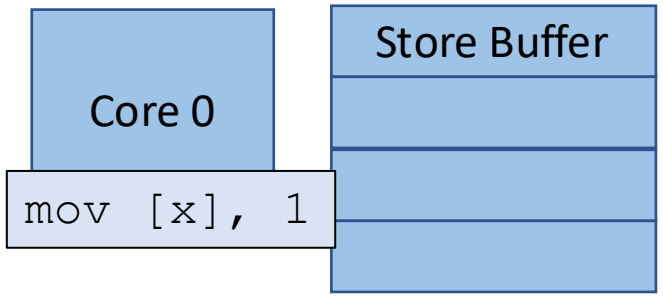
Thread 0:

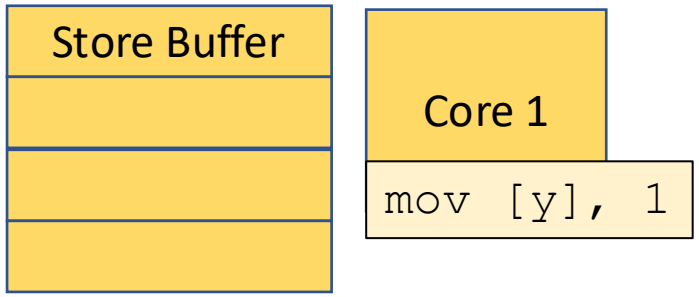
Thread 1:

```
mfence
mov %t0, [y]
```

Execute first instruction

```
mfence
mov %t1, [x]
```







Thread 0:

Thread 1:

```
mfence
mov %t0, [y]
```

Values go into the store buffer

```
mfence
mov %t1, [x]
```





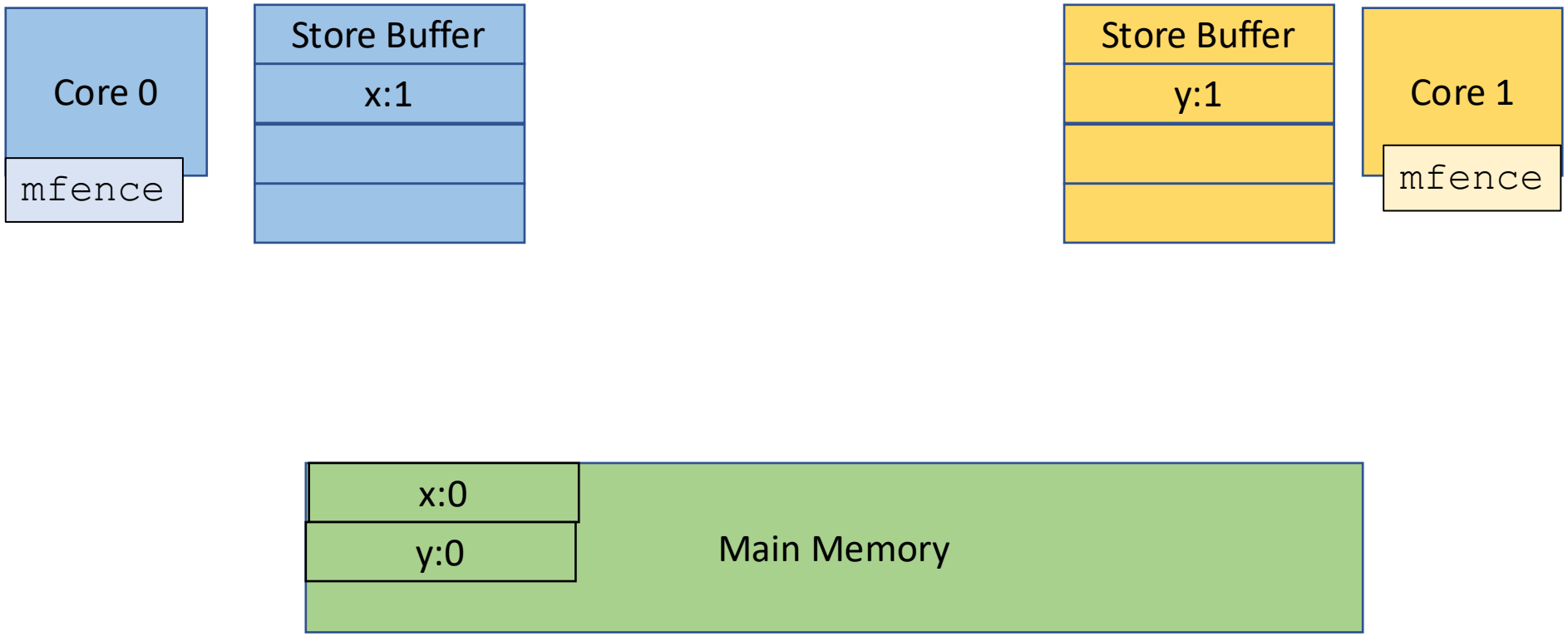
Thread 0:

Thread 1:

Execute next instruction

```
mov %t0, [y]
```

```
mov %t1, [x]
```



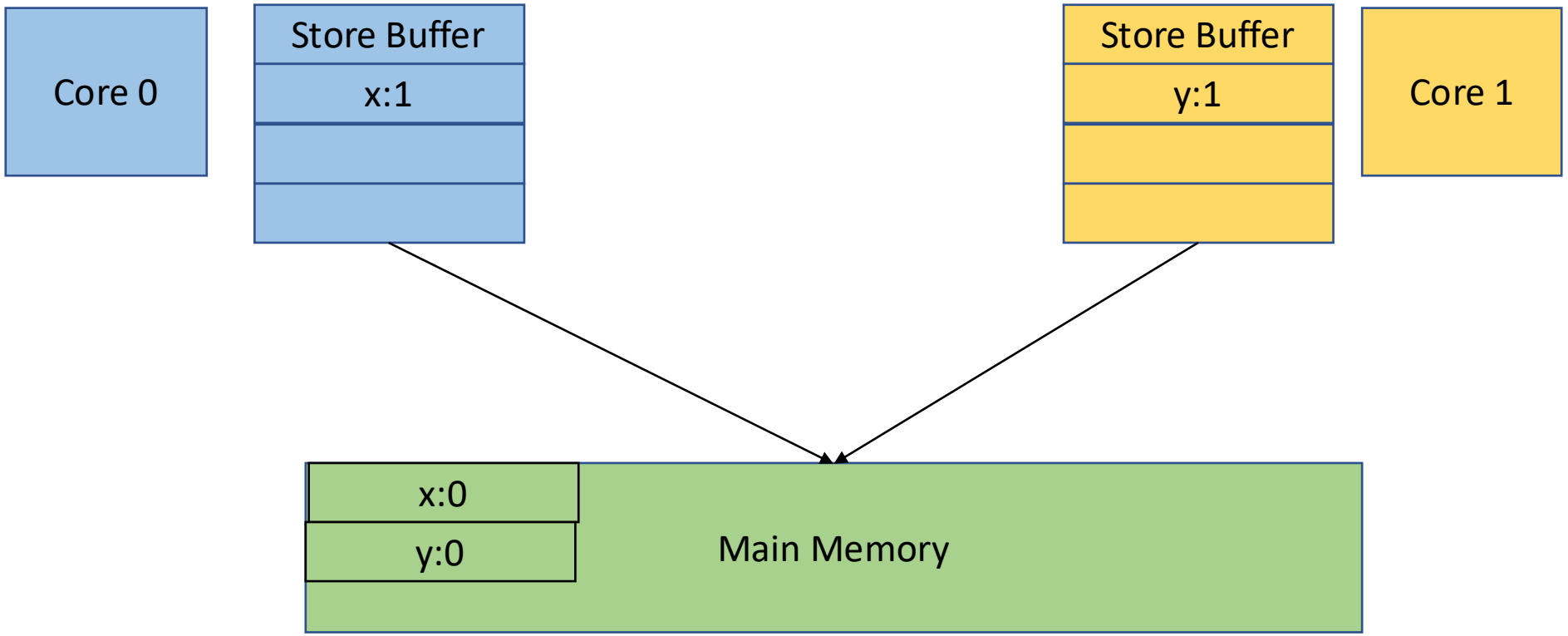
Thread 0:

Thread 1:

store buffers are flushed

```
mov %t0, [y]
```

```
mov %t1, [x]
```



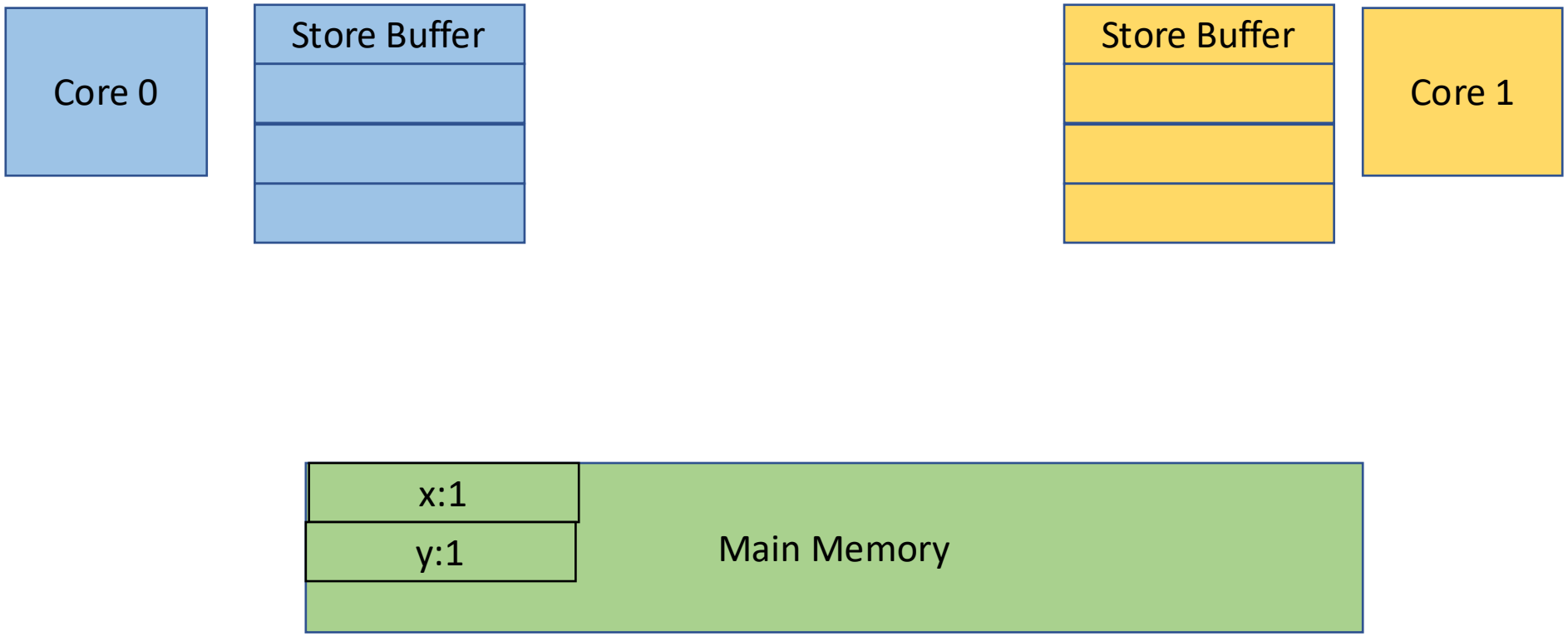
Thread 0:

Thread 1:

store buffers are flushed

```
mov %t0, [y]
```

```
mov %t1, [x]
```



Thread 0:

Thread 1:

execute next instruction

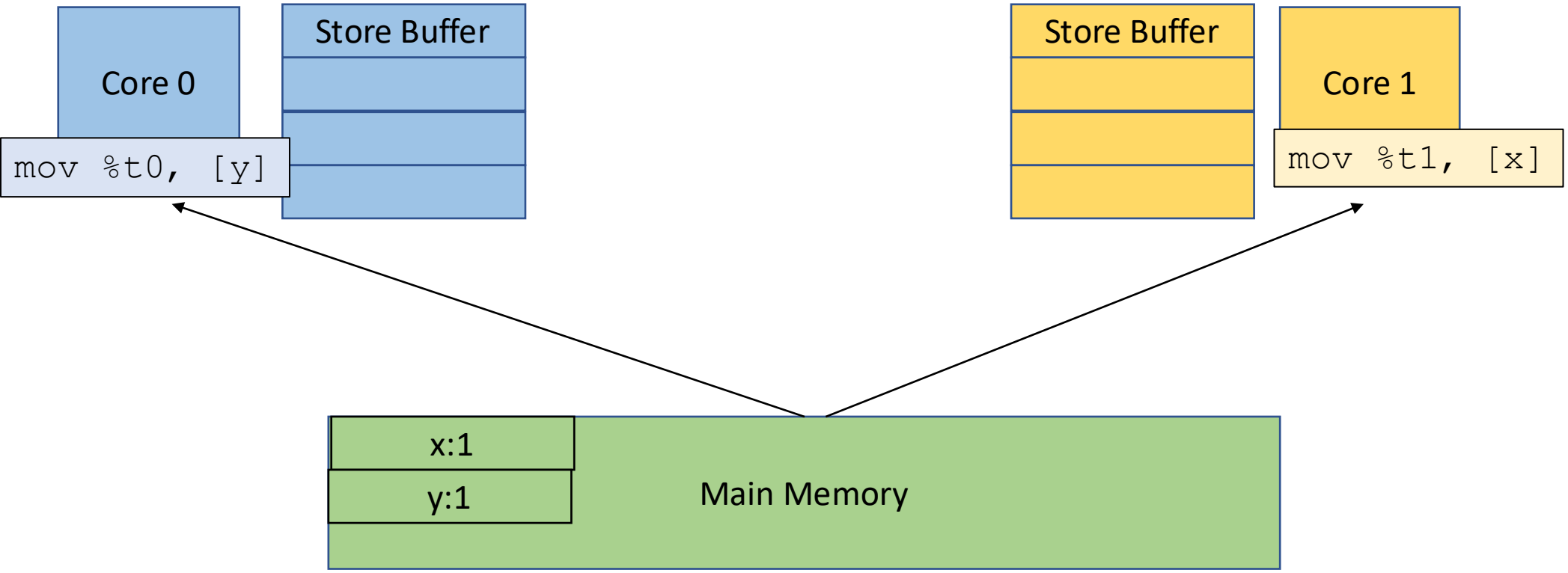




Thread 0:

Thread 1:

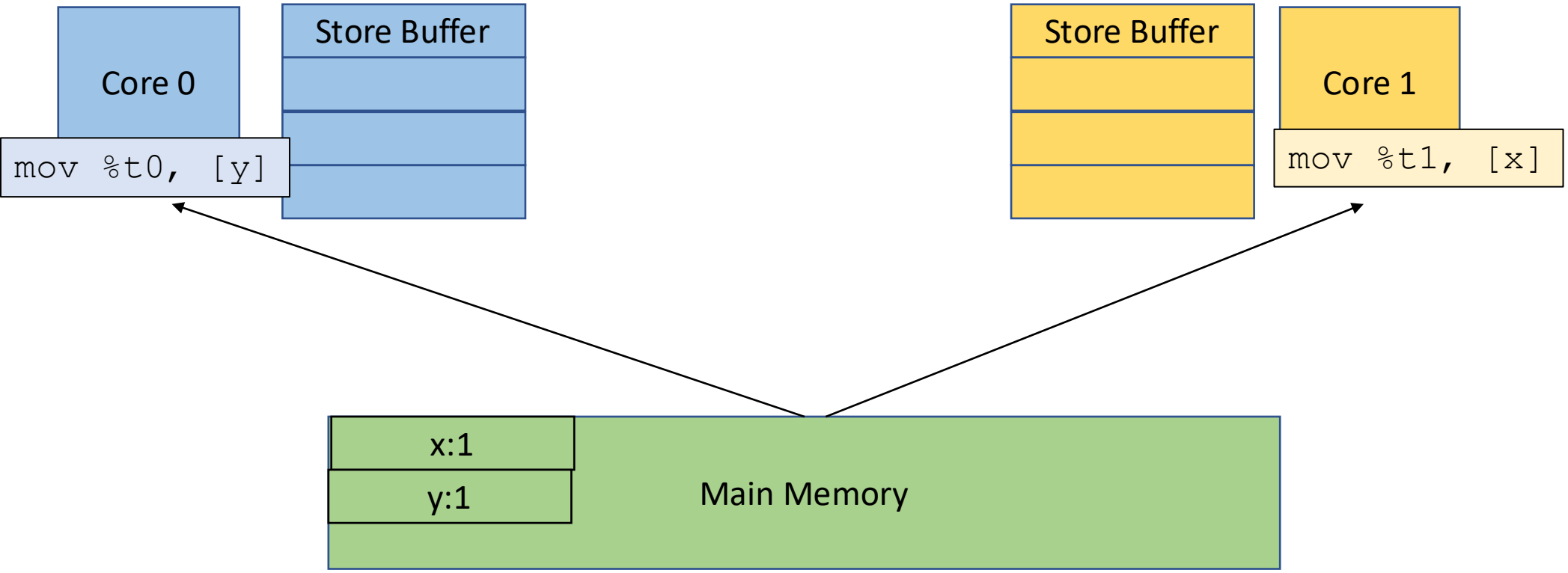
values are loaded from memory



Thread 0:

Thread 1:

We don't get the problematic behavior: `t0 == t1 == 0`



Thread 0:

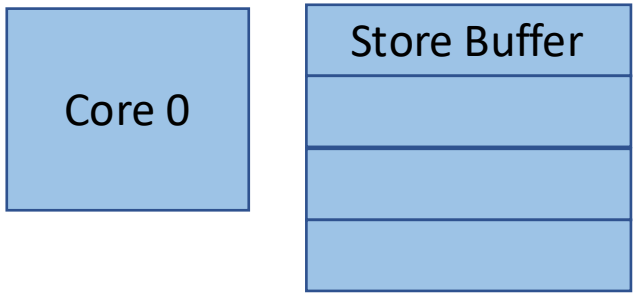
```
mov [x], 1
```

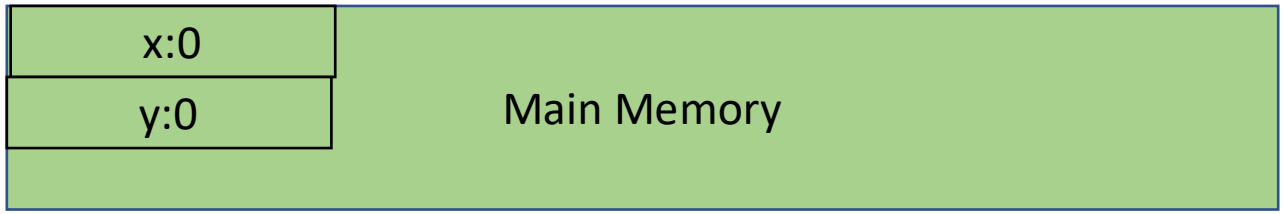
```
mov %t0, [x]
```

single thread
same address

possible outcomes:
t0 = 1
t0 = 0

Which one do you expect?



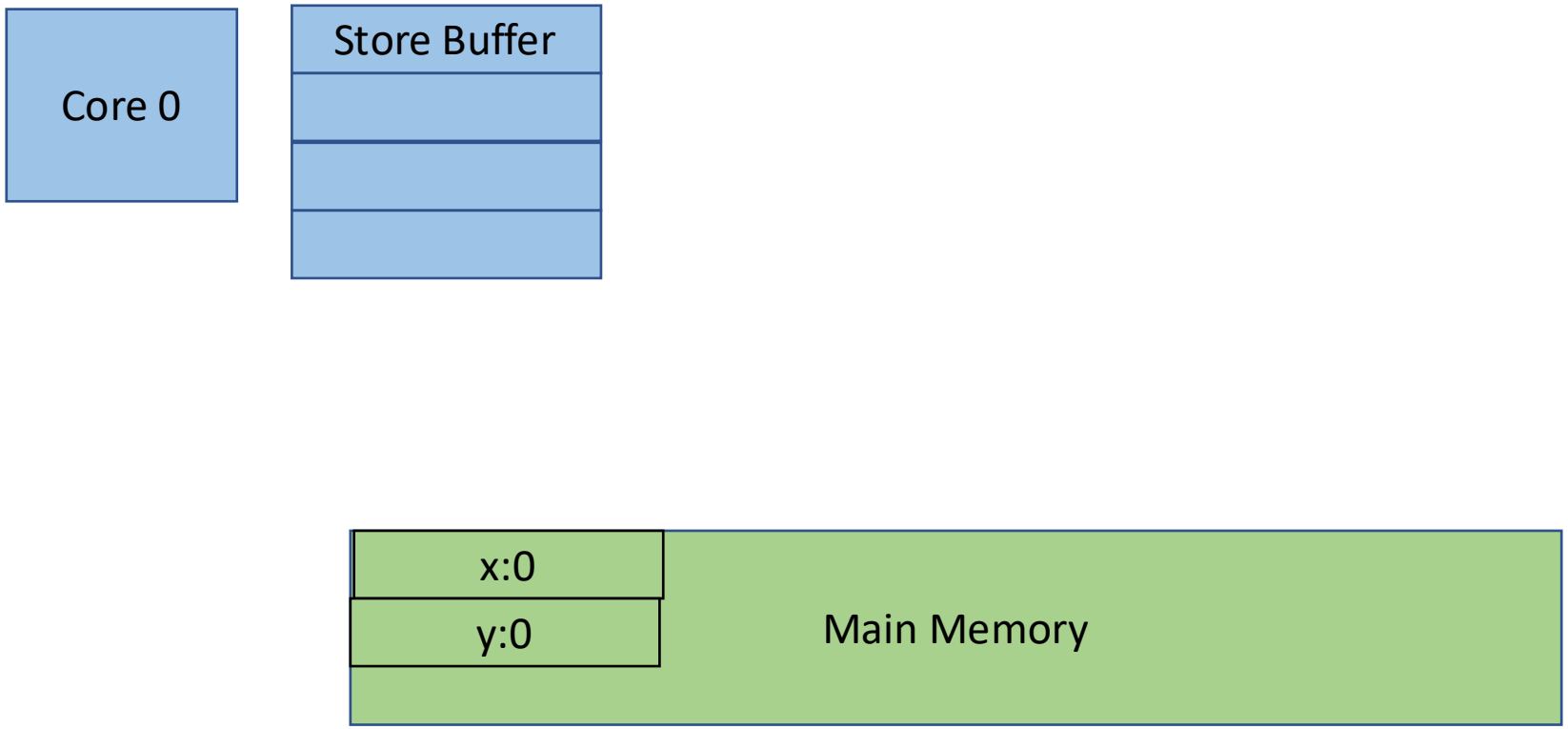


Thread 0:

```
mov [x], 1
```

```
mov %t0, [x]
```

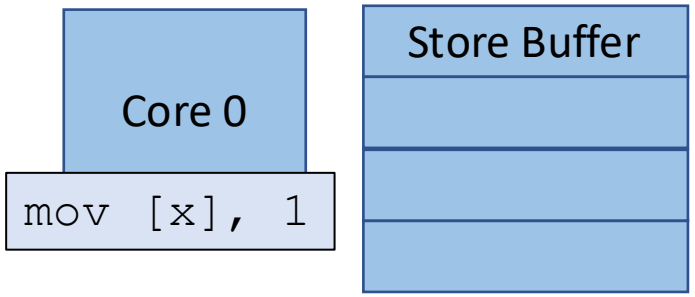
How does this execute?

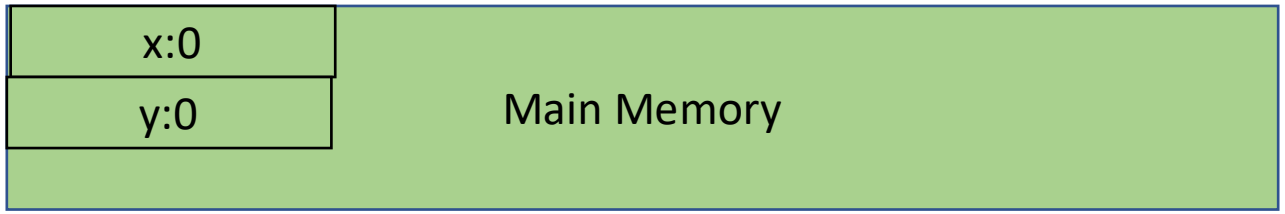


Thread 0:

execute first instruction

```
mov %t0, [x]
```

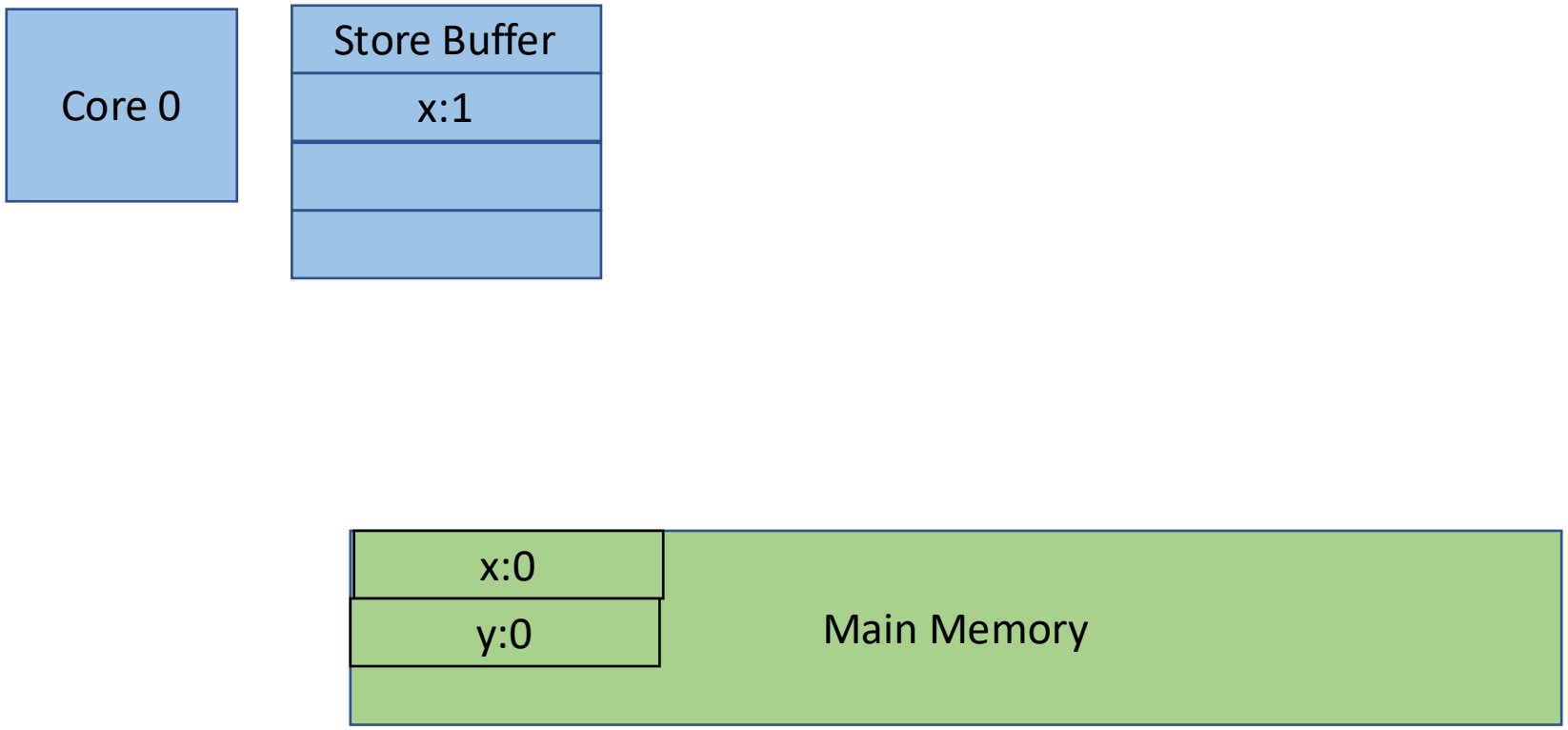




Thread 0:

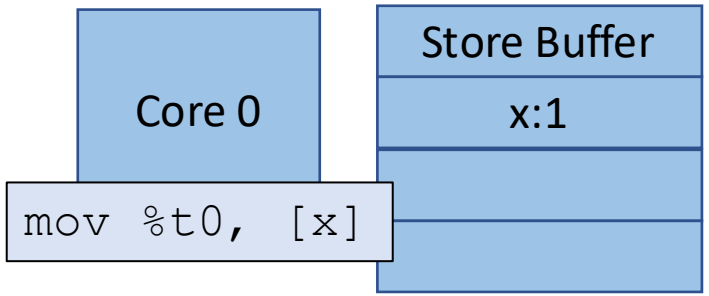
Store the value in the store buffer

```
mov %t0, [x]
```



Thread 0:

Next instruction

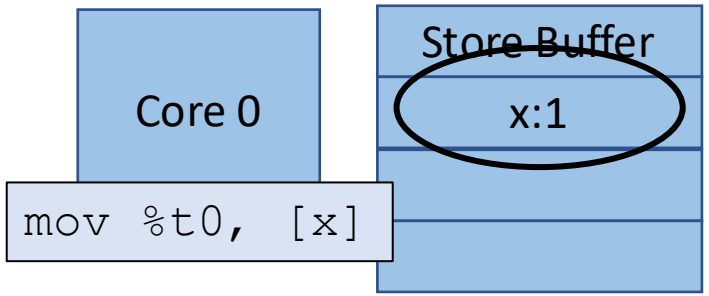


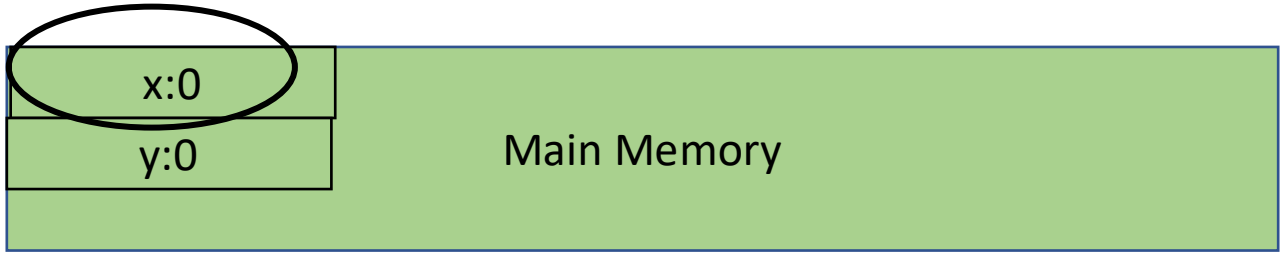


Thread 0:

Where to load??

Store buffer?
Main memory?



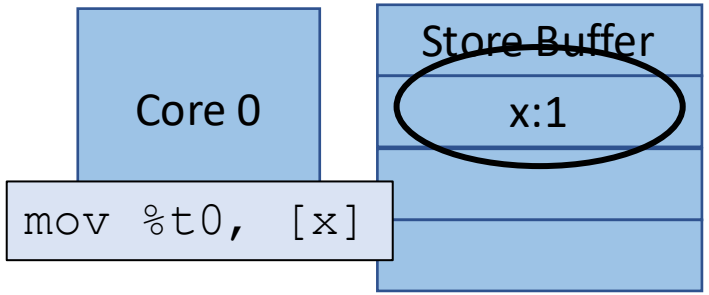


Thread 0:

Where to load??

Threads check store buffer before going to main memory

It is close and cheap to check.





Memory Consistency

- How to specify a relaxed memory model?

- We can do it operationally
  - by constructing a high-level machine and reasoning about operations through the machine.

  - or we can talk about instructions that are allowed to “break” program order.

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == t1 == 0`?

Thread 0:

```
mov [x], 1
mov %t0, [y]
```

Thread 1:

```
mov [y], 1
mov %t1, [x]
```


We will annotate instructions with S for store, and L for loads

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

Can `t0 == t1 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```


We will annotate instructions with S for store, and L for loads

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

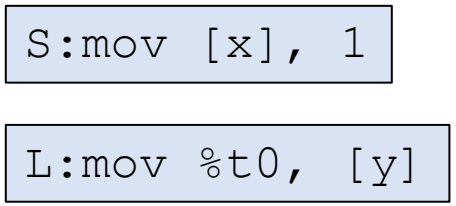
Can `t0 == t1 == 0`?

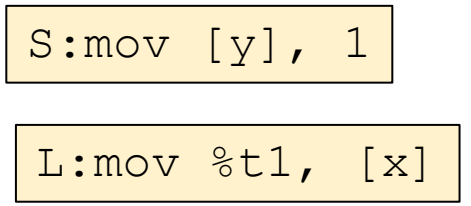
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```





Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

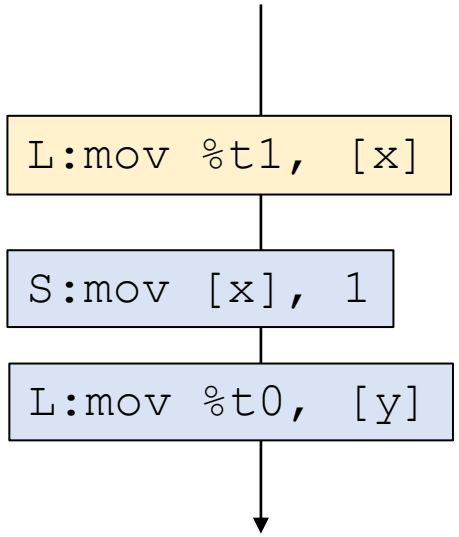
Another test
Can `t0 == t1 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```




```
S:mov [y], 1
```

Now we make a new rule:

S(tores) followed by a L(oad)
do not have to follow program order

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == t1 == 0`?

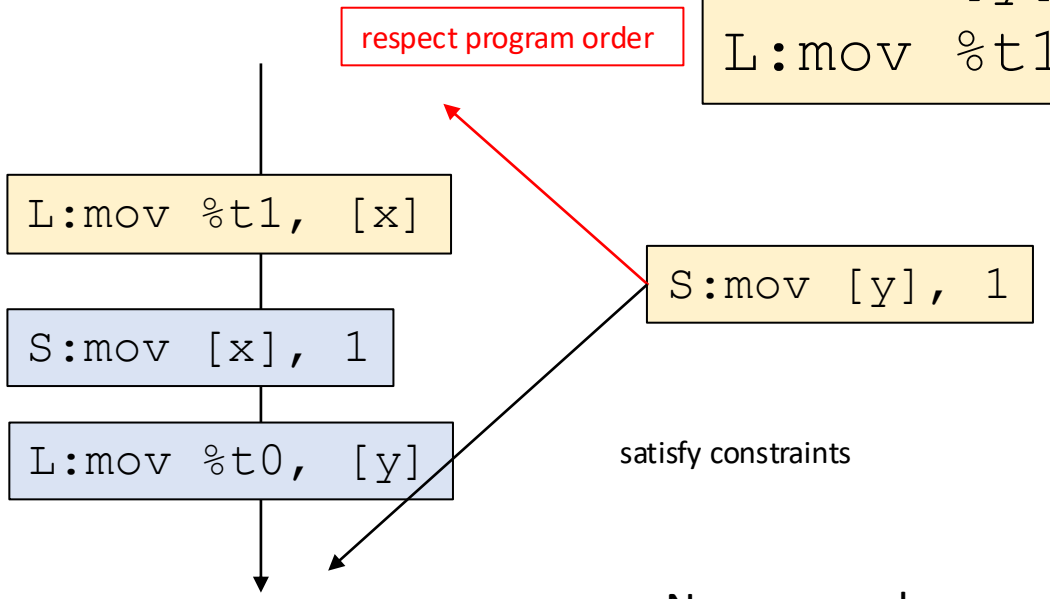
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

we can ignore this condition!!

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```



Now we make a new rule:

S(tores) followed by a L(oad)
do not have to follow program order

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

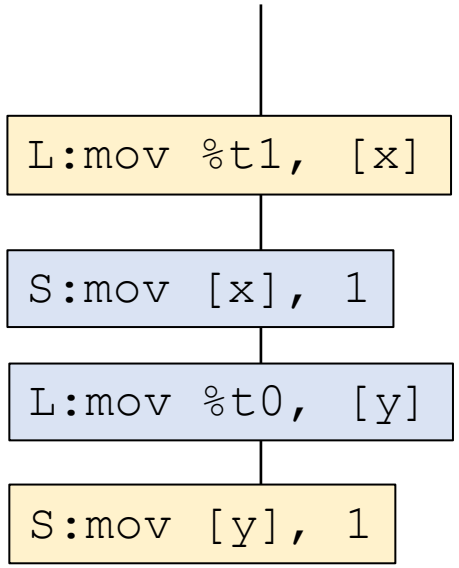
Can `t0 == t1 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```



Now we can satisfy the condition!

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

Can `t0 == t1 == 0`?

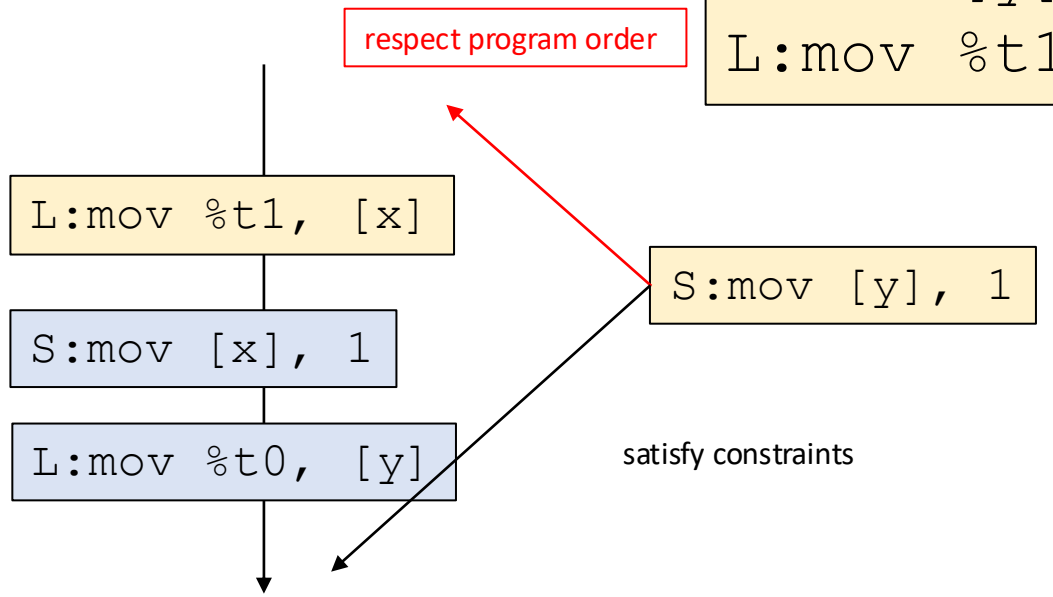
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```

we can ignore this condition!!



Let's peak under the hood here

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == t1 == 0`?

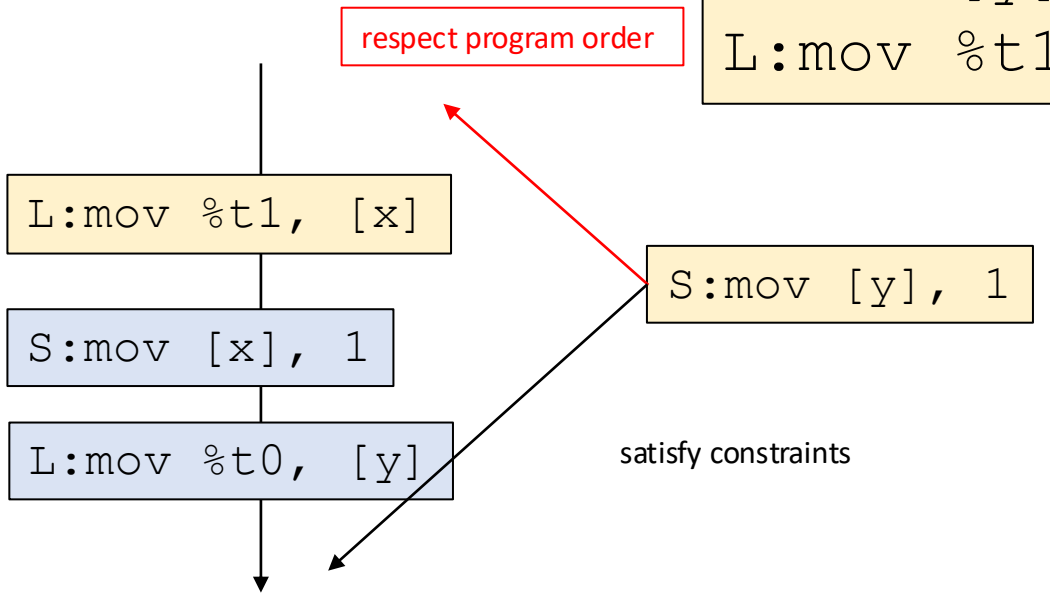
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```

we can ignore this condition!!



Let's peak under the hood here

Global timeline is when the Store operation becomes visible to other threads

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == t1 == 0`?

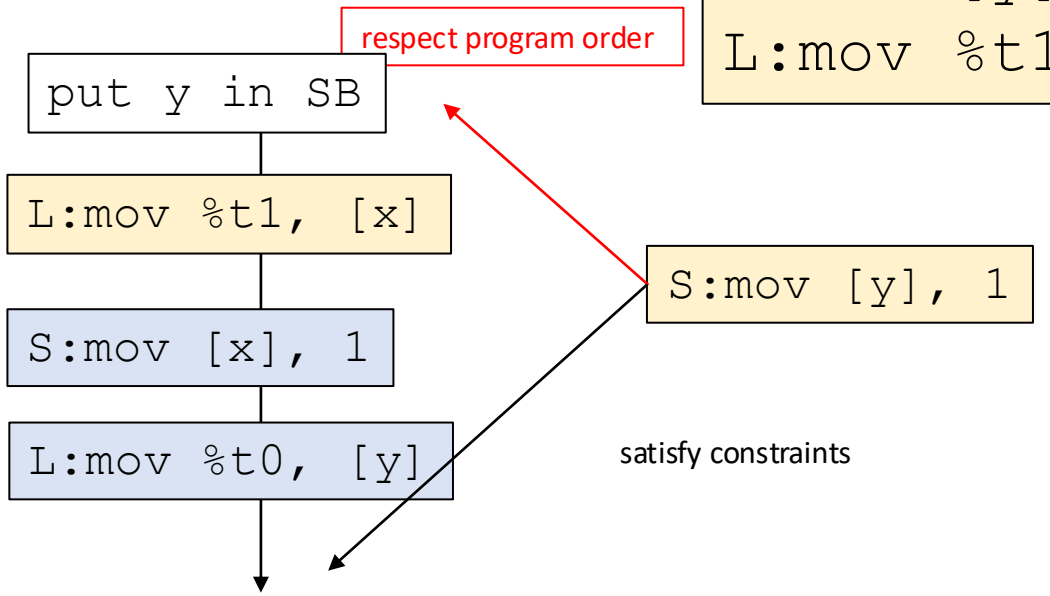
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```

we can ignore this condition!!



Let's peak under the hood here

Global timeline is when the Store operation becomes visible to other threads

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == t1 == 0`?

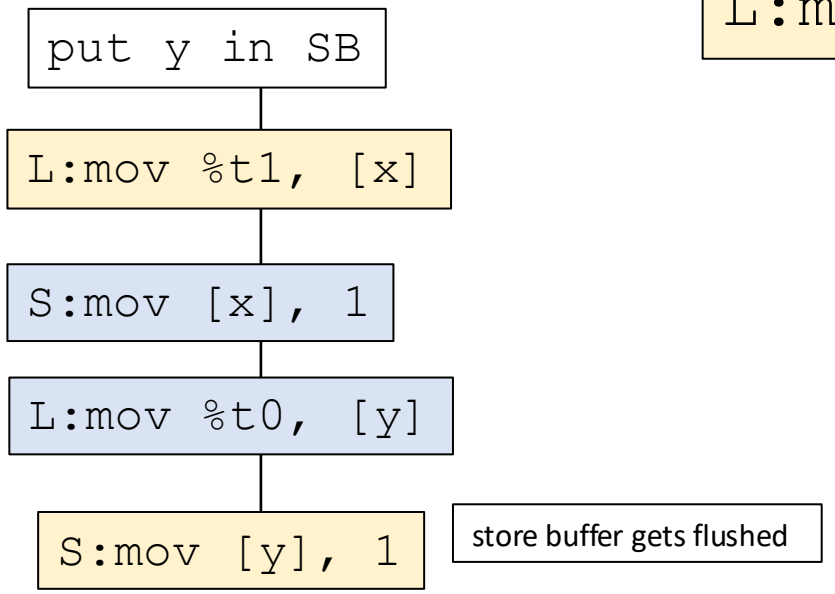
Thread 0:

```
S:mov [x], 1
L:mov %t0, [y]
```

we can ignore this condition!!

Thread 1:

```
S:mov [y], 1
L:mov %t1, [x]
```



Lets peak under the hood here

Global timeline is when the Store operation becomes visible to other threads

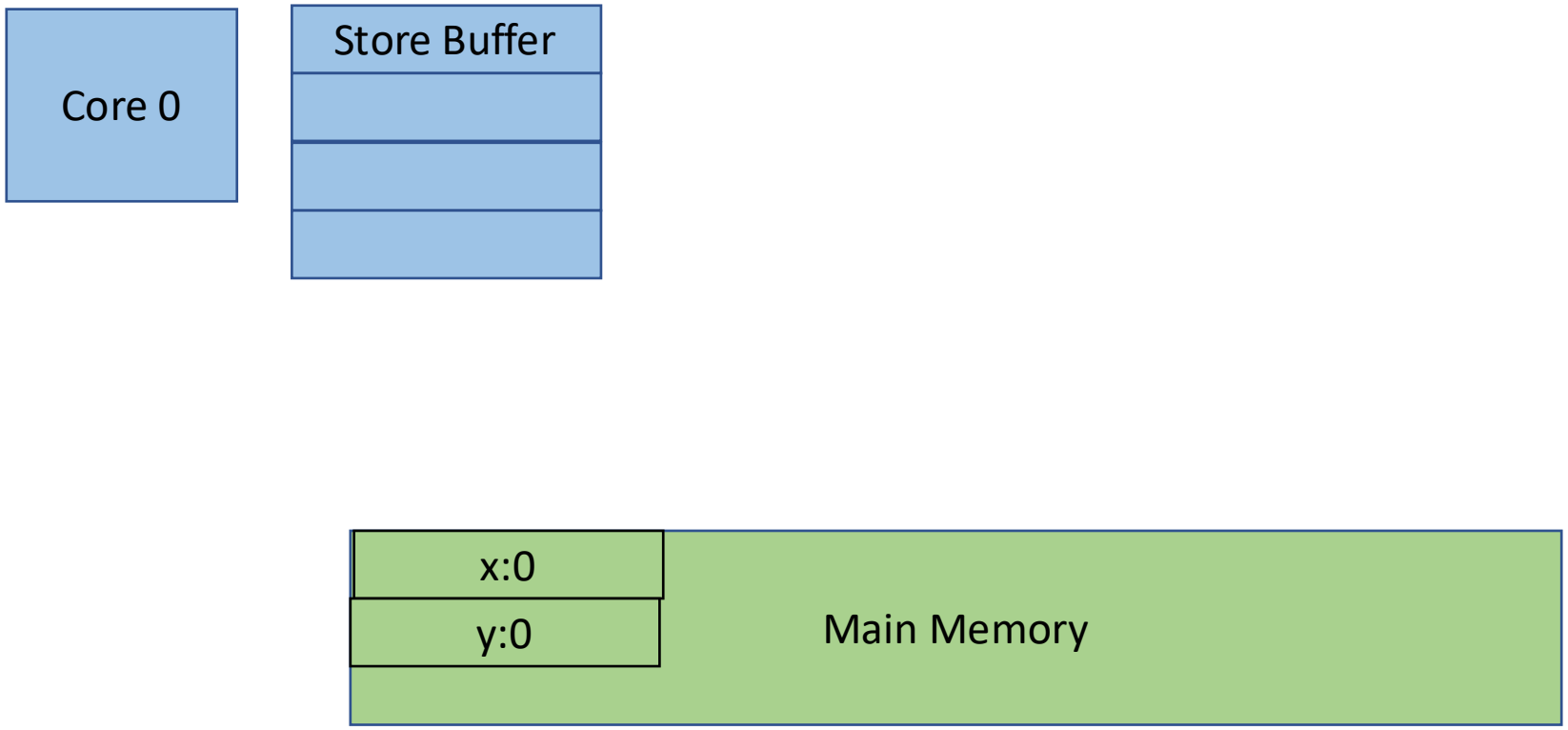
Questions

- Can stores be reordered with stores?

Thread 0:

```
mov [x], 1
```

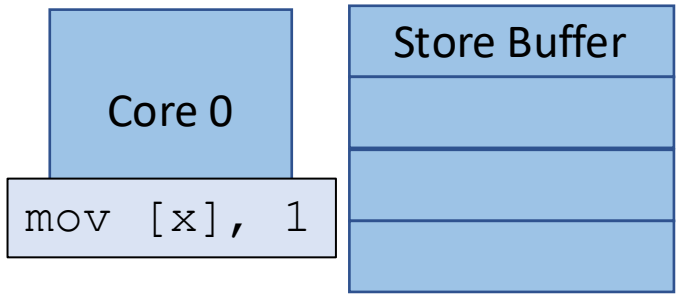
```
mov [y], 1
```

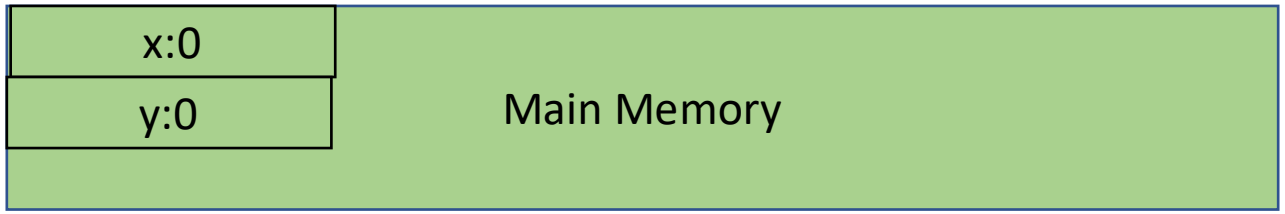


Thread 0:

```
mov [y], 1
```

execute the first instruction

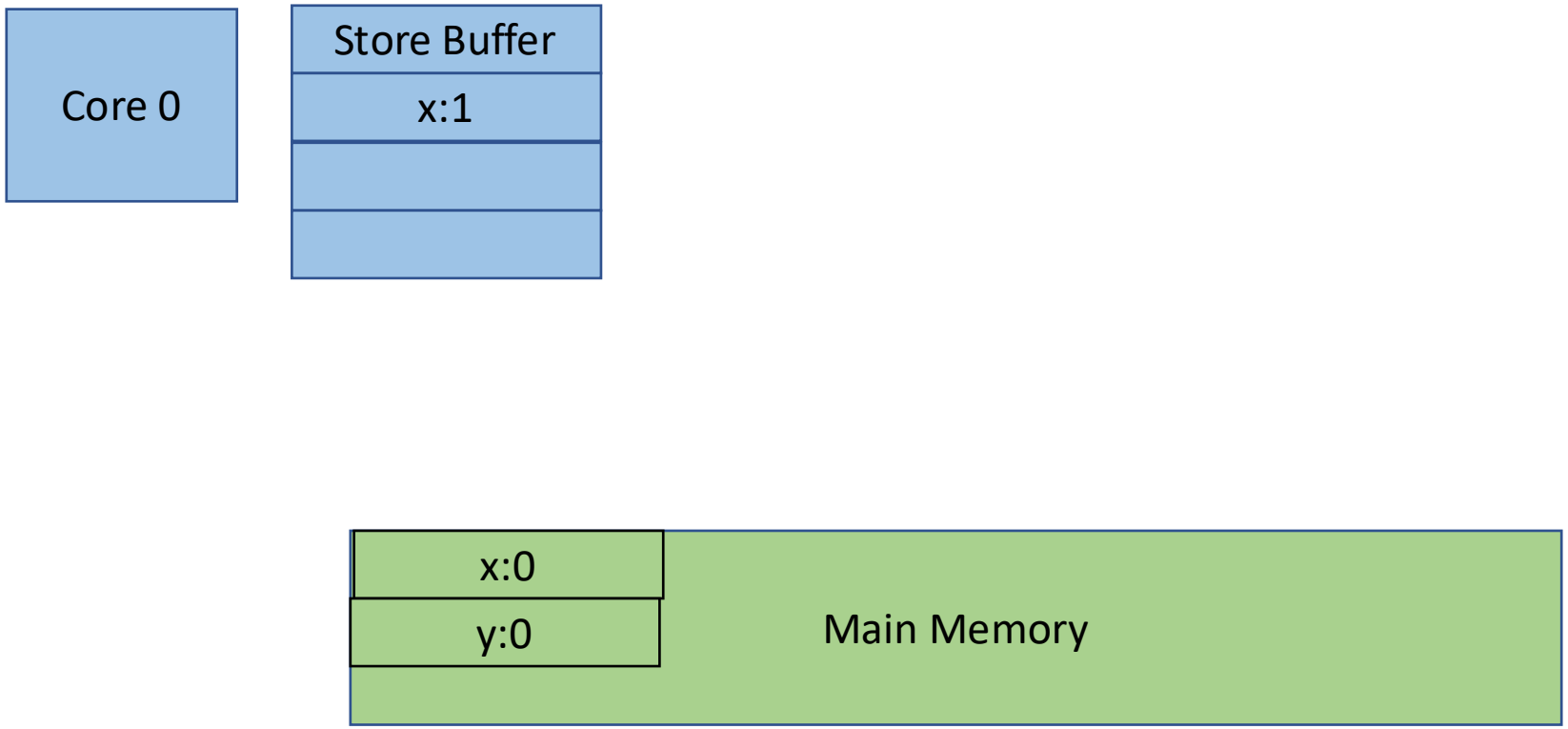




Thread 0:

```
mov [y], 1
```

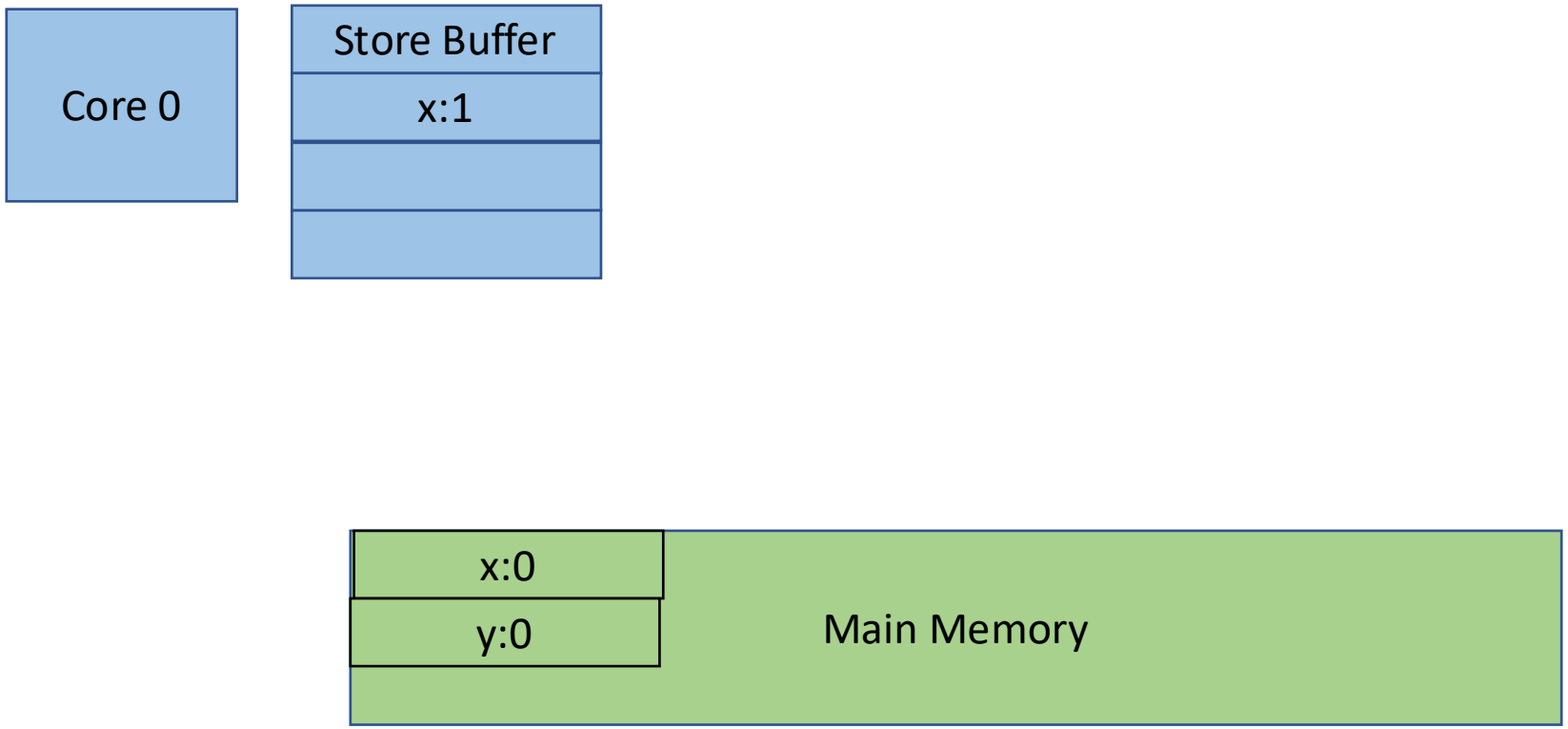
value goes into store buffer



Thread 0:

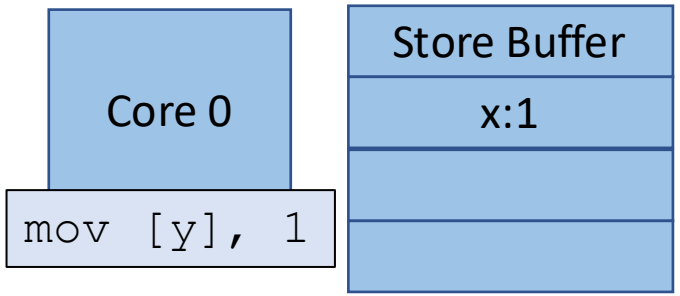
```
mov [y], 1
```

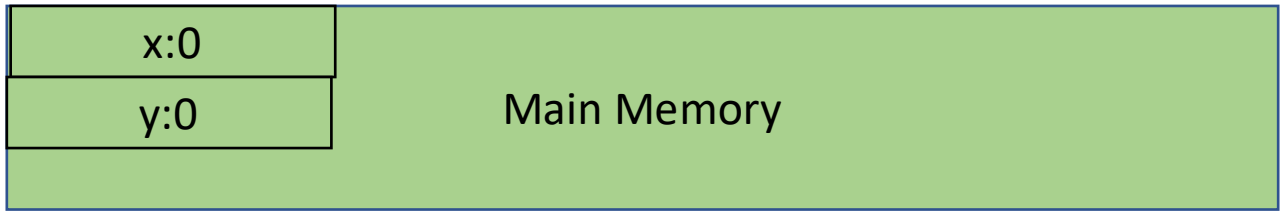
execute next instruction



Thread 0:

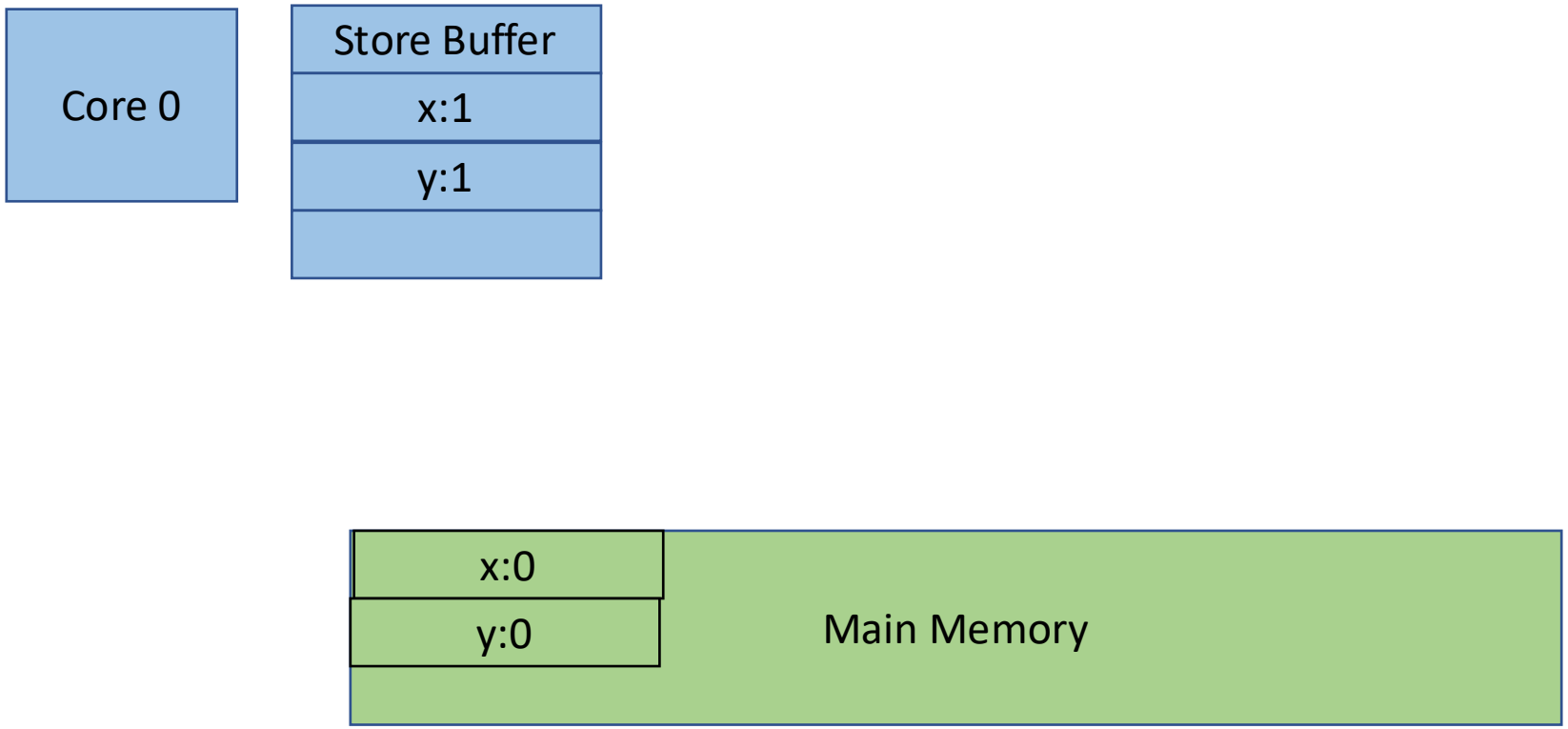
execute next instruction



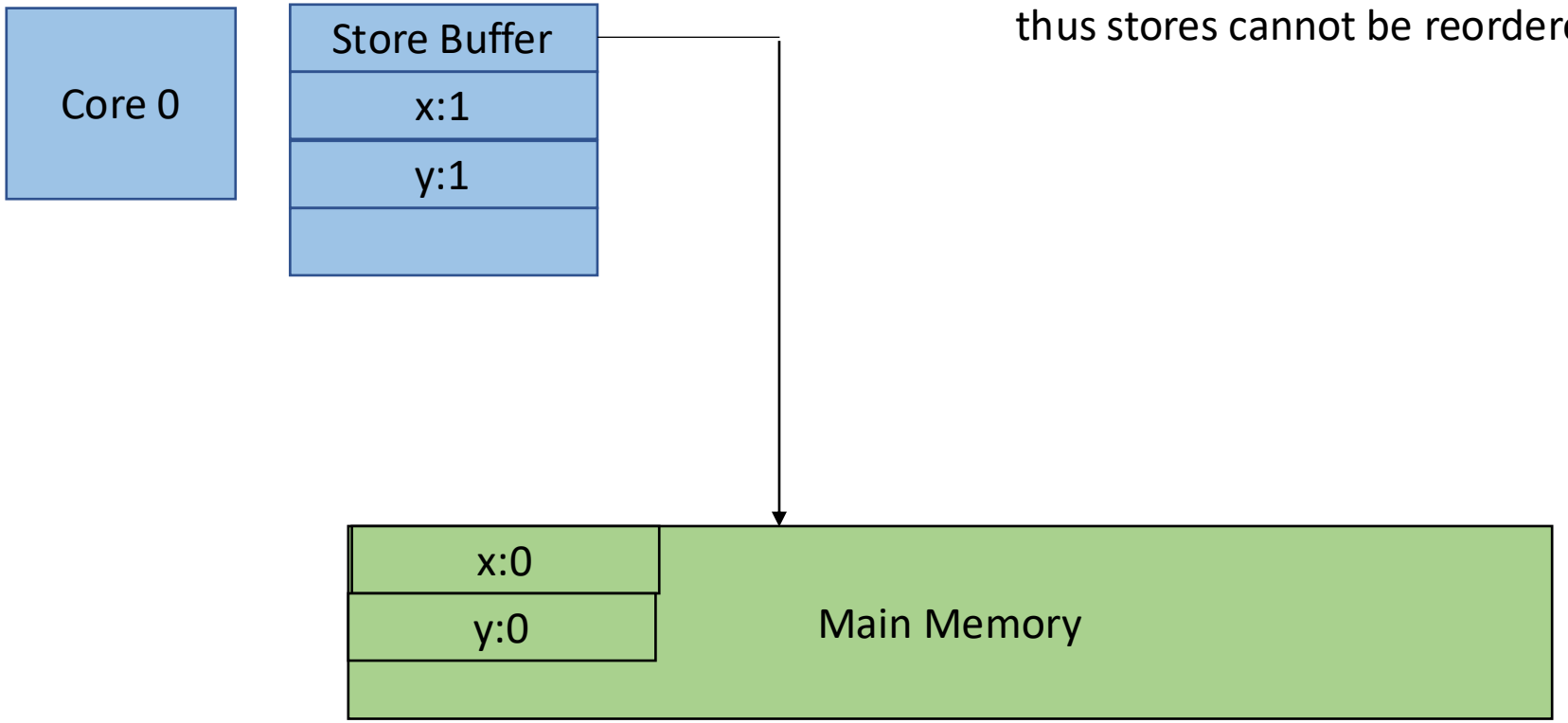


Thread 0:

value goes into the store buffer

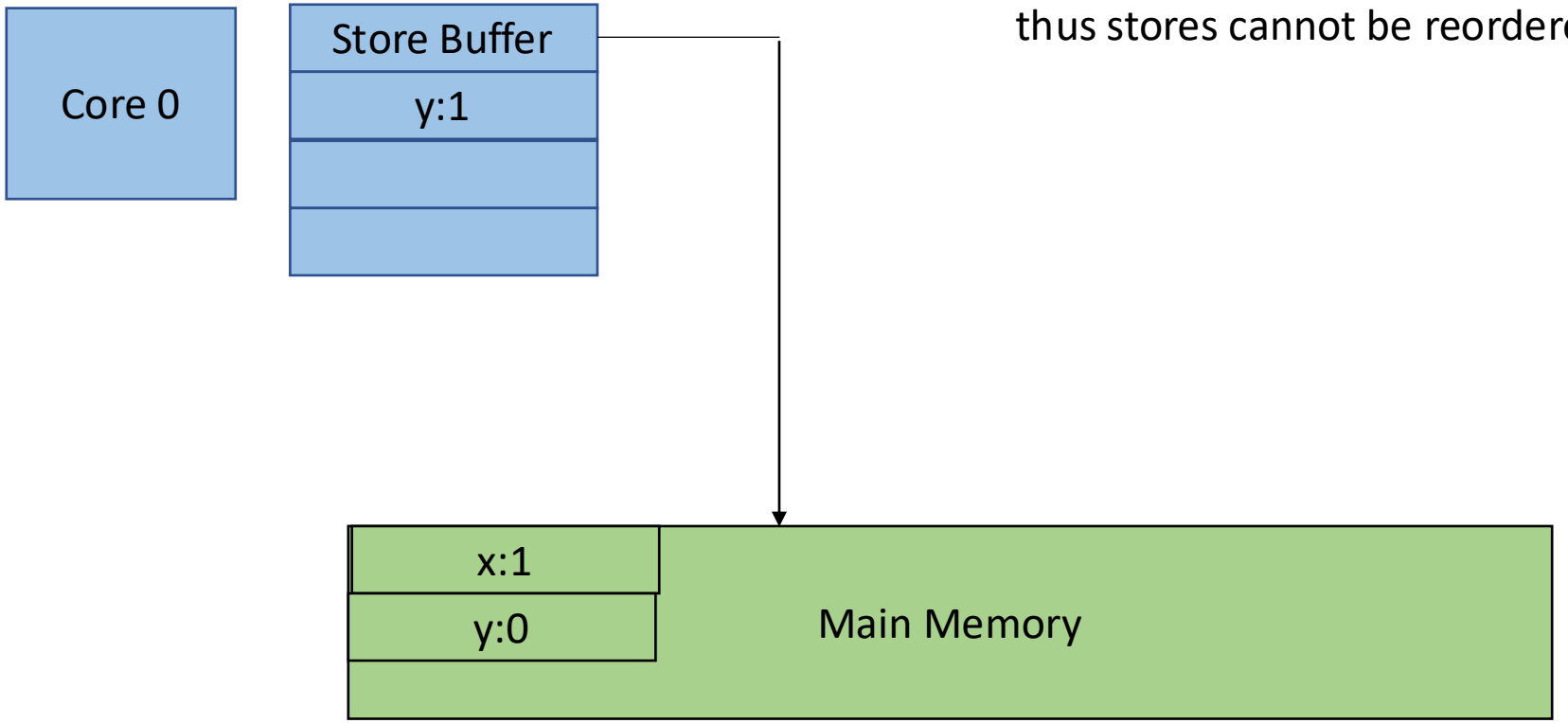


Thread 0:



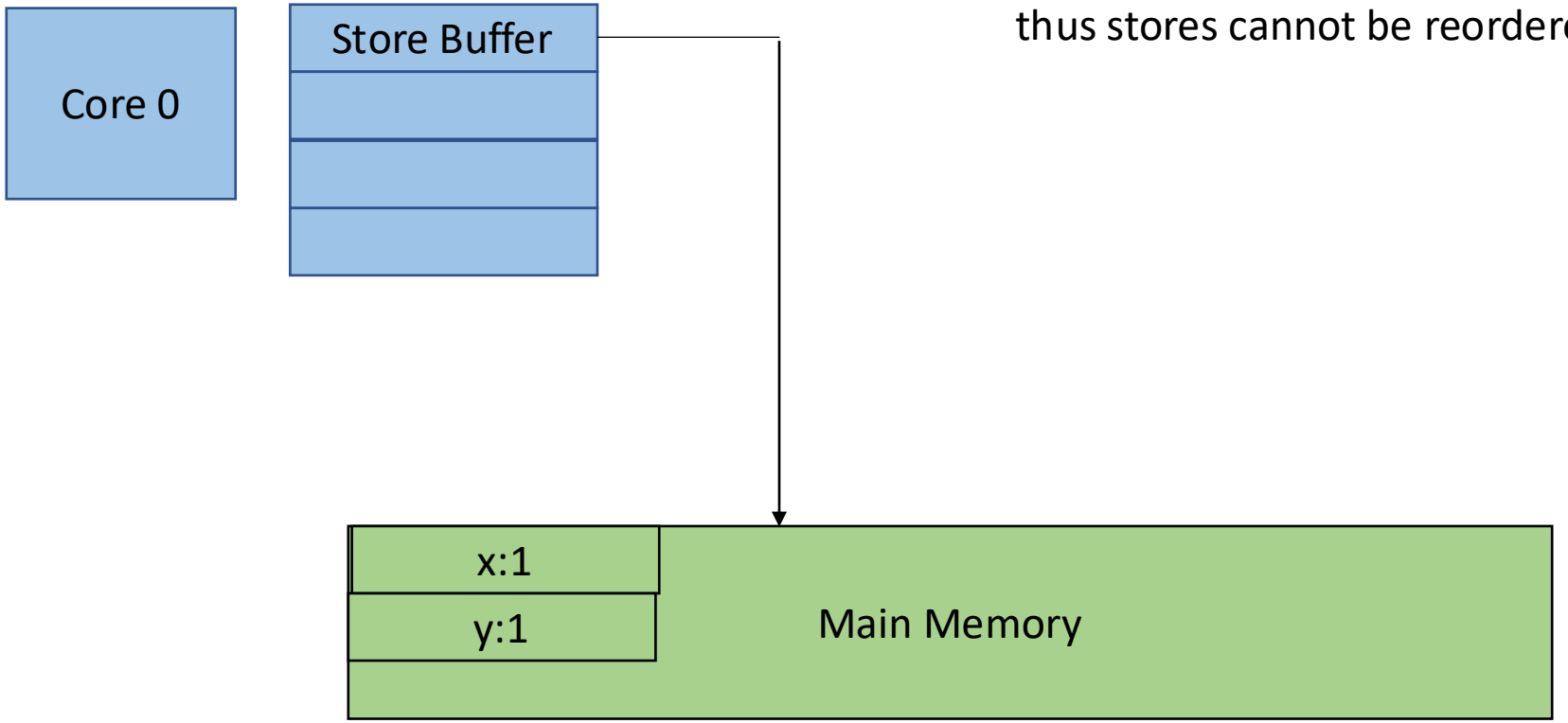
On x86, the store buffer trains in a FIFO way:
thus stores cannot be reordered

Thread 0:



On x86, the store buffer trains in a FIFO way:
thus stores cannot be reordered

Thread 0:



On x86, the store buffer trains in a FIFO way:
thus stores cannot be reordered

Questions

- Can stores be reordered with stores?

- How do we make rules about mfence?

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test

Can `t0 == t1 == 0`?

Thread 0:

```
S:mov [x], 1
mfence
L:mov %t0, [y]
```

```
S:mov [x], 1
```

```
mfence
```

```
L:mov %t0, [y]
```



Thread 1:

```
S:mov [y], 1
mfence
L:mov %t1, [x]
```

```
S:mov [y], 1
```

```
mfence
```

```
L:mov %t1, [x]
```

Rules: S(tores) followed by a L(oad) do not have to follow program order.

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

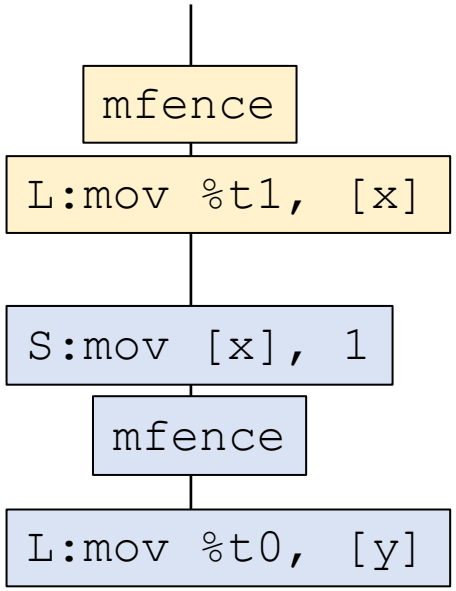
Thread 0:

```
S:mov [x], 1
mfence
L:mov %t0, [y]
```

Another test
Can `t0 == t1 == 0`?

Thread 1:

```
S:mov [y], 1
mfence
L:mov %t1, [x]
```




```
S:mov [y], 1
```

So we can't reorder this instruction at all!

Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence in program order

Rules

- Are we done?

Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence
in program order

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [x]
```

```
S:mov [x], 1
```

```
L:mov %t0, [x]
```


Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence
in program order

Global variable:

```
int x[1] = {0};
int y[1] = {0};
```

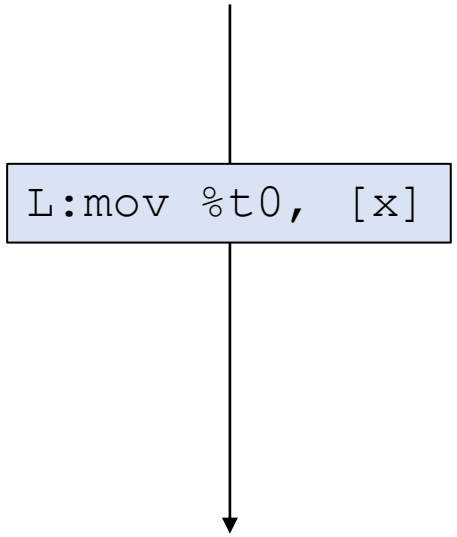
Another test
Can `t0 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [x]
```

`S:mov [x], 1`

where to put this store?



Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence
in program order

Global variable:

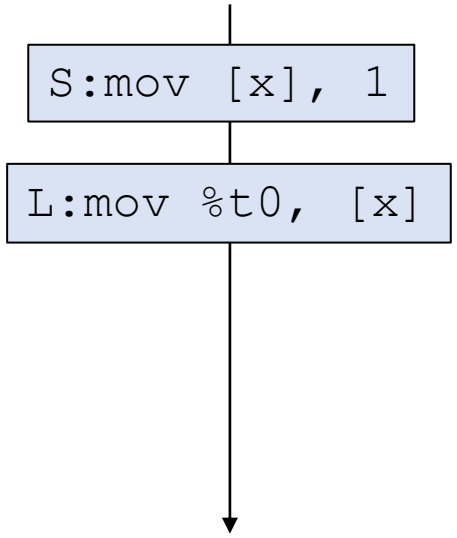
```
int x[1] = {0};
int y[1] = {0};
```

Another test
Can `t0 == 0`?

Thread 0:

```
S:mov [x], 1
L:mov %t0, [x]
```

where to put this store?



Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence in program order

S(tores) cannot be reordered past L(oads) from the same address

TSO - Total Store Order

Rules:
S(tores) followed by a L(oad)
do not have to follow program order.

S(tores) cannot be reordered past a fence
in program order

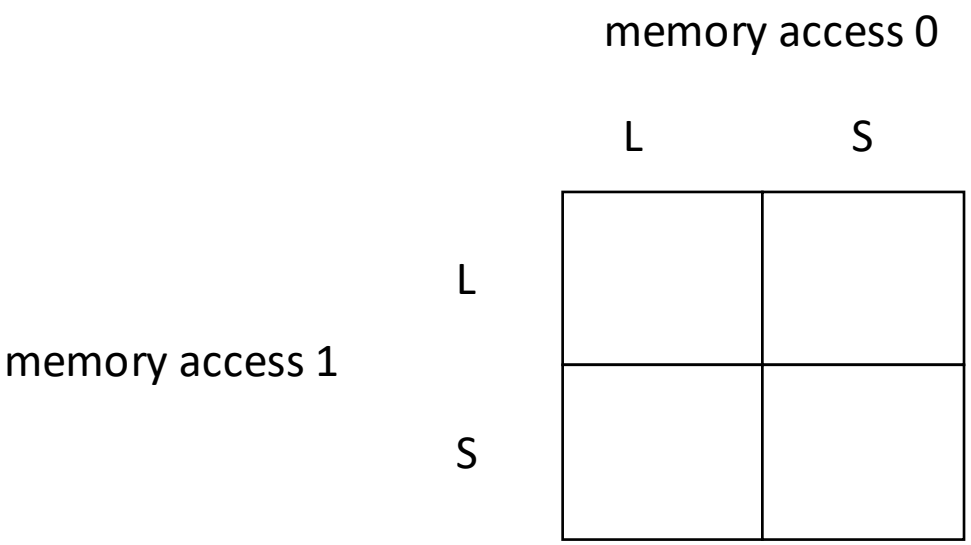
S(tores) cannot be reordered past L(oads)
from the same address

Schedule

- Memory consistency models:
  - Total store order
  - **Relaxed memory consistency**

Other memory models?

- We can specify them in terms of what reorderings are allowed



If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Other memory models?

- We can specify them in terms of what reorderings are allowed

memory access 1 \ memory access 0	L	S
L	NO	NO
S	NO	NO

Sequential Consistency

If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Other memory models?

- We can specify them in terms of what reorderings are allowed

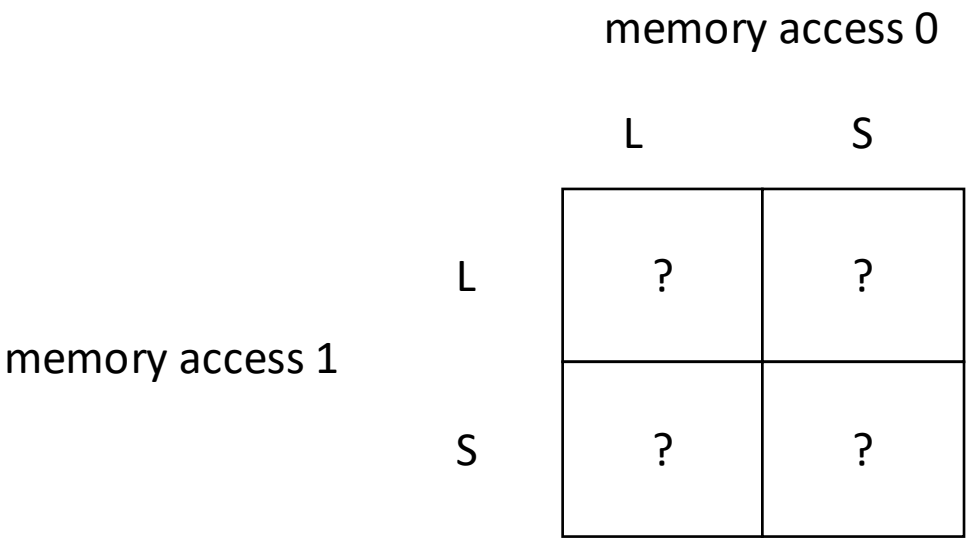
memory access 1 \ memory access 0	L	S
L	NO	Different address
S	NO	NO

TSO - total store order

If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Other memory models?

- We can specify them in terms of what reorderings are allowed



Weaker models?

If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Other memory models?

- We can specify them in terms of what reorderings are allowed

memory access 1 \ memory access 0	L	S
L	NO	Different address
S	NO	Different address

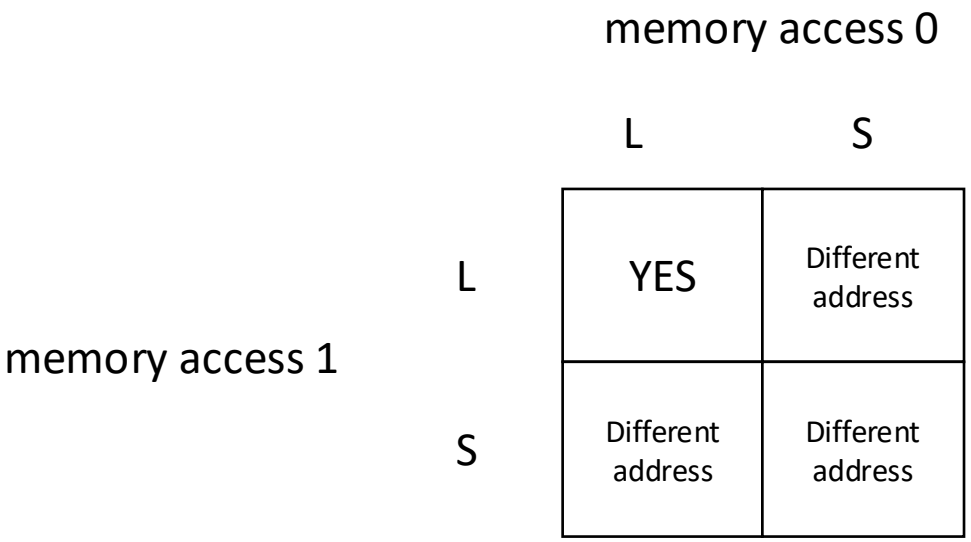
PSO - partial store order

If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Allows stores to drain from the store buffer in any order

Other memory models?

- We can specify them in terms of what reorderings are allowed



RMO - Relaxed Memory Order

If memory access 0 appears before memory access 1 in program order, can it bypass program order?

Very relaxed model!

Other memory models?

- FENCE: can always restore order using fences. Accesses cannot be reordered past fences!

memory access 1 \ memory access 0	L	S
L	NO	NO
S	NO	NO

Any Memory Model

If memory access 0 appears before memory access 1 in program order, and there is a FENCE between the two accesses, can it bypass program order?

Schedule

- Memory consistency models:
  - Total store order
  - Relaxed memory consistency
- **Some Examples**